Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales
Ingeniería en Computación

Redes de Computadoras

MONOGRAFÍA

MQTT y Protocol Buffers

Un stack de comunicación en tiempo real para IoT, robótica y mensajería con servicios en la nube

Equipo
WireGuardians

Viberti, Benjamin[1]	DNI 46224179
Espinoza Sutta, Aaron Alejandro[1]	DNI 96009173
Cleri, Juan Ignacio[1]	DNI 46452662
Pineda, Juan Ignacio[1]	DNI 45591343
Grafión, Atilio Leonel[1]	DNI 43940195
Badenes, Tomás[1]	DNI 44785038
Oviedo, Ignacio Nicolás[1]	DNI 44169239

Docente
Oliva Cuneo, Facundo Nicolas

[1]{b.viberti,aaron.espinoza_4500,ignacio.cleri,juan.ignacio.pineda,atilio.grafion,tomasbadenes,
ignacio.oviedo.239}@mi.unc.edu.ar

9 de noviembre de 2026

Índice

Introducción **4**
1. Objetivos 5
2. Alcance 5
   2.1. Tecnologías estudiadas 5
   2.2. Casos de uso 6
   2.3. Mapeo tecnológico 6
   2.4. Parte experimental 6

Marco teórico **8**
1. Modelo publicador/suscriptor 9
2. MQTT 11
3. Protocol Buffers 13
   3.1. Esquema y generación de código 13
   3.2. Codificación binaria 13
   3.3. Compatibilidad entre versiones 14
   3.4. Limitaciones 14
4. Stack MQTT y Protocol Buffers 14
   4.1. Protocol Buffers como carga útil 14
   4.2. Asociación entre tópico y esquema 15
   4.3. Esquema compartido 16
   4.4. Sparkplug B 16
   4.5. Síntesis 16
5. Comparación con alternativas 17
   5.1. MQTT, HTTP y CoAP 17
   5.2. Protocol Buffers y JSON 20

Desarrollo **23**
1. MQTT sobre TCP y TLS 23
   1.1. Conexión persistente y keep-alive 23
   1.2. Cifrado con TLS 24
   1.3. QoS MQTT sobre la confiabilidad de TCP 25
2. Capa de red 26
   2.1. NAT y firewalls 26
   2.2. Ruteo interno 27
   2.3. Ruteo externo 29
   2.4. Convergencia y conexiones de larga duración 29
3. Ruteo por tópicos y ruteo IP 31
   3.1. Dirección IP y nombre de tópico 32
   3.2. Tablas de ruteo y tablas de suscripciones 32
   3.3. Comodines y prefijos 33
   3.4. Agregación y diseño del espacio de tópicos 34
   3.5. Entrega a uno, a todos y a uno de un grupo 35
   3.6. Límites del ruteo por tópicos 35
4. Casos de uso 36
   4.1. Mensajería instantánea 37
   4.2. Robótica 37
   4.3. Internet de las Cosas industrial 40
   4.4. Internet de las Cosas sobre redes LoRa 41

4.5. Ventajas y limitaciones observadas 43
5. Diseño experimental 44
5.1. Plataforma común 44
5.2. Experimento 1: MQTT contra HTTPS para telemetría 45
5.3. Experimento 2: sincronización de osciladores acoplados 59
6. Mapeo tecnológico 77
6.1. Nodos 77
6.2. Conectividad 77
6.3. Broker 78
6.4. Costo del sistema 78
6.5. Escalabilidad 79
Conclusiones 81
Bibliografía 85

Introducción

Gran parte de la Internet comercial se apoya en el modelo de solicitud y respuesta de HTTP: un cliente consulta a un servidor cuando necesita un dato y recibe la respuesta, que suele venir codificada en formatos de texto como HTML, JSON o XML. Este modelo está pensado para intercambios puntuales, que el cliente inicia a demanda, y no para flujos de datos continuos ni para que el servidor tome la iniciativa.

Un número creciente de sistemas no encaja en ese modelo. Los dispositivos del Internet de las Cosas (IoT, *Internet of Things*), las flotas de robots y las aplicaciones de mensajería móvil generan flujos continuos de telemetría, eventos y comandos a alta frecuencia y en ambos sentidos, desde equipos de recursos acotados que operan sobre redes celulares o inalámbricas inestables y detrás de firewalls fuera del control de quien administra el servidor. Esta combinación plantea dos limitaciones de fondo para la ingeniería de redes: el costo, en ancho de banda y energía, de consultar periódicamente y de codificar los datos en formatos de texto; y la imposibilidad de que un servidor inicie comunicaciones hacia dispositivos que no admiten conexiones entrantes.

Frente a este panorama, muchos de estos sistemas han recurrido a la mensajería de tipo publicador/suscriptor, que desacopla a los participantes mediante un intermediario. En este campo se inscribe la combinación de MQTT y Protocol Buffers, un stack pensado para la comunicación en tiempo real con ancho de banda limitado. MQTT es un protocolo liviano de transporte de mensajes, organizado en torno a un broker, concebido para entornos restringidos (como los de comunicación máquina a máquina y de Internet de las Cosas) en los que se necesita una huella de código reducida o el ancho de banda es escaso. Protocol Buffers es un mecanismo, independiente del lenguaje y de la plataforma, para serializar datos estructurados y tipados en un formato más compacto y rápido que los textuales, a partir de un esquema compartido.

Su adopción en sistemas reales es amplia y heterogénea, como se analiza en la Sección 4. En la mensajería a gran escala, Facebook recurrió a MQTT en Messenger para mantener una conexión persistente con los teléfonos sin agotar su batería. En la robótica móvil, InOrbit, una empresa que ofrece una plataforma para supervisar y operar flotas de robots, comunica los robots con la nube mediante MQTT y mensajes definidos con Protocol Buffers, una tecnología que, según la propia empresa, gestiona miles de robots en producción y que publicó como código abierto en el proyecto OpenRobOps [1]. En la automatización industrial, la especificación Eclipse Sparkplug define precisamente esta combinación para comunicar los equipos de una planta, y en las redes de radio de largo alcance y bajo consumo, proyectos como Meshtastic transmiten mensajes de Protocol Buffers que llegan a Internet a través de un broker MQTT. Otros sistemas, como los estándares de interoperabilidad de robots VDA 5050 e ISO 21423, combinan MQTT con JSON, lo que muestra que las dos piezas del stack pueden elegirse de forma independiente.

Un rasgo de este esquema es especialmente relevante desde el encaminamiento: el dispositivo abre una conexión persistente hacia el broker alojado en la nube, cuya dirección suele estar habilitada en el firewall, y por esa misma conexión el broker le entrega los mensajes, sin necesidad de conexiones entrantes. El principio rector del stack es, entonces, trasladar la complejidad desde la red hacia el protocolo de aplicación: ningún participante necesita conocer la ubicación de los demás, sino solo la del broker, y el contrato sobre el formato de los datos se fija una única vez en un esquema distribuido entre los clientes. De este modo, la comunicación en tiempo real entre miles de entidades heterogéneas resulta viable sobre redes que no fueron concebidas para ese fin.

1. Objetivos

Se desea realizar un análisis riguroso del stack de comunicación formado por MQTT y Protocol Buffers, como alternativa para el intercambio de datos en tiempo real, a alta frecuencia y con ancho de banda limitado, en aplicaciones de Internet de las Cosas, robótica y mensajería. El estudio sigue el recorrido de un mensaje desde el dispositivo que lo genera hasta el broker que lo distribuye, y evalúa cómo cada capa de la arquitectura contribuye a que ese recorrido sea eficiente y confiable.

Más que catalogar las prestaciones de ambas tecnologías, el trabajo busca explicar por qué resultan adecuadas para el problema planteado: sostener una comunicación continua sobre redes que el diseñador no controla, con enlaces inestables, firewalls de terceros y trayectos que atraviesan múltiples sistemas autónomos. Con ese fin, el análisis se articula en torno a dos ejes centrales de la materia, que pertenecen a capas distintas: la capa de aplicación (capa 7), donde operan MQTT y Protocol Buffers, y la capa de red (capa 3), donde el ruteo determina por qué camino llegan los mensajes al broker y cómo se restablece ese camino cuando la red cambia.

Para alcanzar este propósito se plantean los siguientes objetivos específicos:

- Presentar el modelo publicador/suscriptor y describir cómo lo implementa MQTT (broker, tópicos, niveles de calidad de servicio (QoS) y sesiones), junto con el mecanismo de serialización de Protocol Buffers, y compararlos con alternativas como HTTP, CoAP y JSON.
- Analizar el funcionamiento de MQTT sobre la capa de transporte (TCP, keep-alive y TLS) y distinguir las garantías de entrega de sus niveles de calidad de servicio de la confiabilidad que ya ofrece TCP.
- Analizar el recorrido de los mensajes en la capa de red (capa 3): la traducción de direcciones de red (NAT) y los firewalls, y el efecto de la convergencia de los protocolos de ruteo interno (OSPF, con RIP y EIGRP como comparación) y externo (BGP) sobre las conexiones TCP de larga duración en las que se apoya MQTT.
- Contrastar el ruteo por destino IP de la capa de red con el ruteo por tópicos que realiza el broker en la capa de aplicación.
- Examinar casos de uso reales en robótica, mensajería e Internet de las Cosas, como los de InOrbit, para determinar las ventajas y limitaciones del stack en entornos de alta frecuencia y recursos acotados.
- Medir experimentalmente, sobre nodos emulados, el costo en bytes y latencia de MQTT y Protocol Buffers frente a HTTPS (HTTP sobre TLS) y JSON, y su comportamiento ante pérdidas, latencia y cortes de conexión.

2. Alcance

El trabajo se centra en el stack formado por MQTT y Protocol Buffers, con MQTT sobre TCP y TLS (WebSocket se menciona solo como alternativa para redes que únicamente dejan pasar el tráfico web), y en el recorrido de un mensaje desde el dispositivo que lo genera hasta el broker que lo distribuye. A continuación se precisa qué aspectos abarca el análisis y cuáles quedan fuera de él.

2.1. Tecnologías estudiadas

En la capa de aplicación, el estudio considera MQTT en su versión 5.0 y Protocol Buffers en su sintaxis proto3. Del protocolo MQTT se analizan el modelo de broker, los tópicos y sus comodines, los niveles de calidad de servicio, las sesiones, los mensajes retenidos y el mensaje de

última voluntad (*last will*). De Protocol Buffers se analizan el esquema, la codificación binaria y la compatibilidad entre versiones. Como puntos de comparación se toman HTTP (y HTTPS) y CoAP como protocolos de transporte de mensajes, y JSON como formato de serialización.

En la capa de transporte, el análisis se limita a TCP y TLS en la medida en que afectan a MQTT, conceptos como la conexión persistente, el keep-alive, el costo del establecimiento de la sesión segura y la relación entre la confiabilidad de TCP y los niveles de calidad de servicio de MQTT. La seguridad se aborda únicamente desde el cifrado del canal y la autenticación de los clientes, sin un análisis integral de amenazas.

En la capa de red, se estudian NAT y los firewalls, y el ruteo interno y externo por el que los mensajes llegan al broker. Para el ruteo interno se toma OSPF como protocolo principal y se usan RIP y EIGRP como comparación. Para el ruteo externo se considera BGP. Estos protocolos se analizan en cuanto a su convergencia y su efecto sobre las conexiones TCP de larga duración, y no en cuanto a su configuración en equipos concretos.

2.2. Casos de uso

Los casos de uso abarcan la mensajería (Facebook Messenger), la robótica móvil (InOrbit, OpenRobOps y los estándares VDA 5050 e ISO 21423), la automatización industrial (Eclipse Sparkplug) y las redes de radio LoRa (Meshtastic y ChirpStack). Se analizan a partir de la documentación, las especificaciones y el código fuente que publican sus responsables, con foco en cómo emplean MQTT y Protocol Buffers y no en los detalles de cada dominio. Las redes LoRa, excluidas de la parte experimental, se consideran únicamente en este análisis, en el punto en que sus datos ingresan a una red IP.

2.3. Mapeo tecnológico

El trabajo incluye además un mapeo tecnológico, desarrollado en la Sección 6, con los equipos reales con los que se implementaría el escenario de telemetría del primer experimento (Sección 5.2) en lugar de los equipos emulados. Se relevan los nodos, el broker, ya sea en un servidor propio o como servicio administrado, y la conectividad, y para cada uno se indica el modelo, el proveedor y un precio de referencia con su fecha y moneda. El objetivo es estimar la viabilidad de una solución considerando la calidad técnica, el plazo y el presupuesto, y no proponer la alternativa de máxima prestación.

2.4. Parte experimental

Los experimentos se realizan sobre placas ESP32 emuladas con QEMU, un emulador que ejecuta en una computadora el código de la placa, que se comunican con un broker Mosquitto en la misma computadora, y contrastan con mediciones propias lo que el marco teórico y el desarrollo plantean de forma cualitativa.

El primero, en un escenario de telemetría, compara MQTT 3.1.1 (QoS 0 y QoS 1) con HTTP (con y sin reutilización de la conexión), con carga útil en JSON y en Protocol Buffers y con y sin TLS, y mide por separado el costo en bytes y en latencia. Se le aplican pérdidas y latencia de forma artificial, pero no cortes de conexión.

El segundo no compara protocolos. Usa únicamente MQTT 5.0 con Protocol Buffers, sin TLS, y emplea la sincronización de osciladores acoplados como aplicación en la que cada nodo necesita el dato de todos los demás. Allí se ensayan los mecanismos del stack (QoS, keep-alive, última voluntad, alias de tópico y compatibilidad del esquema), su comportamiento ante pérdidas,

cortes de conexión y caídas del broker, que se introducen de forma artificial, y la carga del broker con 5, 10 y 20 nodos.

La emulación impone limitaciones que acotan el alcance de los resultados. No reproduce el comportamiento de una radio real, por lo que no se mide el consumo de energía de los dispositivos y no se incluyen enlaces inalámbricos restringidos como LoRa, NB-IoT o satelitales. Tampoco reproduce los tiempos de procesamiento de un microcontrolador físico. Los escenarios de ruteo y convergencia de la capa de red no se reproducen experimentalmente: se analizan en el desarrollo, y su efecto sobre la conexión se representa con cortes de duración controlada. Por último, no se evalúan la escalabilidad del broker con miles de clientes ni las configuraciones en clúster.

Marco teórico

Las redes de computadoras existen para hacer posibles las aplicaciones de red, es decir, programas que se ejecutan en distintos sistemas terminales y que se comunican entre sí a través de la red. La lógica de estas aplicaciones reside únicamente en los sistemas terminales, mientras que los nodos intermedios operan en la capa de red e inferiores. En consecuencia, quien desarrolla una aplicación no modifica la red, sino que utiliza los servicios que esta ofrece y define cómo se organiza la comunicación entre los extremos. MQTT y Protocol Buffers se ubican en este plano.

La forma predominante de organizar la comunicación a nivel de aplicación es el modelo cliente-servidor [2]. En este modelo, un servidor permanece activo en una dirección conocida a la espera de solicitudes, y un cliente lo contacta cuando requiere un servicio, envía una solicitud y aguarda la respuesta. La Web constituye el ejemplo característico, en el que el navegador solicita un recurso y el servidor lo devuelve. Se trata de un modelo orientado a la demanda, en el que la información se transmite únicamente cuando es solicitada y el intercambio es iniciado siempre por la parte que la requiere.

Este modelo presenta limitaciones para los sistemas considerados en este trabajo. En ellos, los dispositivos generan datos de manera continua y en instantes que no pueden anticiparse, y deben a su vez recibir comandos originados en la nube. Para que el servidor transmita un comando por iniciativa propia, debería actuar como cliente del dispositivo, lo que requiere que este acepte conexiones entrantes. Esta condición rara vez se cumple cuando el dispositivo se encuentra detrás de un NAT o de un firewall administrado por terceros. La alternativa que ofrece el propio modelo es la consulta periódica por parte del dispositivo, cuyo costo en tráfico y en latencia aumenta con la frecuencia de las consultas.

La capa de transporte ofrece una base para superar esta limitación. TCP establece una conexión persistente *full-duplex*, que permite a ambos extremos transmitir datos en cualquier momento con entrega fiable y ordenada. Si es el dispositivo quien abre la conexión hacia un servidor de dirección conocida, no necesita aceptar conexiones entrantes y conserva el rol de cliente. En ese caso, una vez establecida la conexión, el servidor también puede enviarle mensajes al cliente sin esperar que este realice una solicitud, lo que vuelve innecesaria la consulta periódica.

La conexión TCP, sin embargo, solo garantiza la entrega de una secuencia de bytes entre dos procesos. Qué mensajes se intercambian, con qué formato y en qué momento lo determina el protocolo de la capa de aplicación, que especifica los tipos de mensajes, su sintaxis y su semántica, así como las reglas que indican cuándo y cómo se envía y se responde cada uno de ellos [3].

En los sistemas estudiados, el diseño de ese protocolo está condicionado en primer lugar por la forma en que circula la información. Un mismo dato suele interesar a varios consumidores y cada consumidor recibe datos de muchos dispositivos, por lo que resulta conveniente que productores y consumidores puedan comunicarse sin conocerse entre sí ni estar conectados simultáneamente, que es la propuesta del modelo publicador/suscriptor implementado por MQTT. A esto se suma la representación de los datos dentro de cada mensaje, que debe ser interpretada de manera unívoca por equipos heterogéneos y ocupar poco ancho de banda en enlaces que suelen ser limitados, problema del que se ocupa Protocol Buffers.

1. Modelo publicador/suscriptor

El modelo publicador/suscriptor (*publish/subscribe*) ofrece precisamente la posibilidad de que productores y consumidores de información se comuniquen sin conocerse entre sí, lo que lo hace adecuado para los sistemas descritos. En este esquema de interacción, los suscriptores expresan su interés en un evento o en un patrón de eventos, y son notificados de forma asíncrona de todo evento generado por un publicador que coincida con ese interés [4]. El modelo básico se apoya en un servicio de notificación de eventos, una abstracción que almacena las suscripciones y entrega cada evento publicado a todos los suscriptores interesados. Las suscripciones permanecen en ese servicio y son invisibles a los publicadores.

De esta manera, una de las características principales del modelo y común a todas sus variaciones es el desacoplamiento total de las entidades que se comunican, que puede descomponerse en tres dimensiones [4]:

- El **desacoplamiento en el espacio** (*space decoupling*) implica que las partes no necesitan conocerse, ya que los publicadores no mantienen referencias a los suscriptores ni saben cuántos participan de la interacción, y lo mismo ocurre en sentido inverso.
- El **desacoplamiento en el tiempo** implica que las partes no necesitan participar de la interacción al mismo tiempo, por lo que un publicador puede publicar eventos mientras un suscriptor está desconectado, y un suscriptor puede recibir la notificación de un evento cuyo publicador ya se desconectó.
- El **desacoplamiento de sincronización** implica que los publicadores no se bloquean al producir eventos y que los suscriptores son notificados mientras realizan otras actividades, fuera del flujo principal de control de ambos.

Este desacoplamiento reduce la coordinación necesaria entre los dispositivos participantes de la comunicación y favorece la escalabilidad en entornos asíncronos por naturaleza.

Estas tres dimensiones permiten contrastar el modelo con el cliente-servidor descrito al comienzo del capítulo. En el caso del paradigma cliente-servidor, el cliente debe conocer la dirección del servidor, ambos deben estar activos durante el intercambio y el cliente queda a la espera de la respuesta, por lo que la interacción está acoplada en el espacio, en el tiempo y, del lado del cliente, en la sincronización. La misma comparación puede extenderse a otros paradigmas de comunicación distribuida, como resume la Tabla 1. La invocación remota de procedimientos (RPC) presenta el mismo acoplamiento que el modelo cliente-servidor, mientras que sus variantes asíncronas y el patrón *observer* eliminan únicamente el acoplamiento de sincronización. Los espacios de tuplas y las colas de mensajes desacoplan a productores y consumidores en el espacio y en el tiempo, pero obligan a los consumidores a extraer los datos de forma síncrona. El modelo publicador/suscriptor es el único que reúne las tres propiedades.

Tabla 1: Desacoplamiento que ofrece cada paradigma de comunicación [4, p. 121].

Paradigma	**Espacio**	**Tiempo**	**Sincronización**
Pasaje de mensajes	No	No	Solo el productor
RPC/RMI	No	No	Solo el productor
RPC/RMI asíncrona	No	No	Sí
RPC/RMI con futuros	No	No	Sí
Notificaciones (*observer*)	No	No	Sí
Espacios de tuplas	Sí	Sí	Solo el productor
Colas de mensajes (*pull*)	Sí	Sí	Solo el productor
Publicador/suscriptor	Sí	Sí	Sí

Las variantes del modelo publicador/suscriptor se distinguen por la forma en que los suscriptores especifican los eventos de interés. En el esquema basado en tópicos, el más antiguo, los eventos se publican bajo nombres que los sistemas modernos organizan en jerarquías, y la sintaxis de comodines permite suscribirse a todos los tópicos de la jerarquía que coincidan con el patrón dado. Este esquema puede implementarse de manera muy eficiente, aunque puede resultar limitado. El esquema basado en contenido clasifica los eventos según sus propias propiedades, mediante filtros que define el suscriptor, y ofrece mayor expresividad a costa de protocolos más complejos y de un mayor costo de ejecución. El esquema basado en tipos clasifica los eventos según su tipo, lo que permite verificar en tiempo de compilación que productores y consumidores coinciden en la estructura de los datos. MQTT, descrito en la Sección 2, adopta el esquema basado en tópicos.

La implementación del servicio de eventos admite a su vez distintas arquitecturas. En una arquitectura centralizada, los productores envían los mensajes a una entidad que los almacena y los reenvía a los consumidores. En una arquitectura distribuida no existe tal entidad, y los mecanismos de almacenamiento y reenvío residen en los propios procesos de productores y consumidores, mientras que un enfoque intermedio implementa el servicio como una red de servidores dedicados. En la arquitectura centralizada, que es la que adopta MQTT, la entidad central se denomina *broker*. Este suele comunicarse con publicadores y suscriptores mediante canales punto a punto confiables y conservar copias de los eventos en almacenamiento estable, lo que le permite ofrecer persistencia y entrega garantizada. Como contraparte, el uso de un broker agrega un salto a cada mensaje y concentra la carga de todos los participantes en un punto de falla que tiene el potencial de demorar la entrega de los eventos [4].

Sobre una red TCP/IP, la arquitectura centralizada se apoya en interacciones cliente-servidor. Tanto los publicadores como los suscriptores establecen la conexión hacia el broker, por lo que todos ellos actúan como clientes, y el broker es el único participante que debe tener una dirección conocida y aceptar conexiones entrantes. Su función es la de un sistema intermedio que mantiene conexiones TCP con cada extremo y retransmite los mensajes entre ellas, de manera análoga a los intermediarios definidos para HTTP [5]. La diferencia con el modelo cliente-servidor aparece una vez registrada la suscripción, ya que el broker entrega cada evento por la conexión que el suscriptor estableció previamente, sin requerir de una solicitud. Esta capacidad de entregar datos por iniciativa del otro extremo es común a cualquier canal bidireccional persistente, como una conexión WebSocket, pero lo que distingue al modelo publicador/suscriptor es que, sobre esos canales, el broker determina a qué suscriptores corresponde cada evento, de modo que publicadores y suscriptores permanecen desacoplados aun cuando cada uno mantiene una conexión punto a punto con el broker.

2. MQTT

MQTT es un protocolo de transporte de mensajes de tipo publicador/suscriptor, basado en una arquitectura cliente-servidor y diseñado para ser liviano, abierto y fácil de implementar [6]. La primera versión estandarizada por OASIS, el consorcio de normalización que publica la especificación, fue la 3.1.1, adoptada también como norma ISO/IEC 20922, y la versión 5.0, en la que se centra este trabajo, incorpora numerosas funcionalidades nuevas sin alterar el núcleo del protocolo. MQTT adopta la arquitectura centralizada descrita en la sección anterior denominando *broker* al servidor y cliente a todo programa o dispositivo que abre una conexión hacia él para publicar mensajes o suscribirse a ellos, y deja a cada implementación la elección de los mecanismos de autenticación, autorización y cifrado. MQTT delega en la capa de transporte la entrega ordenada y sin pérdidas de los datos, por lo que funciona sobre TCP, TLS o WebSocket, pero no directamente sobre UDP, el protocolo de transporte sin conexión de Internet.

Cada mensaje de aplicación lleva un nombre de tópico, el cual consiste en una cadena de texto que el servidor compara con las suscripciones de los clientes. Los tópicos se organizan en niveles separados por el carácter `/`, y las suscripciones se expresan mediante filtros que se comparan con los tópicos de mensajes entrantes. Las expresiones de suscripción pueden incluir los comodines `+`, que coincide con exactamente un nivel, y `#`, que solo puede ubicarse al final del filtro y coincide con cualquier cantidad de niveles.

De esta manera las expresiones de suscripción permiten, por ejemplo, en una flota de robots que publica en tópicos de la forma `flota/{ID de robot}/{sensor}`, suscribirse con el filtro `flota/+/bateria` que recibe el estado de batería de todos los robots o el filtro `flota/robot1/#`, que recibe todos los mensajes del robot `robot1`.

Al implementar el esquema basado en tópicos, MQTT logra el desacoplamiento en el espacio, ya que el publicador solo conoce el nombre del tópico y es el servidor quien determina qué clientes reciben cada mensaje. La versión 5.0 agrega además las suscripciones compartidas, en las que cada mensaje se entrega a uno solo de los clientes de un grupo, lo que permite repartir el procesamiento entre varios consumidores.

La entrega de los mensajes se rige por tres niveles de calidad de servicio (QoS), cuyos intercambios de paquetes muestra la Figura 1. Con QoS 0, el mensaje se envía sin confirmación ni reintentos y llega una vez o ninguna. Con QoS 1, el receptor confirma cada mensaje con un paquete PUBACK y el emisor lo conserva hasta recibir esa confirmación, lo que asegura que llegue al menos una vez aunque pueden producirse duplicados. Con QoS 2, un intercambio en dos etapas garantiza que el mensaje llegue exactamente una vez, a costa de una mayor sobrecarga.

El nivel de QoS se aplica por separado entre el publicador y el broker y entre el broker y cada suscriptor, y cada suscriptor recibe el mensaje con el menor nivel de QoS entre el usado al publicarlo y el máximo concedido a su suscripción. La relación entre estos niveles y la confiabilidad que ya ofrece TCP se analiza en la Sección 1.3.

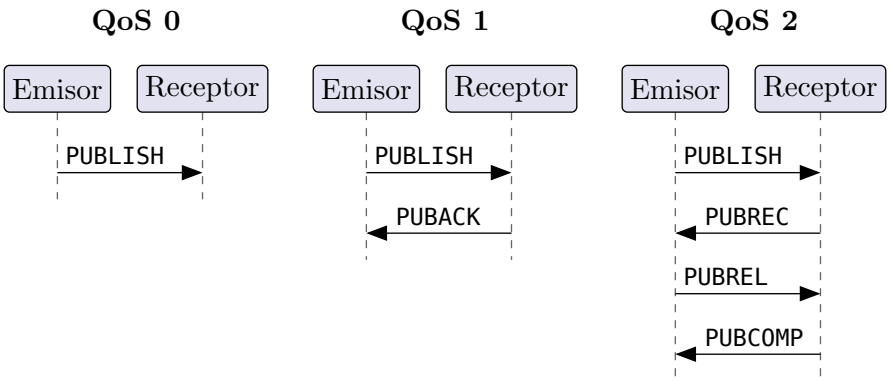


Figura 1: Intercambio de paquetes de control en cada nivel de QoS.

Las confirmaciones de QoS 1 y 2 requieren que cliente y servidor mantengan un estado de sesión. En el servidor, ese estado comprende las suscripciones del cliente y los mensajes destinados a él que aún no fueron enviados o confirmados, y puede conservarse a lo largo de varias conexiones consecutivas. Al conectarse, el cliente indica si inicia una sesión nueva o retoma la anterior, y durante cuánto tiempo debe conservarse el estado una vez cerrada la conexión. Este mecanismo es el que permite el desacoplamiento en el tiempo, ya que el servidor almacena los mensajes dirigidos a un suscriptor desconectado y se los entrega cuando este vuelve a conectarse. Sin embargo, ese desacoplamiento no es incondicional, ya que si la sesión no persiste más allá de la conexión, o si los mensajes se publican con QoS 0 que el servidor no está obligado a almacenar, un suscriptor desconectado nunca recibe los mensajes publicados, por lo que corresponde a la aplicación elegir la combinación adecuada para cada tipo de dato.

Complementando este comportamiento, MQTT suma dos mecanismos más. Un mensaje publicado con el indicador RETAIN se almacena en el servidor como último mensaje retenido de su tópico y se entrega a cada nuevo suscriptor, de modo que un cliente que se suscribe recibe de inmediato el estado más reciente aunque el publicador lo envíe de forma irregular. El mensaje de última voluntad (*Will Message*), por su parte, lo registra el cliente al conectarse y el servidor lo publica a los suscriptores si la conexión se cierra de forma anormal, por ejemplo cuando el cliente deja de comunicarse dentro del intervalo de keep-alive, que es el tiempo máximo que puede transcurrir entre dos paquetes enviados por el cliente. Esto permite notificar a los interesados de la desconexión de un dispositivo.

Por último, toda la comunicación en MQTT se realiza mediante paquetes de control, de los cuales la especificación define quince tipos. Entre ellos se encuentran CONNECT y CONNACK para establecer la conexión, PUBLISH para transportar los mensajes de aplicación, PUBACK y los demás paquetes de confirmación de los niveles de QoS, SUBSCRIBE y UNSUBSCRIBE para gestionar las suscripciones, y PINGREQ y PINGRESP para el keep-alive. Cada paquete se compone de un encabezado fijo, presente en todos ellos, seguido de un encabezado variable y de una carga útil que solo aparecen en algunos tipos. El primer byte del encabezado fijo indica el tipo de paquete y algunos indicadores, como el nivel de QoS y RETAIN en el caso de PUBLISH, y a continuación se codifica la longitud restante del paquete con un entero de longitud variable que ocupa un solo byte para valores de hasta 127. De este modo, el encabezado fijo ocupa apenas dos bytes en los paquetes pequeños, que es también el tamaño total de los paquetes de keep-alive. El efecto de este encabezado sobre el costo de cada mensaje se mide en el experimento de la Sección 5.2.

El contenido de los mensajes queda fuera del protocolo MQTT, que transporta la carga útil sin interpretarla. La versión 5.0 permite declarar su formato mediante las propiedades de indicador

de formato y tipo de contenido, que el servidor reenvía sin modificar, pero no impone ninguno, aspecto que se retoma en la Sección 4. La representación de los datos queda así a cargo de la aplicación, que es el problema que aborda Protocol Buffers, descrito en la Sección 3.

3. Protocol Buffers

Una conexión TCP entrega una secuencia de bytes, y un broker MQTT reenvía el contenido de cada mensaje sin interpretarlo. Para que un dato producido por un microcontrolador sea leído correctamente por un servicio escrito en otro lenguaje y ejecutado en otra arquitectura, ambos extremos deben acordar cómo se traduce una estructura de datos a bytes y viceversa, incluyendo el orden de los bytes, el tamaño de los enteros y la forma de delimitar cada campo. En el modelo OSI esta tarea corresponde a la capa de presentación [5], pero la arquitectura TCP/IP no la incluye, por lo que cada aplicación debe resolverla por su cuenta [2].

Protocol Buffers, también conocido como Protobuf, es un mecanismo para serializar datos estructurados que es neutral respecto del lenguaje de programación y de la plataforma [7]. Fue desarrollado por Google, que lo publicó como software libre y mantiene implementaciones oficiales para una variedad de lenguajes [8]. Su propuesta es comparable a la de JSON, pero con mensajes más pequeños, un procesamiento más rápido y la generación de código nativo para cada lenguaje. Según su documentación, ofrece un formato de serialización para paquetes de datos estructurados y tipados de hasta unos pocos megabytes, adecuado tanto para el tráfico de red efímero como para el almacenamiento de datos a largo plazo, y que puede extenderse con nueva información sin invalidar los datos existentes ni obligar a actualizar el código [7].

3.1. Esquema y generación de código

La estructura de los datos se define una única vez en un archivo de esquema con extensión `.proto`, en el que cada mensaje se describe como un conjunto de campos con un tipo, un nombre y un número que lo identifica. Por ejemplo, la lectura de un sensor ambiental podría definirse así:

```
syntax = "proto3";

message Lectura {
  float temperatura = 1;
  float humedad     = 2;
  uint32 bateria    = 3;
}
```

A partir de este archivo, el compilador `protoc` genera el código fuente que permite construir, serializar y deserializar el mensaje en cada lenguaje de destino. De este modo, el `.proto` cumple a la vez la función de documentación y la de contrato explícito entre los equipos o servicios que intercambian los datos, y un mismo esquema produce implementaciones compatibles en todos los lenguajes soportados.

3.2. Codificación binaria

En la codificación binaria de Protobuf, cada campo se transmite precedido de una etiqueta (*tag*) que combina el número de campo definido en el esquema con un tipo de codificación (*wire type*), que indica cómo leer el valor a continuación, como un entero de longitud variable, un valor de 32 o de 64 bits de longitud fija, o un bloque precedido de su longitud, que se usa para cadenas,

bytes y mensajes anidados [9]. Se trata, en esencia, de una codificación de tipo-longitud-valor (TLV), como la de BER.

Los enteros se codifican como *varints*, en los que cada byte aporta siete bits del valor y utiliza el bit más significativo para indicar si el número continúa en el byte siguiente. Así, los valores pequeños, que son los más frecuentes, ocupan un solo byte. Por ejemplo, un campo número 1 con el valor 150 se codifica en tres bytes, `08 96 01`: el primero es la etiqueta y los dos restantes el valor [9]. Para los enteros con signo existe además la codificación *zigzag*, que intercala positivos y negativos para que los valores negativos pequeños también resulten cortos. El mensaje no incluye los nombres de los campos, que solo existen en el esquema, y en proto3 los campos que tienen su valor por defecto directamente no se transmiten [10]. Esta combinación es la que hace que un mensaje Protobuf pese varias veces menos que su equivalente en JSON y que se procese más rápido.

3.3. Compatibilidad entre versiones

Dado que los campos se identifican por número y no por posición ni por nombre, el esquema puede evolucionar sin romper a quienes usan versiones anteriores. Un programa que recibe un campo cuyo número no conoce simplemente lo ignora, de modo que los clientes viejos pueden leer mensajes nuevos (compatibilidad hacia adelante). En sentido inverso, un programa nuevo que lee un mensaje viejo encuentra los valores por defecto en los campos ausentes (compatibilidad hacia atrás). La condición para que esto funcione es no reutilizar números de campo, para lo cual el lenguaje ofrece la palabra clave `reserved`, que impide volver a asignar los números o nombres de campos eliminados [10]. Esta compatibilidad en ambos sentidos es la principal fortaleza de Protobuf frente a otros formatos binarios.

3.4. Limitaciones

Las mismas decisiones que reducen el tamaño de los mensajes tienen costos. En primer lugar, el formato no es legible por personas, por lo que la depuración requiere herramientas como `protoc --decode` o conversores a JSON. En segundo lugar, no es autodescriptivo: sin el esquema no es posible interpretar los datos, ya que el binario no indica qué mensaje contiene ni qué significa cada campo. En tercer lugar, la codificación no delimita los mensajes, por lo que el protocolo que los transporta debe indicar dónde termina cada uno. Por último, agrega al proyecto un paso de compilación y de generación de código, y resulta menos práctico que JSON para APIs públicas simples o para su uso directo desde navegadores, aunque para ese caso existen alternativas como gRPC-Web.

4. Stack MQTT y Protocol Buffers

Las secciones anteriores describieron MQTT y Protocol Buffers por separado. Esta sección analiza cómo se combinan: qué lugar ocupa cada uno en un mensaje, cómo sabe el receptor qué tipo de mensaje recibe, qué implica compartir el esquema entre los participantes y qué requisitos de los sistemas estudiados cubre cada pieza.

4.1. Protocol Buffers como carga útil

MQTT y Protocol Buffers resuelven problemas distintos y complementarios:

- MQTT define cómo se distribuyen los mensajes entre los participantes, pero no impone ningún formato para su contenido, ya que la carga útil de cada paquete PUBLISH es una secuencia de bytes cuyo contenido y formato son específicos de la aplicación [6].

- Protocol Buffers define cómo representar datos estructurados en una secuencia de bytes compacta, pero no cómo transportarla, como se vio en la Sección 3.

La combinación consiste, entonces, en serializar cada mensaje con Protocol Buffers y publicarlo como carga útil de un paquete PUBLISH, que el broker reenvía a los suscriptores sin interpretarlo. A modo de analogía, el mensaje de Protocol Buffers es la carta y el paquete PUBLISH, el sobre que la transporta: el broker lee la dirección escrita en el sobre, que es el tópico, pero nunca lo abre.

La división de tareas se complementa también en la delimitación de los mensajes. Como se señaló en la Sección 3, la codificación de Protocol Buffers no indica dónde termina un mensaje, por lo que, si se envían varios por un mismo flujo, la aplicación debe anteponer el tamaño de cada uno [11]. Sobre una conexión TCP, que solo entrega una secuencia continua de bytes, esa tarea quedaría a cargo de la aplicación. MQTT, en cambio, transporta cada mensaje de aplicación en un único paquete PUBLISH, cuya longitud se obtiene de su encabezado fijo [6], de modo que cada paquete contiene exactamente un mensaje de Protocol Buffers.

4.2. Asociación entre tópico y esquema

Un mensaje de Protocol Buffers tampoco indica a qué tipo pertenece, y sin su esquema no puede interpretarse [7], por lo que el receptor debe saber de antemano cuál de los mensajes definidos en el archivo `.proto`, descrito en la Sección 3, contiene cada carga útil. La práctica habitual es asociar cada tópico, o cada patrón de tópicos, a un único tipo de mensaje. De este modo, el tópico cumple una doble función:

- Determina a qué suscriptores entrega el broker cada mensaje, como en cualquier sistema basado en tópicos.
- Identifica el esquema con el que se interpreta el contenido del mensaje.

Por ejemplo, en la flota de robots que publica en tópicos de la forma `flota/{ID de robot}/{sensor}`, el tópico `flota/robot1/bateria` puede transportar siempre un mensaje `EstadoBateria`, y un suscriptor con el filtro `flota/+/bateria` sabe que todo lo que recibe se interpreta con ese mismo esquema.

Además de esta convención, la versión 5.0 de MQTT incorpora propiedades opcionales que el publicador puede agregar al paquete PUBLISH y que resultan útiles para la combinación. El broker reenvía estas propiedades a cada suscriptor junto con el mensaje, sin modificarlas, con excepción del alias de tópico, que solo tiene validez dentro de cada conexión [6]:

- El **indicador de formato de la carga útil** (*Payload Format Indicator*) distingue entre bytes sin especificar, como un mensaje de Protocol Buffers, y texto codificado en UTF-8, como un documento JSON.
- El **tipo de contenido** (*Content Type*) es una cadena que puede nombrar el tipo de mensaje transportado. La especificación no fija ningún valor para Protocol Buffers, por lo que el emisor y el receptor deben acordarlo. Los sistemas que todavía usan MQTT 3.1.1, como los de robótica analizados en la Sección 4.2, no disponen de esta propiedad y deducen el tipo de cada mensaje a partir del tópico.
- Las **propiedades de usuario** (*User Property*) admiten pares nombre-valor definidos por la aplicación, como la versión del esquema.
- El **tópico de respuesta** (*Response Topic*) y los **datos de correlación** (*Correlation Data*) permiten implementar solicitudes y respuestas sobre MQTT (como un comando enviado a un robot y su confirmación), ambos codificados con Protocol Buffers.

- El **alias de tópico** (*Topic Alias*) reemplaza un nombre de tópico largo y repetido por un entero de dos bytes, lo que complementa en el encabezado la reducción de tamaño que Protocol Buffers logra en el contenido.

4.3. Esquema compartido

El archivo `.proto` constituye así un contrato entre los participantes, ya que el formato de los datos se fija una única vez en un esquema distribuido entre los clientes. A partir de ese archivo se genera el código de cada participante, ya sea el firmware de un microcontrolador o un servicio en la nube [7]. En el experimento de la Sección 5.2, por ejemplo, un mismo esquema genera el código de las placas ESP32 y el del backend que recibe sus datos.

Ese contrato introduce un acoplamiento en el formato de los datos que el modelo publicador/suscriptor no elimina. Como publicadores y suscriptores no se conocen ni se actualizan al mismo tiempo, un cambio en un mensaje puede llegar a unos participantes antes que a otros. El desacoplamiento en el tiempo acentúa el problema, ya que un mensaje retenido, o almacenado en la sesión de un suscriptor desconectado, puede leerse mucho después de su publicación con una versión más nueva del esquema. Las reglas de compatibilidad de la Sección 3.3 vuelven manejable esta situación: un publicador puede agregar campos sin afectar a los suscriptores que usan el esquema anterior, y un suscriptor actualizado puede leer los mensajes de los publicadores que no se actualizaron, siempre que no se modifiquen ni se reutilicen los números de campo existentes.

La combinación tiene, a su vez, costos propios que se suman a las limitaciones de cada pieza:

- El broker solo puede encaminar los mensajes según su tópico, ya que no interpreta su contenido, lo que corresponde a la menor expresividad del esquema basado en tópicos frente al basado en contenido [4].
- Todo componente de la infraestructura que necesite leer los datos debe contar con el esquema. Esto alcanza también a los servicios que reciben los mensajes en la nube. Por ejemplo, AWS IoT Core, el servicio de Amazon Web Services que actúa como broker MQTT para dispositivos, puede decodificar mensajes de Protocol Buffers en sus reglas de procesamiento, pero solo si se le proporciona un archivo descriptor generado a partir del `.proto` [12].
- Las herramientas genéricas de MQTT permiten verificar que los mensajes circulan, pero muestran su contenido como bytes ilegibles, lo que dificulta la depuración [13].

4.4. Sparkplug B

La madurez de esta combinación se refleja en su estandarización. La especificación Eclipse Sparkplug, adoptada como norma ISO/IEC 20237 [14], define para los sistemas industriales precisamente lo que MQTT deja abierto, y en su versión B codifica la carga útil con Protocol Buffers [15]. Entre otras cosas, fija el espacio de tópicos, y con él la asociación entre tópico y esquema, y la forma de informar el estado de conexión de cada nodo mediante el mensaje de última voluntad. Su funcionamiento se analiza en la Sección 4.3.

4.5. Síntesis

Cada pieza del stack cubre una parte de los requisitos planteados al comienzo de la sección, como resume la Tabla 2. Cuánto aporta cada una al tamaño y a la latencia de los mensajes se mide en el experimento de la Sección 5.2.

Tabla 2: Requisitos de los sistemas estudiados y pieza del stack que cubre cada uno.

Requisito	**Pieza del stack que lo cubre**
Enviar datos al dispositivo sin que acepte conexiones entrantes	Conexión TCP abierta por el dispositivo y entrega por parte del broker
Comunicar productores y consumidores que no se conocen	MQTT: broker y tópicos
Tolerar desconexiones	MQTT: sesiones persistentes, mensajes retenidos y mensaje de última voluntad
Delimitar cada mensaje	MQTT: un paquete PUBLISH por mensaje
Identificar el tipo de cada mensaje	Asociación entre tópico y esquema; en MQTT 5.0, tipo de contenido
Interpretar los datos de forma unívoca entre equipos heterogéneos	Protocol Buffers: esquema compartido y código generado para cada lenguaje
Reducir el uso de ancho de banda	MQTT: encabezado fijo mínimo y alias de tópico; Protocol Buffers: codificación binaria
Modificar los mensajes sin afectar a los demás participantes	Protocol Buffers: reglas de compatibilidad del esquema

5. Comparación con alternativas

Para intercambiar datos entre un dispositivo (ya sea un sensor, un celular o un robot) y la nube deben tomarse dos decisiones:

1. Qué protocolo de capa de aplicación implementar (define cómo se intercambian los mensajes).
2. Qué mecanismo de serialización se utiliza para representar los datos dentro de cada mensaje (con qué formato viaja la información).

Las secciones anteriores describieron la combinación de MQTT y Protocol Buffers como respuesta a ambas decisiones para el intercambio de datos en tiempo real con ancho de banda limitado, pero no es la única opción, ya que existen otras alternativas consolidadas. Esta sección analiza esas alternativas para identificar en qué aspectos MQTT y Protocol Buffers aventajan a cada una de ellas y en cuáles resultan inferiores. En particular, se compara MQTT con otros dos protocolos de capa de aplicación: HTTP, sobre el que se apoya gran parte de la Internet comercial, y CoAP, diseñado específicamente para dispositivos restringidos. Del mismo modo, se compara Protocol Buffers con JSON, un mecanismo de serialización basado en texto y de uso extendido.

La comparación de protocolos considera el modelo de interacción, el protocolo de transporte, el tamaño de los encabezados, la confiabilidad y el paso por NAT y firewalls, mientras que la de mecanismos de serialización considera el tamaño de los mensajes, el costo de procesamiento y la necesidad de un esquema compartido. El análisis se apoya en las especificaciones de cada tecnología y en estudios publicados.

5.1. MQTT, HTTP y CoAP

Se definirán brevemente los protocolos HTTP y CoAP, para posteriormente adentrarse en sus diferencias con MQTT.

5.1.1. HTTP

Se trata de un protocolo de aplicación **sin estado** de **solicitud y respuesta**. Cada solicitud se interpreta de manera independiente, por lo que el servidor no conserva información sobre los

clientes entre una solicitud y la siguiente [3], [5, p. 780]. Es el protocolo de la Web y el ejemplo característico del modelo cliente-servidor: el cliente envía una solicitud que indica un método (como GET o POST) y el recurso al que se refiere (identificado por una URI, *Uniform Resource Identifier*), y el servidor devuelve una respuesta con un código de estado y, eventualmente, el contenido solicitado [16].

HTTP utiliza TCP como protocolo de transporte, del que obtiene la entrega fiable y ordenada de los datos [3]. En el modelo de solicitud y respuesta, el servidor no puede iniciar una conexión con el cliente ni enviarle una respuesta que no haya sido solicitada. Por eso, un cliente que necesita enterarse de eventos nuevos debe consultar periódicamente al servidor, técnica conocida como *polling* [17].

5.1.2. CoAP

CoAP (*Constrained Application Protocol*) es un protocolo de transferencia web estandarizado por el IETF (*Internet Engineering Task Force*) para nodos y redes restringidos, como microcontroladores con poca memoria que operan sobre redes de bajo consumo y con pérdidas. Está orientado a aplicaciones máquina a máquina (M2M) [18].

Adopta el mismo modelo de solicitud y respuesta que HTTP, con recursos identificados por URI, métodos como GET, POST, PUT y DELETE, y códigos de respuesta. Sin embargo, no busca comprimir HTTP, sino implementar el subconjunto de esa arquitectura que necesitan las aplicaciones M2M, y está diseñado para traducirse fácilmente a HTTP mediante *proxies*, lo que facilita su integración con la Web [18].

CoAP funciona sobre UDP, con DTLS, la variante de TLS para datagramas, como mecanismo de seguridad, y codifica sus mensajes en un formato binario compacto [18]. Como UDP no garantiza la entrega, el propio protocolo incorpora una confiabilidad liviana: los mensajes confirmables (CON) se retransmiten hasta que el receptor responde con un acuse de recibo (ACK), mientras que los no confirmables (NON) se envían sin confirmación, lo que resulta adecuado, por ejemplo, para cada lectura de un flujo de datos de un sensor [18].

Por último, la extensión Observe permite que un cliente se registre en un recurso para que el servidor le envíe una nueva representación cada vez que ese recurso cambia, lo que evita el *polling* [19].

5.1.3. Comparación

De las descripciones anteriores y de los estudios que comparan estos protocolos se desprenden las siguientes diferencias:

1. **Modelo de interacción.** HTTP y CoAP siguen el modelo de solicitud y respuesta, en el que el cliente aplica métodos sobre recursos y recibe códigos de respuesta. En ese modelo, el dato viaja cuando el cliente lo pide, por lo que para enterarse de eventos nuevos debe recurrir al *polling*, cuyo costo crece con la frecuencia de las consultas, o al *long polling*, en el que cada notificación sigue siendo un mensaje HTTP completo, con todos sus encabezados [17]. La extensión Observe acerca CoAP a un modelo de notificaciones, pero sin un broker que desacople a las partes, y su propia especificación aclara que no pretende reemplazar a los sistemas publicador/suscriptor [19]. MQTT, en cambio, implementa el modelo publicador/suscriptor: el broker entrega cada mensaje en el momento en que se publica, mediante la conexión que el suscriptor estableció previamente, y puede conservar los mensajes destinados a un cliente desconectado mediante sesiones persistentes y mensajes retenidos [6], algo que HTTP y CoAP no prevén.

2. **Protocolo de transporte.** HTTP y MQTT funcionan sobre TCP, del que heredan la entrega fiable y ordenada, pero deben establecer una conexión antes de intercambiar datos. MQTT la establece una sola vez y la mantiene abierta, mientras que HTTP puede abrir una conexión por cada solicitud o reutilizarla para varias [3]. CoAP, en cambio, funciona sobre UDP y no necesita establecer una conexión, de modo que una transacción se resuelve con dos datagramas, uno en cada sentido [20].
3. **Tamaño de los encabezados.** HTTP utiliza encabezados de texto de longitud variable, mientras que los de CoAP y MQTT son binarios: el encabezado fijo de CoAP ocupa cuatro bytes [18] y el de MQTT, dos bytes como mínimo [6]. Sin embargo, la conexión TCP que requiere MQTT aumenta su sobrecarga total. Por eso, en las comparaciones cualitativas, HTTP resulta el más pesado en tamaño de mensaje, consumo de energía, ancho de banda y latencia, CoAP el más liviano y MQTT ocupa una posición intermedia (esas comparaciones no consideran las retransmisiones) [20].
4. **Confiabilidad.** HTTP depende exclusivamente de la confiabilidad de TCP. MQTT le agrega tres niveles de calidad de servicio y el mensaje de última voluntad, mientras que CoAP ofrece, mediante sus mensajes no confirmables y confirmables, dos niveles aproximadamente equivalentes a QoS 0 y QoS 1 [20]. Ante pérdidas, MQTT presenta menor retardo que CoAP cuando las pérdidas son bajas y mayor cuando son elevadas, y con mensajes pequeños y pérdidas de hasta el 25 % CoAP genera menos tráfico adicional para garantizar la entrega [21]. En un estudio sobre la comunicación entre vehículos y la nube, CoAP obtuvo las menores latencias y la menor sobrecarga, pero no logró entregar todos los mensajes con pérdidas elevadas, ni siquiera con mensajes confirmables, mientras que los protocolos basados en TCP, entre ellos MQTT, los entregaron todos [22].
5. **Paso por NAT y firewalls.** Muchos firewalls están configurados para bloquear el tráfico UDP [3]. Entre el 2 % y el 4 % de las redes de acceso terrestres lo bloquean por completo, y las restricciones se concentran en las redes empresariales; además, los NAT descartan las asociaciones UDP mucho antes que las TCP (en promedio, a los 160 segundos frente a los 386 minutos), lo que obliga a enviar mensajes de mantenimiento con mayor frecuencia [23]. Esto favorece a HTTP y a MQTT frente a CoAP, que para estos casos cuenta con una variante sobre TCP, TLS y WebSocket, a costa de un mayor tamaño de código, más memoria y más intercambios [23].
6. **Seguridad.** HTTP y MQTT se protegen con TLS sobre TCP, mientras que CoAP utiliza DTLS sobre UDP [20]. MQTT, además, deja a cada implementación la elección de los mecanismos de autenticación y autorización [6].
7. **Interoperabilidad y formato de los datos.** HTTP es el protocolo más interoperable, ya que basta con disponer de una pila HTTP en cada extremo [20], y CoAP se integra con la Web mediante *proxies* que traducen a HTTP [18]. MQTT, en cambio, requiere un broker y no indica en sus mensajes el formato del contenido, por lo que todos los clientes deben conocerlo de antemano [20]; la versión 5.0 agrega propiedades opcionales para declararlo, pero no impone ninguno [6]. Ninguno de los tres protocolos impone un mecanismo de serialización, aspecto que se analiza en la siguiente subsección.

5.1.4. Ventajas y desventajas

Frente a HTTP, la principal ventaja de MQTT es que el broker puede entregarle datos al dispositivo sin que este los solicite, lo que evita el *polling*. A esto se suman encabezados mínimos, una única conexión que se mantiene abierta y sirve en ambos sentidos, niveles de calidad de servicio propios y la posibilidad de conservar los mensajes de un cliente desconectado. Como contrapartida, MQTT es menos interoperable que HTTP, requiere un broker, que agrega un

salto a cada mensaje y concentra la carga de todos los participantes, como se señaló al describir el modelo publicador/suscriptor, y obliga a que los clientes acuerden de antemano el formato de los datos.

Frente a CoAP, la ventaja de MQTT es la confiabilidad: al funcionar sobre TCP, en los estudios citados entregó todos los mensajes aun con pérdidas elevadas, y atraviesa con menos dificultad los NAT y firewalls, y su broker ofrece un desacoplamiento completo entre publicadores y suscriptores, que las notificaciones de mejor esfuerzo de Observe no alcanzan. Como contrapartida, CoAP es más liviano: genera menos sobrecarga, no requiere la pila TCP y admite solicitudes *multicast* [18], para las que MQTT no está diseñado [20]. Para redes en las que TCP no es viable, como algunas redes inalámbricas de sensores, existe una variante, MQTT-SN, que funciona sobre un servicio de datagramas y se conecta al broker mediante un *gateway* [24].

En síntesis, MQTT resulta preferible cuando se necesita una entrega confiable de datos en ambos sentidos sobre redes que no se controlan, con dispositivos detrás de NAT o firewalls; CoAP conviene para dispositivos muy restringidos que envían actualizaciones pequeñas sin necesidad de entrega garantizada [18]; y HTTP, para intercambios puntuales iniciados por el cliente o cuando la integración con la Web es prioritaria. La Tabla 3 resume la comparación.

Tabla 3: Comparación entre HTTP, CoAP y MQTT.

Criterio	**HTTP**	**CoAP**	**MQTT**
Modelo de interacción	Solicitud y respuesta	Solicitud y respuesta (con Observe, notificaciones de mejor esfuerzo)	Publicador/suscriptor
Entrega sin solicitud previa	No (requiere *polling*)	Solo con Observe	Sí, a través del broker
Clientes desconectados	Sin entrega posterior	Sin entrega posterior	Sesiones persistentes y mensajes retenidos
Transporte	TCP	UDP	TCP
Encabezado	Texto, de longitud variable	Binario, 4 bytes fijos	Binario, 2 bytes fijos como mínimo
Confiabilidad	La de TCP	Mensajes confirmables y no confirmables	La de TCP y QoS 0, 1 y 2
Paso por NAT y firewalls	Favorable (TCP)	Desfavorable (UDP)	Favorable (TCP)
Seguridad	TLS	DTLS	TLS
Interoperabilidad	La más alta	Alta, mediante *proxies* a HTTP	Requiere un broker y un formato acordado

5.2. Protocol Buffers y JSON

Se definirá brevemente JSON, para posteriormente contrastarlo con Protocol Buffers.

5.2.1. JSON

JSON (*JavaScript Object Notation*) es un formato de intercambio de datos liviano, basado en texto e independiente del lenguaje de programación, derivado del estándar del lenguaje ECMAScript [25]. Representa los datos mediante objetos, que son conjuntos de pares nombre--valor, y arreglos, cuyos elementos pueden ser cadenas de texto, números, valores booleanos, el valor nulo u otros objetos y arreglos. Fuera de un ecosistema cerrado, los textos JSON deben

codificarse en UTF-8 [25]. Por ejemplo, una lectura de un sensor con su temperatura y su nivel de batería puede representarse así:

```
{"temperatura":23.5,"bateria":80}
```

Como cada mensaje incluye los nombres de sus campos, JSON es autodescriptivo: puede leerse e interpretarse sin conocer de antemano la estructura de los datos, del mismo modo que los mensajes de HTTP/1.1, escritos en texto legible [3]. Esa característica determina su principal ventaja y su principal desventaja:

- **Ventaja:** cualquier persona o programa puede leer el mensaje sin un acuerdo previo sobre su estructura, lo que facilita la depuración y la interoperabilidad.
- **Desventaja:** los nombres de los campos viajan en cada mensaje y los números se representan como texto decimal. El ejemplo anterior ocupa 33 bytes, de los cuales 22 corresponden a los nombres de los campos y sus comillas. Además, la especificación recomienda no esperar más precisión que la de un número de punto flotante de doble precisión, por lo que solo garantiza la interoperabilidad de los enteros de hasta $2^{53} - 1$ en valor absoluto [25].

5.2.2. Contraste con Protocol Buffers

De la descripción anterior, de la presentada para Protocol Buffers y de los estudios que comparan ambos mecanismos se desprenden las siguientes diferencias:

1. **Representación.** JSON codifica los datos como texto, mientras que Protocol Buffers utiliza un formato binario en el que cada campo se codifica como un registro de tipo, longitud y valor, identificado por un número de campo en lugar de su nombre [9].
2. **Esquema compartido.** JSON no requiere un esquema previo. Protocol Buffers, en cambio, requiere que emisor y receptor compartan el archivo `.proto` que define la estructura de los mensajes, ya que sin él no es posible interpretar los datos recibidos. A cambio, ese esquema permite generar código para cada lenguaje y fijar el tipo de cada campo [7].
3. **Tamaño de los mensajes.** Al reemplazar los nombres por números de campo y codificar los valores en binario, Protocol Buffers produce mensajes considerablemente más pequeños. En un relevamiento sobre 27 documentos JSON de distintos ámbitos, Protocol Buffers redujo el tamaño con una mediana del 70,6 % [26]; con cargas reales tomadas de mensajes MQTT públicos, el tamaño medio pasó de 111 bytes en JSON a 40 bytes [27]; y con mensajes de producción de vehículos conectados, de 12.187 a 1.157 bytes en promedio [22]. La reducción, sin embargo, no es universal: en uno de los 27 documentos, formado por arreglos muy anidados y casi sin nombres de campo, Protocol Buffers resultó más grande que JSON [26].
4. **Costo de procesamiento.** Aunque la documentación presenta a Protocol Buffers como más rápido que JSON [7], en microcontroladores la diferencia no es clara: en algunas arquitecturas, la biblioteca de JSON evaluada serializó más rápido. Aun así, con radios lentas transmitir un byte consume unas diez mil veces más energía que un ciclo de CPU, por lo que Protocol Buffers resultó la opción más eficiente en energía, a costa de un mayor uso de memoria RAM [27].
5. **Tipado y evolución.** JSON ofrece solo tipos genéricos y no impone una estructura, por lo que nada garantiza que emisor y receptor interpreten los datos de la misma manera. Protocol Buffers asigna un tipo concreto a cada campo y controla la evolución de los mensajes: el código nuevo puede leer mensajes antiguos, y los campos ausentes toman un valor por defecto [7].

6. **Estandarización y alcance.** JSON es un estándar del IETF [25], mientras que Protocol Buffers no es un estándar formal de ningún organismo, lo que puede ser una limitación en entornos que exigen basarse en estándares. Además, Protocol Buffers supone que cada mensaje entra completo en memoria, por lo que no resulta adecuado para datos de más de unos pocos megabytes [7].

5.2.3. Ventajas y desventajas

La principal ventaja de Protocol Buffers frente a JSON es el tamaño de los mensajes, que en los estudios citados resulta de casi tres a más de diez veces menor. Esta diferencia es especialmente relevante en los sistemas considerados, que envían mensajes pequeños a alta frecuencia por enlaces limitados: de poco sirve que MQTT o CoAP reduzcan sus encabezados a unos pocos bytes si la carga útil se codifica en texto. A esto se suman el tipado de los campos y la evolución controlada del esquema. Como contrapartida, Protocol Buffers sacrifica la legibilidad, obliga a distribuir y mantener el esquema entre todos los participantes, lo que introduce un acoplamiento en el formato de los datos que el modelo publicador/suscriptor no elimina, y no es un estándar formal. JSON, por su parte, conviene cuando la legibilidad, la depuración o la integración con sistemas existentes, como las aplicaciones web, pesan más que el tamaño de los mensajes.

En síntesis, Protocol Buffers resulta preferible cuando el tamaño de los mensajes y el costo de transmitirlos son críticos y es posible distribuir un esquema común entre los participantes, mientras que JSON conviene cuando se priorizan la legibilidad, la flexibilidad y la interoperabilidad sin acuerdos previos. La Tabla 4 resume la comparación.

Tabla 4: Comparación entre JSON y Protocol Buffers.

Criterio	**JSON**	**Protocol Buffers**
Representación	Texto (UTF-8)	Binaria
Esquema	No requiere	Requiere un archivo `.proto` compartido
Autodescriptivo	Sí: incluye los nombres de los campos	No: incluye números de campo
Legibilidad	Legible por personas	Requiere herramientas y el esquema
Números	Texto decimal	Binarios, de longitud variable o fija
Tamaño de los mensajes	Mayor	De casi tres a más de diez veces menor
Tipado	Genérico	Un tipo concreto por campo
Evolución	Sin control	Controlada por números de campo y valores por defecto
Estandarización	Estándar del IETF	Sin estandarización formal

Desarrollo

Este capítulo sigue el recorrido de un mensaje desde el dispositivo hasta el broker. Analiza la conexión TCP y TLS sobre la que viaja MQTT, la capa de red que esa conexión atraviesa y el ruteo por tópicos que realiza el broker en la capa de aplicación (Sección 3). Examina después cómo emplean el stack sistemas reales (Sección 4), contrasta lo analizado con dos experimentos propios (Sección 5.2 y Sección 5.3) y releva el hardware con el que se implementaría una solución (Sección 6).

1. MQTT sobre TCP y TLS

MQTT delega en TCP la entrega fiable y ordenada de los mensajes (Sección 2). Esta sección analiza qué implica esa dependencia, en particular cómo se mantiene viva la conexión, cuánto cuesta cifrarla con TLS y qué agregan los niveles de QoS a la confiabilidad que ya ofrece TCP.

1.1. Conexión persistente y keep-alive

Como se planteó en el marco teórico (Sección 2), el cliente MQTT abre una única conexión TCP hacia el broker y la mantiene abierta mientras dura la sesión. A diferencia de HTTP, en el que cada solicitud puede abrir una conexión nueva o reutilizar una abierta hasta que el servidor la cierra por inactividad [3], en MQTT el cierre anormal se interpreta como la caída del cliente y el broker publica su mensaje de última voluntad. Esto plantea el problema de saber si el otro extremo sigue ahí. Un dispositivo que se apaga o sale del alcance de la red no envía ningún segmento de cierre, y lo mismo ocurre cuando un NAT descarta la entrada de una conexión inactiva, situación que se analiza más adelante, en la Sección 2.1. Un extremo que solo recibe, como un suscriptor, no se entera nunca, porque TCP no verifica de forma continua que el otro extremo sea alcanzable [2], y su keep-alive propio es opcional, viene desactivado por defecto y no debe tener un intervalo menor que dos horas [28].

MQTT lo resuelve con un keep-alive propio. En el paquete CONNECT, el cliente declara un intervalo K en segundos, y el valor 0 desactiva el mecanismo. Mientras no sea cero, el cliente debe enviar algún paquete de control dentro de cada intervalo y, si no tiene nada que publicar, envía un PINGREQ, que el broker contesta con un PINGRESP. Si el broker no recibe ningún paquete del cliente durante 1,5 K, debe cerrar la conexión como si la red hubiera fallado [6]. El tiempo que tarda en detectar una caída depende de cuándo ocurre, por ejemplo, si la falla sucede justo después de un paquete, el broker la detecta 1,5 K más tarde, y si sucede justo antes del siguiente envío, 0,5 K más tarde. Con el intervalo de 60 s del Experimento 1, que se describe en el diseño experimental (Sección 5.2), la detección demora entre 30 y 90 s. Hasta entonces el broker no publica la última voluntad y, en Sparkplug, la aplicación de supervisión sigue considerando vigentes los datos de un nodo caído, porque su certificado de muerte es justamente ese mensaje.

Elegir K implica un compromiso, que resume la Tabla 5. Un intervalo corto detecta antes la caída y mantiene viva la entrada del NAT, pero tiene dos costos. El primero es el tráfico de un cliente que no publica, como un suscriptor que solo espera órdenes. Con TLS, cada intercambio de PINGREQ y PINGRESP ocupa unos 182 B, y con un intervalo de 10 s suma unos 47 MB por mes, más que los 36 MB que ocupan las lecturas de un nodo que publica cada 10 s (Sección 6). Ese nodo, en cambio, no envía PINGREQ, porque cada publicación ya mantiene activa la conexión. El segundo son las desconexiones innecesarias, ya que el umbral de 1,5 K es el corte del camino que el stack soporta sin perder la conexión, y con K de 10 s es de 15

s, menor que los 40 s que OSPF tarda en detectar una falla por ausencia de Hellos [29]. Ese corte cerraría la conexión y obligaría al dispositivo a repetir los handshakes de TCP y de TLS (Sección 1.2). En la práctica se usan intervalos de decenas de segundos, por ejemplo, 30 s en el Gateway Bridge de ChirpStack [30], menos de un minuto recomendado por Digitransit [31] y 60 s en el Experimento 1. Un valor de entre 30 y 60 s detecta una caída en menos de un minuto y medio y absorbe los cortes breves del ruteo.

Tabla 5: Efecto del intervalo de keep-alive K*. El tráfico es una estimación con TLS, a razón de 182 B por intercambio de PINGREQ y PINGRESP (Tabla 17), durante 30 días. No se midió en el Experimento 1.*

Intervalo *K*	**Umbral del broker (1,5 *K*)**	**Detección de una falla**	**Tráfico en una conexión inactiva**
10 s	15 s	5 a 15 s	47,2 MB por mes
30 s	45 s	15 a 45 s	15,7 MB por mes
60 s	90 s	30 a 90 s	7,9 MB por mes
300 s	450 s	150 a 450 s	1,6 MB por mes

1.2. Cifrado con TLS

MQTT no define un mecanismo de cifrado propio y delega la protección de los datos en el transporte. Sobre TCP se utiliza TLS, en el puerto 8883 registrado para ese fin [6]. TLS ofrece confidencialidad, integridad de cada mensaje y autenticación del servidor mediante un certificado [3], [5], y se apoya en TCP para obtener un servicio fiable extremo a extremo [5], la misma exigencia que MQTT plantea al transporte [6]. TLS se ubica entre TCP y MQTT, primero se establece la conexión TCP, luego se realiza el handshake de TLS y recién después el cliente envía el CONNECT, que ya viaja cifrado, de modo que el usuario y la contraseña quedan protegidos.

El costo del cifrado tiene dos partes. La primera es el handshake, que se realiza una sola vez al conectar. En TLS 1.2, la versión empleada en el Experimento 1, requiere dos idas y vueltas [32], de modo que, sumado al handshake de TCP, el cliente envía el CONNECT recién después de unos tres tiempos de ida y vuelta (RTT, *round-trip time*). Con 200 ms de RTT agregados, el tiempo de conexión aumentó entre 340 y 570 ms (Sección 5.2). TLS 1.3 reduce el handshake completo a una sola ida y vuelta [33]. La segunda parte es el recargo de cada mensaje, con TLS 1.2 y AES-GCM, cada registro agrega 29 B (5 de encabezado, 8 de *nonce* explícito y 16 de etiqueta de autenticación), lo que explica casi todo el aumento de 46,5 B por lectura de MQTT QoS 0, cuya latencia crece menos de 0,6 ms.

Lo que determina cuánto pesa el handshake es cuántos mensajes se envían por cada conexión. En MQTT la conexión es persistente, por lo que el handshake se paga una vez y se reparte entre todas las lecturas. HTTPS puede lograr lo mismo, por ejemplo: HTTP/1.1 mantiene las conexiones abiertas por defecto [3], y con una conexión reutilizada el costo es comparable. Pero un dispositivo que duerme entre envíos, o un servidor que cierra las conexiones inactivas, paga el handshake completo en cada lectura. En el Experimento 1 con una conexión por lectura, TLS multiplicó por 3,1 los bytes por lectura y sumó 3,4 s de latencia, mientras que en las conexiones persistentes sumó entre 46 y 97 B y menos de 0,6 ms (Sección 5.2). Por eso la conexión persistente y el keep-alive de la Sección 1.1 son lo que hace barato el cifrado.

Por último, TLS protege el tramo entre el cliente y el broker, no el recorrido completo del mensaje. Como el broker termina la conexión TLS, ve el contenido en claro, y un mensaje que va de un publicador a un suscriptor atraviesa dos conexiones TLS independientes, una por tramo.

Proteger el contenido frente al propio broker exige cifrar la carga útil en la aplicación, como hace Meshtastic con la clave de cada canal.

1.3. QoS MQTT sobre la confiabilidad de TCP

Los niveles de calidad de servicio de MQTT, presentados en el marco teórico (Sección 2), plantean una pregunta, ya que el protocolo opera sobre TCP, que entrega los datos de forma fiable y ordenada. TCP construye sobre el servicio no confiable de IP un flujo de bytes sin errores, huecos ni duplicados que llega en el orden en que se envió [3], mediante números de secuencia, confirmaciones y retransmisiones. Por eso un mensaje de QoS 0 sobre TCP no equivale a un datagrama que puede perderse en el camino: mientras la conexión existe, un segmento perdido se retransmite, y lo que se degrada es la latencia y no la entrega, como se verificó en el Experimento 1 (Tabla 15). La confiabilidad de TCP y la de MQTT se refieren, entonces, a cosas distintas, y los niveles de QoS cubren lo que TCP no garantiza, que son tres cosas.

1. **Que el mensaje llegó a su destino final.** Un mensaje MQTT atraviesa dos conexiones TCP independientes, la del publicador con el broker y la del broker con cada suscriptor, y el nivel de QoS se aplica por separado en cada tramo.
2. **Que la aplicación lo recibió.** El ACK de TCP lo genera la pila del receptor cuando coloca los datos en el buffer de recepción, y el programa los lee después [3]. Si el programa falla o la conexión se aborta entre ambos momentos, el emisor da por entregado un dato que no lo está.
3. **Que sobreviva a la conexión.** La confiabilidad de TCP existe mientras la conexión existe. Después de un corte mayor que 1,5 K, un reset del broker o un cambio de NAT, el emisor no sabe cuáles de los datos que envió llegaron.

QoS 0 no agrega nada a TCP [6], sin embargo QoS 1 agrega una confirmación de la aplicación, esta es que el emisor conserva el mensaje hasta recibir el PUBACK, de modo que sabe que el broker lo aceptó y no solo que su pila TCP lo recibió, y si la conexión se rompe lo reenvía al retomar la sesión [6]. Su garantía es de al menos una vez, porque si el corte ocurre después de que el receptor procesó el mensaje y antes del PUBACK, el receptor lo recibe dos veces. QoS 2 agrega un intercambio de cuatro paquetes con el que el receptor evita entregar el mensaje dos veces, a costa de dos idas y vueltas por mensaje. La Tabla 6 resume qué cubre cada nivel. Ninguno resuelve la primera carencia (cada tramo se confirma por separado, y el publicador no sabe si el mensaje llegó a los suscriptores).

Tabla 6: Qué garantiza TCP y qué agrega cada nivel de QoS en cada tramo del recorrido de un mensaje [6].

Situación	**TCP**	**QoS 0**	**QoS 1**	**QoS 2**
Segmento perdido en la red, con la conexión viva	TCP lo retransmite; los niveles de QoS no agregan nada			
El receptor MQTT confirma que aceptó el mensaje, en cada tramo	No: el ACK confirma a la pila, no a la aplicación	No	Sí, con PUBACK	Sí, con PUBREC y PUBCOMP
Conexión cortada con el mensaje en tránsito	Se pierde	Se pierde	Se reenvía si se retoma la sesión	Se reenvía si se retoma la sesión
Entregas de cada mensaje	No aplica	A lo sumo una	Al menos una	Exactamente una

Los reenvíos de QoS 1 y 2 solo funcionan si el estado sobrevive a la conexión, estos ocurren cuando el cliente se reconecta con Clean Start en 0 y el broker conservó la sesión, que vence según el Session Expiry Interval [6]. Si la sesión es nueva o expiró, después de un corte QoS 1

y 2 no aportan nada frente a QoS 0. Además, dentro de una conexión viva los reintentos de la aplicación se superponen con los de TCP: si el PUBACK se demora, el cliente reenvía el PUBLISH mientras TCP todavía está recuperando el original. Así, QoS 1 no mejora la entrega y puede generar duplicados, como se midió en el Experimento 1 (Tabla 15), y su costo por lectura, sin contar el de abrir la conexión, es de 202 B contra 140 B de QoS 0 (Tabla 18).

La elección del nivel depende de qué ocurre cuando falta un mensaje. Si el siguiente lo reemplaza, como en una lectura periódica, QoS 0 sobre TCP alcanza, y su única exposición son los cortes de la conexión. Sparkplug, por otra parte, publica con QoS 0 y suple la confirmación con un número de secuencia de 0 a 255, y si la aplicación advierte un salto le pide al nodo un nuevo certificado de nacimiento para reconstruir su estado (Sección 4.3, en los casos de uso). Si el mensaje es una orden o un evento que no se repite, conviene QoS 1 con sesión persistente y un receptor que tolere duplicados, y QoS 2 queda para los casos en que el duplicado es inaceptable. Los cortes de conexión, que el Experimento 1 no cubrió, quedan para el ensayo de la Sección 5.3.

2. Capa de red

La conexión TCP que abre el dispositivo atraviesa redes que ni el dispositivo ni el operador del broker controlan. Esta sección analiza los dos aspectos de la capa de red que más la afectan, los NAT y firewalls que se interponen en el camino y los protocolos de ruteo que lo determinan y lo restablecen cuando una falla lo interrumpe.

2.1. NAT y firewalls

La mayoría de los dispositivos considerados en este trabajo no tiene una dirección IP pública propia. Un robot en un depósito, un sensor en una planta industrial o un teléfono en una red celular se encuentran detrás de un traductor de direcciones (NAT), que reemplaza la dirección y el puerto de origen de cada datagrama saliente por los de su interfaz pública y registra la correspondencia en una tabla, para poder traducir en sentido inverso las respuestas que llegan [3]. Ante el agotamiento de las direcciones IPv4, muchos proveedores de acceso agregan además su propio NAT de gran escala, de modo que el tráfico del dispositivo puede atravesar dos traducciones antes de llegar a Internet [34].

La tabla de traducción solo incorpora una entrada cuando un host interno inicia una conexión. Un segmento que llega desde Internet sin una entrada que le corresponda no tiene destinatario y se descarta, de manera que un dispositivo detrás de un NAT no puede actuar como servidor sin una configuración explícita en el router, como el reenvío de puertos. Los firewalls con estado producen el mismo efecto, por diseño admiten el tráfico de respuesta de las conexiones iniciadas desde la red interna y bloquean las conexiones entrantes no solicitadas [3]. En ambos casos, quien administra el broker no controla esos equipos, que pertenecen al cliente o a su proveedor de acceso.

El modelo de MQTT se ajusta a esta restricción, ya que, como se vio en la Sección 2, todos los participantes son clientes y es el dispositivo quien abre la conexión TCP hacia el broker. Esa conexión saliente crea la entrada en el NAT y en el firewall, y a partir de ella el broker puede entregarle mensajes al dispositivo, incluidos los comandos que se originan en la nube, como segmentos de una conexión ya establecida. El único requisito sobre la red del cliente es que permita las conexiones salientes hacia la dirección y el puerto del broker. IANA, el organismo que asigna los números de puerto, tiene registrados para MQTT los puertos TCP 1883, sin cifrado, y 8883, sobre TLS, y, cuando una red restrictiva solo deja pasar el tráfico web, MQTT puede transportarse sobre WebSocket [6], a cambio de la sobrecarga de ese protocolo adicional.

La contracara de este esquema es que las entradas de la tabla no son permanentes. El NAT no tiene forma de saber si una conexión inactiva sigue en uso, por lo que la elimina tras un tiempo sin tráfico. Para las conexiones TCP establecidas, el IETF exige que ese tiempo no sea menor que dos horas y cuatro minutos [35]. Se trata, sin embargo, de un mínimo recomendado y no de una garantía, y un equipo que no lo cumple, o un firewall intermedio configurado con un plazo menor, descarta la entrada mucho antes. Si la entrada desaparece mientras la conexión está inactiva, ninguno de los extremos se entera. El siguiente segmento del broker hacia el dispositivo se descarta en el NAT, y el siguiente del dispositivo genera una entrada nueva, con otro puerto público, que el broker no reconoce como parte de ninguna conexión. El resultado es una conexión que figura como abierta en ambos extremos pero que ya no transporta datos, situación especialmente grave para un suscriptor que espera comandos y no transmite por iniciativa propia.

El keep-alive de MQTT, analizado en la Sección 1.1, resuelve también este problema. Al obligar al cliente a enviar algún paquete de control dentro de cada intervalo, aunque sea un PINGREQ, garantiza que la conexión nunca permanezca inactiva más que ese intervalo, y con ello refresca la entrada en cada NAT y firewall del trayecto, siempre que el intervalo sea menor que su tiempo de expiración. El keep-alive propio de TCP, descrito en la Sección 1.1, no alcanza, porque su intervalo es demasiado largo frente a los plazos de muchos equipos intermedios. Por eso el mantenimiento de la conexión se resuelve en la capa de aplicación, donde el intervalo es configurable por cada cliente. La especificación de MQTT indica que su valor depende de la aplicación y que suele ser de algunos minutos [6]. Al compromiso entre detección y tráfico de la Sección 1.1 se suma, en los dispositivos móviles, el gasto de energía, porque cada PINGREQ obliga a activar la radio.

2.2. Ruteo interno

Entre el dispositivo y el broker, cada datagrama atraviesa la red local del cliente, la de su proveedor de acceso, eventualmente las de uno o más proveedores de tránsito y la del proveedor de nube que aloja el broker, es decir, redes administradas por organizaciones distintas. Internet organiza esas redes en sistemas autónomos (AS), conjuntos de routers bajo una misma administración, y divide el problema del encaminamiento en dos niveles. Dentro de cada AS, un protocolo de ruteo interno (IGP) calcula los caminos entre sus routers, mientras que entre AS es BGP el que intercambia la información de alcanzabilidad [3]. Para MQTT, ambos niveles son transparentes mientras funcionan. Su comportamiento importa cuando un enlace o un router falla y la red debe encontrar un camino alternativo, porque durante ese lapso los segmentos de la conexión se pierden. Esta subsección y la siguiente describen cómo y en cuánto tiempo se recupera cada protocolo.

OSPF (*Open Shortest Path First*) es un protocolo de estado de enlace. Cada router descubre a sus vecinos mediante paquetes Hello y describe sus enlaces, con el costo de cada uno, en anuncios de estado de enlace (LSA) que se inundan a todos los routers del área. Así, todos construyen la misma base de datos con la topología completa, y cada uno calcula, con el algoritmo de Dijkstra, el árbol de caminos más cortos con raíz en sí mismo [29]. Ante una falla, el router que la detecta inunda un LSA actualizado y cada router recalcula sus rutas de forma independiente, sin esperar a que lo hagan sus vecinos, lo que le da a OSPF una convergencia rápida. El factor que suele dominar es la detección. Si la falla deja la interfaz caída, el router la advierte de inmediato; si no, como cuando el enlace pasa por un switch intermedio, recién la advierte cuando deja de recibir paquetes Hello durante el RouterDeadInterval, que con los valores habituales de 10 segundos entre Hellos es de 40 segundos [29]. Mecanismos como BFD (*Bidirectional Forwarding*

Detection) reducen esa detección a fracciones de segundo [36]. OSPF admite además dividir el AS en áreas, de modo que la inundación y el recálculo se limitan al área afectada [3].

RIP (*Routing Information Protocol*), en cambio, es un protocolo de vector distancia basado en el algoritmo de Bellman-Ford. Cada router conoce solo las distancias, medidas en saltos, que le anuncian sus vecinos, y les envía su tabla completa cada 30 segundos [37]. La métrica admite como máximo 15 saltos, ya que el valor 16 representa un destino inalcanzable, lo que limita el tamaño de las redes en las que puede usarse. Su convergencia es lenta: una ruta se da por perdida recién después de 180 segundos sin anuncios que la confirmen, y la noticia de la falla avanza de a un router por intercambio. Además, como los vecinos pueden seguir anunciando caminos que pasaban por el enlace caído, la métrica de la ruta puede crecer de a poco hasta llegar a 16, el problema conocido como conteo hasta el infinito [3]. El horizonte dividido con reversa envenenada y las actualizaciones disparadas atenúan el problema, pero no lo eliminan en los bucles de tres o más routers [37].

EIGRP (*Enhanced Interior Gateway Routing Protocol*), desarrollado por Cisco y publicado como RFC informativo, es un protocolo de vector distancia avanzado que evita estas limitaciones mediante el algoritmo DUAL [38]. Usa una métrica compuesta, que por defecto combina ancho de banda y retardo, y envía actualizaciones solo cuando hay cambios. Para cada destino registra, además del sucesor (el vecino por el que pasa la mejor ruta), los sucesores factibles: vecinos cuya distancia anunciada es menor que la mejor distancia que el router conoce hacia ese destino, condición que garantiza que el camino a través de ellos no forma un bucle. Si el sucesor falla y existe un sucesor factible, el router conmuta a él de forma local e inmediata; solo si no lo hay pasa la ruta a estado activo y consulta a sus vecinos, lo que demora la convergencia [38]. Para detectar vecinos caídos, en los enlaces de alta velocidad envía Hellos cada 5 segundos con un tiempo de espera de 15 segundos [38]. La Tabla 7 resume las diferencias entre los tres protocolos.

Tabla 7: Protocolos de ruteo interno, con los temporizadores predeterminados de sus especificaciones [29], [37], [38].

	RIP	**OSPF**	**EIGRP**
Tipo	Vector distancia	Estado de enlace	Vector distancia avanzado
Algoritmo	Bellman-Ford	Dijkstra	DUAL
Métrica	Saltos (máximo 15)	Costo por enlace	Compuesta (ancho de banda y retardo)
Actualizaciones	Tabla completa cada 30 s	LSA ante cambios y refresco cada 30 min	Solo ante cambios
Detección de una falla no visible en la interfaz	180 s sin anuncios	40 s sin Hellos	15 s sin Hellos
Recuperación	Propagación de a un salto; posible conteo hasta el infinito	Recálculo independiente en cada router del área	Inmediata con sucesor factible; si no, consulta a los vecinos

Para una conexión MQTT, estas diferencias se traducen en el tiempo durante el cual sus segmentos se pierden: desde fracciones de segundo, cuando la falla se detecta en la interfaz o con BFD, hasta decenas de segundos con los temporizadores predeterminados de OSPF y EIGRP, y varios minutos en RIP. Qué ocurre con la conexión según la duración de ese lapso se analiza en la Sección 2.4.

2.3. Ruteo externo

El broker no suele estar en el mismo AS que el dispositivo. Un robot conectado a la red de un cliente llega a Internet a través del AS de su proveedor de acceso, el broker se aloja en el AS de un proveedor de nube y entre ambos puede haber uno o más proveedores de tránsito. El camino entre AS lo determina BGP (*Border Gateway Protocol*), el protocolo de ruteo externo de Internet [39].

BGP es un protocolo de vector de caminos. Los routers de borde de AS vecinos establecen sesiones eBGP sobre conexiones TCP y se anuncian prefijos acompañados de atributos, entre ellos el AS-PATH, la lista de AS que atravesó el anuncio, que permite descartar las rutas que contienen al propio AS y evitar así los bucles. Dentro de cada AS, las sesiones iBGP distribuyen esas rutas a los demás routers [3], [39]. A diferencia de un IGP, BGP no busca el camino más corto. Cada AS elige entre las rutas disponibles según sus políticas, que reflejan relaciones comerciales, como la de no ofrecer tránsito gratuito a otro proveedor, y solo después considera la longitud del AS-PATH y otros criterios de desempate [3]. En consecuencia, el trayecto entre un dispositivo y el broker no es necesariamente el más corto ni el de menor latencia, y puede cambiar por decisiones administrativas que no controlan ni el dispositivo ni el operador del broker.

La convergencia de BGP es mucho más lenta que la de un IGP. Un router detecta la caída de un vecino cuando expira el tiempo de espera de la sesión, para el que la especificación sugiere 90 segundos, con mensajes KEEPALIVE cada 30 segundos. Además, para limitar la cantidad de actualizaciones, un router espera un intervalo mínimo entre anuncios sucesivos hacia un mismo vecino, con un valor sugerido de 30 segundos entre vecinos externos [39]. Cuando un prefijo deja de ser alcanzable por un camino, los routers no descubren de inmediato cuál es la alternativa válida, sino que van probando caminos más largos que todavía figuran en sus tablas, cada uno demorado por ese intervalo. En mediciones sobre Internet, Labovitz et al. observaron que la conmutación a un camino alternativo entre dominios demoraba en promedio tres minutos, y que algunas recuperaciones provocaban oscilaciones de las tablas de ruteo de hasta 15 minutos [40].

Para el stack estudiado, esto tiene dos consecuencias. La primera es que una falla entre AS puede dejar al dispositivo sin camino hacia el broker durante un tiempo comparable o mayor que el intervalo de keep-alive de MQTT, lo que fuerza el cierre de la conexión y la reconexión del cliente. La segunda es que la mitigación está, en su mayor parte, fuera del alcance de quien diseña el sistema. El único factor que controla es la ubicación del broker: alojarlo en una región del proveedor de nube cercana a los dispositivos reduce la cantidad de AS del trayecto y, con ella, la exposición a la convergencia de BGP.

2.4. Convergencia y conexiones de larga duración

Las subsecciones anteriores describieron cuánto tarda la red en encontrar un camino alternativo después de una falla. Mientras tanto, el trayecto entre el dispositivo y el broker se comporta como un agujero negro: los routers que todavía apuntan al enlace caído descartan los segmentos, y durante la reconvergencia pueden formarse bucles transitorios, en los que los segmentos circulan hasta que se agota su tiempo de vida (TTL) [41]. Ni TCP ni MQTT reciben aviso de lo que ocurre, porque el ruteo opera en la capa de red y las direcciones de los extremos no cambian. Lo único que observa el emisor es que sus segmentos dejan de ser confirmados. Lo que le pasa a la conexión depende, entonces, de cómo se compara la duración del corte con dos temporizadores de los extremos: el de retransmisión de TCP y el keep-alive de MQTT.

TCP reacciona a la falta de confirmaciones con su temporizador de retransmisión (RTO). Cuando este vence, retransmite el segmento más antiguo sin confirmar y duplica el RTO, de modo que los intentos sucesivos quedan cada vez más espaciados [42]. Con un RTO inicial de 1 s, los reintentos ocurren aproximadamente a 1, 3, 7, 15, 31 y 63 s del primer envío. Esto tiene una consecuencia poco intuitiva, ya que la conexión no se recupera en el instante en que la red converge, sino en el reintento siguiente. Si el camino se restablece a los 20 s, por ejemplo, el próximo intento sale recién a los 31 s, y la demora total resulta un 50 % mayor que el propio corte; en el peor caso, puede llegar casi al doble. Por otra parte, TCP tolera cortes largos antes de abandonar la conexión: la especificación exige que siga reintentando durante al menos 100 s [28], y con los valores predeterminados de Linux, el sistema operativo del servidor de los experimentos, el plazo es de unos 15 minutos [43].

En la práctica, quien suele cerrar primero la conexión es el keep-alive de MQTT. Si el broker no recibe ningún paquete de control del cliente durante una vez y media el intervalo de keep-alive, debe cerrar la conexión, y el cliente que no recibe respuesta a un PINGREQ en un tiempo razonable también debería cerrarla [6]. Con el intervalo de 60 s del Experimento 1, el broker da por caído a un cliente tras 90 s sin recibir nada de él. El umbral efectivo es incluso menor, porque durante el corte el PINGREQ queda detrás de los segmentos pendientes y depende de los mismos reintentos. Si los reintentos caen a los 63 y a los 127 s, un corte de algo más de un minuto ya alcanza para que el broker cierre la conexión, aunque la red haya convergido antes de los 90 s. Así, según la duración del corte se distinguen tres regímenes, que resume la Tabla 8.

Tabla 8: Efecto de un corte del camino sobre una conexión MQTT según su duración. Con el keep-alive de 60 s del Experimento 1, el umbral es de 90 s, o algo menos por el espaciamiento de los reintentos de TCP.

Duración del corte	**Ejemplos**	**Conexión**	**Efecto sobre los mensajes**
Unos pocos segundos	IGP con detección en la interfaz o con BFD; EIGRP con sucesor factible	Se mantiene; TCP retransmite una o dos veces	Llegan todos, con más latencia; QoS 1 puede duplicarlos
Menor que 1,5 veces el keep-alive	OSPF o EIGRP con detección por ausencia de Hellos	Se mantiene, con reintentos cada vez más espaciados	Llegan todos, juntos y con una demora del orden del corte
Mayor que 1,5 veces el keep-alive	RIP con detección por tiempo; BGP ante el retiro de una ruta	El broker la cierra y publica la última voluntad; el cliente se reconecta	Se pierden los de QoS 0 en tránsito; los de QoS 1 y 2 se reenvían si se retoma la sesión

Cortes de hasta unos pocos segundos. Son los que produce un IGP cuando la falla se detecta en la interfaz o mediante BFD, o EIGRP cuando cuenta con un sucesor factible. TCP los absorbe con uno o dos reintentos y la única consecuencia es una demora adicional del orden del RTO. Es el único régimen que cubrió el Experimento 1, de forma indirecta, con pérdidas aleatorias en lugar de cortes, y en él la entrega fue completa, aunque la cola de la latencia creció por los reintentos de TCP (Tabla 15). En este régimen, QoS 1 no mejora la entrega y agrega duplicados (Sección 1.3).

Cortes menores que una vez y media el keep-alive. Corresponden, por ejemplo, a una falla que OSPF o EIGRP detectan por la ausencia de Hellos con sus temporizadores predeterminados (Tabla 7). La conexión sobrevive, pero las lecturas generadas durante el corte se acumulan sin confirmar en el dispositivo y llegan juntas cuando el camino se restablece, con una demora del

orden del corte más la espera hasta el reintento siguiente. Ningún mensaje se pierde, cualquiera sea el nivel de QoS, pero su valor puede haber caducado: una lectura de telemetría que llega con decenas de segundos de atraso, o un comando que el robot recibe tarde, pueden resultar tan inútiles como uno perdido. La memoria del dispositivo impone además un límite, ya que un microcontrolador no puede acumular indefinidamente las lecturas pendientes.

Cortes mayores que una vez y media el keep-alive. Son los de RIP ante una falla que no se ve en la interfaz, o los de BGP cuando se retira una ruta, que suelen durar minutos (Sección 2.3). El broker cierra la conexión y, como el cierre es anormal, publica el mensaje de última voluntad del cliente, con lo que los demás participantes se enteran de la caída (Sección 2). Cuando el camino se restablece, el siguiente segmento del dispositivo llega a un broker que ya no reconoce la conexión y que responde con un *reset*, de modo que el cliente también la da por cerrada y se reconecta. La reconexión tiene un costo: un nuevo handshake TCP y TLS, que en el Experimento 1 constituye el costo fijo por conexión (Tabla 18) y que al ESP32 le insume unos 3 s de cómputo. Lo que se pierde depende del nivel de QoS y de la sesión. Los mensajes de QoS 0 que estaban en tránsito se pierden sin aviso. Los de QoS 1 y 2 que no habían sido confirmados se reenvían al reconectarse, siempre que el cliente retome la sesión anterior y el broker la haya conservado [6], y en ese caso el broker conserva también las suscripciones y los mensajes que llegaron para el cliente durante el corte. Es en este régimen donde los niveles de QoS y las sesiones persistentes aportan lo que TCP no puede dar, porque la confiabilidad de TCP termina con la conexión.

Hay un caso en el que la duración del corte deja de importar: cuando el camino nuevo atraviesa un NAT distinto del original, como cuando un proveedor conmuta a otro equipo de NAT de gran escala, o cuando el dispositivo cambia de red, por ejemplo de Wi-Fi a la red celular. La traducción de la conexión no existe en el nuevo NAT (Sección 2.1), por lo que la conexión se pierde aunque la red converja de inmediato, con las mismas consecuencias que en el tercer régimen.

El Experimento 1 no incluyó cortes de conexión, por lo que los dos últimos regímenes no se midieron allí (Sección 5.2). Se ensayan en el experimento de la Sección 5.3, sobre la misma plataforma, aplicando con `tc netem` una pérdida del 100 % sobre la interfaz de un nodo durante un intervalo fijo, con duraciones por debajo y por encima del umbral del keep-alive. Esas mediciones muestran además que un corte breve puede cerrar la conexión por el lado del cliente, antes del umbral de una vez y media el keep-alive que aplica el broker.

3. Ruteo por tópicos y ruteo IP

Las secciones anteriores describieron cómo la red lleva cada segmento desde el dispositivo hasta el broker. Entre el publicador y los suscriptores hay, sin embargo, una segunda decisión de encaminamiento, que se toma en la capa de aplicación. Un router que recibe un datagrama examina la dirección IP de destino de su encabezado y decide, a partir de su tabla, si lo entrega directamente, a qué otro router lo reenvía o si debe descartarlo e informar el error al origen [5, pp. 602–603]. El broker, al recibir un paquete PUBLISH, examina el nombre de tópico y decide, a partir de las suscripciones registradas, a qué clientes debe entregarlo. Ambos son mecanismos de encaminamiento basados en una tabla y en una clave que viaja en el encabezado, y compararlos permite ver qué ideas del ruteo IP reaparecen en MQTT y en qué puntos se aparta de ellas.

3.1. Dirección IP y nombre de tópico

La clave del router es la dirección IP de destino, un número de longitud fija (32 bits en IPv4) dividido en una parte de red y una parte de computador. La frontera entre ambas la indica la máscara: el router realiza una operación AND bit a bit entre la dirección y la máscara y obtiene así el número de red sobre el que decide [5, pp. 612–613]. Con el direccionamiento sin clases (CIDR), la parte de red puede tener cualquier longitud y se escribe como un prefijo de la forma `a.b.c.d/x`, donde `x` es la cantidad de bits que la componen [3, sec. 4.3.3], [44].

La clave del broker es el nombre de tópico, una cadena de texto UTF-8 de hasta 65 535 bytes, organizada en niveles separados por `/`, sin cantidad fija de niveles y en la que se distinguen mayúsculas de minúsculas [6]. Al igual que una dirección IP, se lee de izquierda a derecha, de lo general a lo particular: en el tópico `flota/robot1/bateria` de la Sección 2, cada nivel restringe el anterior, como cada bit adicional del prefijo restringe el bloque de direcciones. La diferencia está en quién define la jerarquía. En IP la fijan la asignación de bloques a proveedores y organizaciones, y la máscara se expresa en bits; en MQTT la define quien diseña el sistema, y cada nivel tiene el significado que la aplicación le asigne.

Hay, además, una diferencia de fondo en lo que identifica cada clave. Una dirección IP de unidifusión identifica una interfaz, es decir, un receptor, y es el emisor quien elige a quién se dirige. Un tópico identifica el contenido del mensaje y no a sus receptores: el publicador no sabe quién lo recibirá, y son los suscriptores quienes eligen qué reciben. Es el desacoplamiento en el espacio del modelo publicador/suscriptor descrito en la Sección 1.

3.2. Tablas de ruteo y tablas de suscripciones

Cada entrada de una tabla de ruteo asocia un destino con el siguiente nodo por el que debe salir el paquete, de modo que cada router solo necesita conocer el próximo salto hacia cada destino [5, p. 386]. Esas entradas las instala el administrador o las calculan los protocolos de ruteo, como OSPF, RIP, EIGRP y BGP, a partir del conocimiento que cada router tiene de la topología [5, p. 642], tal como se analizó en la Sección 2.2 y la Sección 2.3. Los hosts de destino no participan en su construcción.

La tabla del broker asocia, en cambio, filtros de tópico con sesiones de clientes, junto con las opciones de cada suscripción, como el nivel máximo de QoS. Sus entradas no las calcula ningún algoritmo a partir de una topología, sino que las crean los propios receptores con un paquete SUBSCRIBE y las eliminan con UNSUBSCRIBE o al terminar la sesión [6]. En un broker único no hay ningún protocolo que sincronice tablas entre nodos, y la topología se reduce al conjunto de conexiones abiertas.

Por eso, el análogo más cercano en IP no es el ruteo de unidifusión, sino la multidifusión. Allí, las estaciones usan IGMP (*Internet Group Management Protocol*) para suscribirse a un grupo de multidifusión y para abandonarlo, y los routers consultan periódicamente qué grupos interesan a las estaciones de cada red [5, p. 638]. El router no necesita saber qué estaciones pertenecen a cada grupo, sino solo si queda al menos un miembro activo en la red [5, p. 641], y debe traducir cada dirección de grupo a la lista de redes que contienen miembros [5, p. 636]. El paquete SUBSCRIBE de MQTT cumple la función del informe de pertenencia de IGMP y el tópico, la de la dirección de grupo. Como en la multidifusión, las fuentes no necesitan suscribirse para enviar [5, p. 638]. La diferencia es que las direcciones de grupo IP son identificadores planos, mientras que los tópicos son jerárquicos y admiten comodines.

La Tabla 9 muestra una tabla de cada tipo y la decisión que produce cada una ante un mismo caso: un datagrama dirigido a una dirección que coincide con varias entradas, y un mensaje cuyo tópico coincide con varias suscripciones.

Tabla 9: Tabla de reenvío de un router (izquierda) y tabla de suscripciones de un broker (derecha). Ante varias coincidencias, el router elige una sola entrada, la de prefijo más largo, mientras que el broker entrega el mensaje a todas.

Prefijo	**Interfaz de salida**
`10.1.0.0/16`	`eth1`
`10.1.2.0/24`	`eth2`
`0.0.0.0/0`	`eth0`
Destino `10.1.2.7`: coinciden las tres entradas y se reenvía solo por `eth2`.	

Filtro	**Cliente**
`flota/robot1/#`	`panel`
`flota/+/bateria`	`monitor-energia`
`#`	`registro`
Tópico `flota/robot1/bateria`: coinciden los tres filtros y se entrega a los tres clientes.	

3.3. Comodines y prefijos

El comodín `#` cumple en una suscripción el papel de un prefijo en una tabla de ruteo. El filtro `flota/robot1/#` abarca todos los tópicos que comienzan con esos dos niveles, incluido el propio `flota/robot1`, del mismo modo que `200.23.16.0/20` abarca todas las direcciones cuyos primeros 20 bits coinciden con los del bloque, y el filtro `#` por sí solo, que coincide con cualquier tópico, se asemeja a la ruta por defecto `0.0.0.0/0`. Por eso, la especificación exige que `#` sea el último carácter del filtro y ocupe un nivel completo [6]: como un prefijo, solo puede dejar libre el final de la clave.

La semejanza se rompe en la regla que se aplica cuando hay varias coincidencias. Una dirección de destino puede coincidir con más de una entrada de la tabla, y en ese caso el router aplica la regla del prefijo más largo: elige la entrada más específica y reenvía el datagrama solo por la interfaz asociada a ella [3, sec. 4.2.1]. Esa regla es la que permite que un bloque más específico «perfore» un bloque agregado más general sin que se pierda la agregación [3, sec. 4.3.3]. El broker, en cambio, no elige: entrega el mensaje a todos los clientes cuyos filtros coinciden. En la Tabla 9, la ruta por defecto solo recibe el tráfico que ninguna otra entrada captura, mientras que el cliente suscrito a `#` recibe todos los mensajes, además de los que reciben los otros dos. Cuando las coincidencias corresponden a un mismo cliente, como `flota/robot1/#` y `flota/+/bateria` suscritos por la misma sesión, la especificación de la versión 5.0 exige que el broker respete el mayor de los niveles de QoS de las suscripciones coincidentes y le permite enviar una sola copia o una por cada suscripción [6], de modo que el resultado depende de la implementación. Mosquitto, por ejemplo, ofrece una opción de configuración para elegir entre ambos comportamientos [45]. En la versión 5.0, el cliente puede además asociar un identificador a cada suscripción, que el broker incluye en los mensajes entregados para indicar qué suscripciones los originaron [6].

El comodín `+`, que reemplaza exactamente un nivel en cualquier posición del filtro, no tiene equivalente en el ruteo con prefijos. Un prefijo CIDR solo puede fijar los bits del comienzo de la dirección y dejar libres los del final, mientras que `flota/+/bateria` fija el primer y el tercer nivel y deja libre el del medio. Para expresar lo mismo con prefijos haría falta una entrada por robot. El análogo más cercano en la capa de red es el reenvío generalizado de las redes definidas por software, en el que cada entrada de la tabla de flujos compara varios campos del encabezado a la vez y cada campo puede contener comodines, como una dirección `128.119.*.*` [3, sec. 4.4.1]. Cada nivel de un tópico se comporta, en ese sentido, como un campo independiente del encabezado. Aun así, la regla de decisión vuelve a ser distinta, porque cuando

un paquete coincide con varias entradas de la tabla de flujos se aplica solo la de mayor prioridad [3, sec. 4.4.1].

El costo de la búsqueda también se resuelve de manera diferente. Un router de alta velocidad debe encontrar el prefijo más largo en nanosegundos, por lo que la búsqueda se implementa en hardware y con algoritmos más eficientes que un recorrido lineal de la tabla [3, sec. 4.2.1]. Un broker hace la búsqueda en software. EMQX, por ejemplo, organiza los filtros en un árbol de tópicos, una estructura jerárquica en la que compara cada tópico publicado con los filtros suscritos, incluidos los que tienen comodines [46], y su documentación advierte que el uso de comodines en un gran número de clientes puede afectar el desempeño [47].

3.4. Agregación y diseño del espacio de tópicos

La utilidad de un prefijo depende de cómo se asignaron las direcciones. Si los bloques se reparten de manera jerárquica, primero a los proveedores y luego a sus clientes, un único prefijo puede anunciar muchas redes, lo que se conoce como agregación de rutas y mantiene pequeñas las tablas [3, sec. 4.3.3], [44]. Stallings señala, a propósito de IPv6, que las direcciones pueden agruparse por jerarquías de red, por proveedores de acceso, por proximidad geográfica o por institución, y que esas agrupaciones conducen a tablas más pequeñas y a consultas más rápidas [5, p. 624]. Si las direcciones no siguen esa jerarquía, la agregación deja de ser posible.

Con los tópicos ocurre lo mismo. Un filtro con comodines solo puede seleccionar conjuntos de mensajes que la jerarquía de tópicos agrupa, y lo que no se codifica en el tópico no puede seleccionarse con ninguna suscripción. El orden y el contenido de los niveles equivalen, por lo tanto, al plan de direccionamiento de una red, y se fijan al diseñar el sistema. En el Experimento 1, cada nodo publica en `exp/tel/{ID del nodo}` y el backend recibe las lecturas de todos con una única suscripción a `exp/tel/+` (Sección 5.2), una entrada que cumple la función de una ruta agregada.

La API de posiciones de vehículos de Digitransit, el servicio de información de transporte público de Finlandia, lleva esta idea a la ubicación geográfica, de manera análoga al agrupamiento por proximidad que menciona Stallings. Su broker publica la posición de cada vehículo codificada como GTFS-Realtime en Protocol Buffers, y el tópico incluye, después de niveles como la ciudad, la línea, el viaje y el identificador del vehículo, una forma simplificada de *geohash*: un nivel con las partes enteras de la latitud y la longitud, en la forma `60;24`, seguido de tres niveles que intercalan el primer, el segundo y el tercer decimal de ambas coordenadas. Así, la posición (60,123; 24,789) se codifica como `60;24/17/28/39` [31]. Cada nivel adicional reduce diez veces el lado de la celda en cada coordenada, del mismo modo que cada bit adicional de un prefijo reduce a la mitad el bloque de direcciones, y la documentación destaca que este formato permite suscribirse con facilidad a zonas geográficas y recomienda especificar los niveles del tópico para limitar lo que se recibe [31]. A partir de la estructura documentada, por ejemplo, el filtro `/gtfsrt/vp/tampere/+/+/+/+/+/+/+/+/+/+/+/61;23/47/#` recibe los vehículos de Tampere ubicados en una celda de una décima de grado de lado, cualquiera sea su línea o su viaje.

El ejemplo muestra también los límites de la analogía. Una zona que no coincide con las celdas exige varias suscripciones, del mismo modo que un rango de direcciones que no está alineado con una potencia de dos exige varios prefijos. Y, a diferencia de una dirección IP, el tópico de un vehículo cambia a medida que se mueve, de modo que un suscriptor que sigue una zona ve aparecer y desaparecer vehículos sin que nadie modifique su suscripción. El costo de esa expresividad es el tamaño del encabezado: un tópico de unos veinte niveles se repite en cada

PUBLISH, un costo que el alias de tópico de la versión 5.0, descrito en la Sección 4, permite reducir.

3.5. Entrega a uno, a todos y a uno de un grupo

IP distingue tres tipos de direcciones: las de unidifusión, que identifican una única interfaz; las de multidifusión, cuyo paquete se entrega a todas las interfaces de un conjunto; y las de monodifusión (*anycast*), cuyo paquete se entrega a una sola interfaz de un conjunto, la más cercana según la métrica de los protocolos de ruteo [5, p. 624]. Una suscripción ordinaria de MQTT se comporta como la multidifusión, ya que el mensaje llega a todas las sesiones con un filtro coincidente. La versión 5.0 agrega las suscripciones compartidas, que usan filtros de la forma `$share/{nombre del grupo}/{filtro}` y entregan cada mensaje a una sola de las sesiones del grupo [6], lo que las asemeja a la monodifusión. La diferencia es el criterio de elección, ya que en IP decide la métrica de ruteo, mientras que la especificación de MQTT no establece cómo elige el broker y en la práctica suele usarse para repartir la carga entre varias copias de un mismo servicio, como el servidor de red de ChirpStack (Sección 4.4). Las suscripciones ordinarias que coinciden con el mismo tópico siguen recibiendo cada una su copia.

MQTT no tiene, en cambio, una forma de dirigir un mensaje a un cliente concreto. Para enviar una orden a `robot1` se publica en un tópico al que, por convención, solo ese robot se suscribe, como `flota/robot1/comandos`. La unidifusión surge así de la convención de nombres, y solo queda garantizada si el broker controla quién puede suscribirse a cada tópico. Mosquitto lo permite con listas de control de acceso cuyos patrones sustituyen el identificador o el usuario de cada cliente, de modo que una regla como `pattern read flota/%c/comandos` deja leer ese tópico solo al cliente cuyo identificador coincide con el nivel correspondiente [45]. Es el esquema que aplica OpenRobOps para que un robot no pueda leer los datos de otro (Sección 4.2.1), y cumple en el broker la función que en la red cumplen las listas de acceso de un router o un firewall, aunque filtra por tópico y no por dirección.

3.6. Límites del ruteo por tópicos

El ruteo por tópicos se apoya en el ruteo IP y no lo reemplaza. Con un único broker, la decisión por tópico se toma una sola vez, en un solo salto de la capa de aplicación, y no requiere mecanismos contra bucles como el tiempo de vida de IP. Pero el publicador y cada suscriptor tienen su propia conexión TCP con el broker, y cada una se encamina por IP de manera independiente, de modo que una falla de ruteo como las analizadas en la Sección 2.4 puede cortar la entrega a un suscriptor sin afectar a los demás. Como el broker es el punto por el que pasan todos los mensajes, el ruteo por tópicos funciona como una red superpuesta a Internet, cuyo único nodo intermedio es el broker.

Además, el broker, como el router, decide únicamente a partir del encabezado. El router no examina el segmento que transporta el datagrama, y el broker no interpreta la carga útil, una limitación del esquema basado en tópicos que ya se señaló en la Sección 4. Sparkplug muestra la consecuencia. Como las métricas viajan dentro de la carga útil de Protocol Buffers, el tópico solo identifica el nodo o el dispositivo que las publica, y una aplicación interesada en una única métrica debe recibir y decodificar todos los mensajes de ese nodo (Sección 4.3) [15]. Es el equivalente a pedirle a un router que encamine según el contenido de un segmento, algo que su tabla no puede expresar.

Por último, el broker no reconoce mensajes duplicados. Stallings describe la técnica de inundación, en la que cada nodo reenvía el paquete por todos sus enlaces salvo aquel por el que llegó,

de modo que el destino recibe varias copias, y por eso el paquete debe llevar un identificador único, como el nodo de origen y un número de secuencia, para que el destino conserve una sola y descarte el resto [5, p. 387]. La malla de Meshtastic funciona de este modo, y cuando varios puentes reciben el mismo paquete de la malla y lo publican, el broker trata cada copia como un mensaje independiente y las entrega todas, ya que no interpreta el identificador que viaja en la carga útil. Son los suscriptores los que deben descartar las copias, a partir del número que la radio de origen asigna a cada paquete (Sección 4.4) [13]. La multidifusión IP evita ese problema en los propios routers, que encaminan cada paquete según un árbol de expansión con raíz en la fuente, de modo que cada red reciba una sola copia [5, p. 637]. El broker no tiene un mecanismo equivalente porque no tiene forma de saber que dos PUBLISH son copias del mismo mensaje.

La Tabla 10 resume la comparación.

Tabla 10: Comparación entre el ruteo IP y el ruteo por tópicos de MQTT [3], [5], [6].

	Router IP	**Broker MQTT**
Clave de la decisión	Dirección IP de destino, de longitud fija	Nombre de tópico, cadena UTF-8 de niveles separados por `/`
Qué identifica la clave	El receptor (una interfaz)	El contenido del mensaje
Quién crea las entradas	El administrador o los protocolos de ruteo	Los suscriptores, con SUBSCRIBE
Agrupamiento de claves	Prefijos CIDR: solo fijan el comienzo de la dirección	`#` al final del filtro; `+` en cualquier nivel
Varias coincidencias	Se usa solo la de prefijo más largo	Se entrega a todas las suscripciones coincidentes
Destinos por mensaje	Uno en unidifusión; todos los miembros en multidifusión; uno del conjunto en monodifusión	Todos los suscriptores; uno por grupo en suscripciones compartidas
Alcance de la decisión	Salto a salto, en cada router del camino	Una vez, en el broker
Lo que no examina	El segmento de transporte y los datos	La carga útil
Duplicados	La multidifusión los evita con árboles de expansión	No los detecta; quedan a cargo de los suscriptores

4. Casos de uso

Las secciones anteriores analizaron la combinación de MQTT y Protocol Buffers a partir de las capas y protocolos que atraviesa un mensaje. Esta sección examina cómo la emplean sistemas reales de distintos dominios, con el fin de identificar qué propiedades de cada tecnología aprovechan y qué ventajas y limitaciones aparecen en la práctica. En cada caso se consideran, en la medida en que las fuentes lo permiten, el problema de comunicación que enfrenta, la combinación de protocolo y formato que adopta, los mecanismos de MQTT que utiliza, los aspectos de red involucrados y sus limitaciones. Los mecanismos analizados, como los niveles de QoS o el mensaje de última voluntad, existen tanto en la versión 3.1.1 de MQTT como en la 5.0, por lo que la versión solo se aclara cuando un caso depende de una función que agregó la 5.0.

No todos los casos combinan MQTT y Protocol Buffers. Algunos emplean MQTT con otro formato, y su inclusión permite distinguir el aporte de cada pieza, ya que, como se planteó en

la Sección 5, el protocolo de aplicación y el mecanismo de serialización constituyen decisiones independientes.

4.1. Mensajería instantánea

La mensajería en teléfonos móviles ofrece uno de los ejemplos más conocidos de uso masivo de MQTT. Al desarrollar su aplicación de mensajería Messenger, Facebook (hoy Meta) necesitaba entregar los mensajes a los teléfonos con baja latencia y sin agotar su batería, y recurrió a MQTT por tratarse de un protocolo diseñado para hacer un uso moderado del ancho de banda y de la energía. Sobre él construyó un mecanismo que mantiene una conexión persistente entre cada teléfono y los servidores de la empresa, con el que logró entregar en muchos casos los mensajes de un teléfono a otro en cientos de milisegundos en lugar de varios segundos [48]. La documentación pública no detalla la estructura de tópicos ni los niveles de QoS empleados, por lo que el análisis de este caso se limita al modelo de conexión, al formato de los mensajes y a los límites que la propia empresa encontró en el protocolo.

La evolución posterior de Messenger muestra cuánto influye el formato de los mensajes sobre esa conexión persistente. En su diseño original, la aplicación recibía primero un aviso breve de que había datos nuevos, y luego debía pedirlos mediante una consulta HTTPS, es decir, HTTP protegido con TLS, cuya respuesta en JSON era de gran tamaño. Cada mensaje nuevo exigía así un intercambio adicional y una transferencia voluminosa. Una reforma de 2014 cambió ese esquema. La aplicación pasó a obtener el estado completo de las conversaciones una sola vez, al iniciarse, y a recibir después por MQTT solo los cambios, a medida que se producían. Esa misma reforma reemplazó JSON por Thrift, una herramienta desarrollada originalmente en Facebook que, de forma análoga a Protocol Buffers, define los mensajes mediante un esquema y los codifica por lo general en formato binario [49]. El cambio de formato redujo aproximadamente a la mitad el tamaño de los mensajes [50]. Aunque el formato no es Protocol Buffers, la reducción es coherente con la ventaja de un formato con esquema frente a JSON descrita en la Sección 5.2.

Un artículo que la empresa presentó en 2021 muestra, en cambio, un límite de MQTT a gran escala. Allí se describe el sistema que mantiene actualizada la información que el usuario tiene en pantalla, como los comentarios nuevos en una entrada, los contactos que están en línea o el aviso de que alguien está escribiendo. Cada uno de esos elementos constituye un flujo de datos, es decir, una secuencia de actualizaciones que el teléfono recibe mientras esa información está a la vista, y un mismo teléfono puede tener muchos flujos abiertos a la vez. En MQTT, todos ellos pertenecerían a una única sesión, atendida por un único servidor. Si ese servidor se sobrecarga o falla, no es posible trasladar solo algunos flujos a otro servidor, sino que debe trasladarse la sesión completa. Por eso la empresa diseñó un protocolo propio que viaja por la misma conexión del teléfono, pero permite que cada flujo sea atendido por un servidor distinto y se recupere por separado [51].

4.2. Robótica

En la mensajería, la mayor parte del tráfico va del servidor al teléfono. En la robótica móvil, en cambio, el tráfico es intenso en ambos sentidos. Los robots móviles autónomos (AMR) y guiados (AGV) que se emplean en la industria y la logística, pero también en otras tareas como la limpieza, trabajan en flotas, es decir, en conjuntos de robots que comparten una misma instalación, y deben enviar de forma continua su posición y su estado, y al mismo tiempo recibir órdenes. Además, operan en depósitos o fábricas cuya red administra la empresa que los utiliza, y no siempre quien provee el software que los supervisa, por lo que suelen quedar detrás de NAT y firewalls que no aceptan conexiones entrantes. En este dominio se analizan tanto un servicio

comercial que combina MQTT con Protocol Buffers como estándares que adoptan MQTT con JSON para que robots y sistemas de distintos fabricantes puedan comunicarse entre sí.

4.2.1. InOrbit y OpenRobOps

InOrbit es una empresa que ofrece una plataforma primariamente basada en la nube para supervisar y operar flotas heterogéneas de robots móviles y dispositivos de distintos fabricantes. La empresa publica como código abierto un kit de desarrollo o SDK que desarrolladores pueden utilizar para incorporar dispositivos con la plataforma.

Con el SDK, cada robot abre su propia conexión MQTT 3.1.1 hacia el broker de la plataforma, cifrada con TLS. Antes de conectarse, obtiene la dirección del broker y el usuario y la contraseña con los que debe identificarse mediante una consulta HTTPS, de modo que HTTP se usa una sola vez, al iniciar, y MQTT para la conexión que permanece abierta. Cada tópico indica el robot y el tipo de dato que transporta, y por la misma conexión viajan en un sentido los datos del robot y en el otro las órdenes de la plataforma. La mayor parte de los datos se codifica con Protocol Buffers, con un tipo de mensaje para cada clase de dato, como la posición, el mapa del lugar o las imágenes de una cámara. A la reducción de tamaño que aporta la codificación se suman decisiones del propio SDK, como limitar la frecuencia con que se publican ciertos datos o eliminar puntos intermedios de los recorridos antes de enviarlos [52].

Los mecanismos de MQTT se eligen según la naturaleza de cada dato. La posición y los demás datos que cambian continuamente se publican con QoS 0, ya que si una muestra se pierde, la siguiente la reemplaza. El mapa, por el contrario, cambia rara vez, y como su pérdida dejaría a la plataforma sin él durante mucho tiempo, se publica con QoS 1 y como mensaje retenido, de modo que quien empieza a recibir datos del robot lo obtiene de inmediato. Algo similar ocurre con el estado de conexión. Al conectarse, el robot publica que está en línea mediante un mensaje retenido y registra en el mismo tópico un mensaje de última voluntad, también retenido. Si la conexión se interrumpe, el broker publica la última voluntad, que reemplaza al mensaje anterior, y quien se suscribe después recibe así el estado vigente.

InOrbit publicó además, bajo el nombre OpenRobOps, una versión de código abierto de la plataforma, que permite examinar también el lado del servidor, incluida la definición de los mensajes en archivos `.proto`. Como broker utiliza Mosquitto, un broker MQTT de código abierto, configurado para que cada robot tenga sus propias credenciales y solo pueda acceder a sus tópicos, de modo que un robot comprometido no pueda leer los datos de otros ni hacerse pasar por ellos. Su esquema ilustra la evolución descrita en la Sección 3.3, ya que los campos que dejan de usarse permanecen con una marca que indica que no deben emplearse, y la información nueva se agrega con números de campo no utilizados, de modo que los programas antiguos siguen siendo comprendidos [1].

Tanto el SDK de InOrbit como OpenRobOps utilizan MQTT 3.1.1, y esa elección explica parte de sus limitaciones. Esta versión no permite indicar en el mensaje qué formato tiene su contenido, por lo que el receptor deduce el tipo de cada mensaje a partir del tópico en el que llega. Esa dependencia es frágil, porque junto con los mensajes de Protocol Buffers conviven formatos más antiguos, como texto plano o documentos JSON guardados dentro de un campo de texto de un mensaje de Protocol Buffers, y como ningún mensaje declara su formato, cada programa que los recibe debe saber de antemano qué formato corresponde a cada tópico. La versión 3.1.1 tampoco ofrece una forma de asociar una respuesta con la solicitud que la originó. Para confirmar que una orden llegó, el robot devuelve una copia de cada mensaje que recibe, y la plataforma identifica a qué orden corresponde mediante un número que la propia orden

incluía. La versión 5.0 resuelve ambas necesidades con propiedades del protocolo. Una indica el tipo de contenido del mensaje, y otras dos permiten que quien solicita indique en qué tópico espera la respuesta y agregue un identificador que quien responde devuelve sin cambios, de modo que el solicitante puede asociar cada respuesta con su solicitud [6].

Otras limitaciones provienen de la arquitectura y no de la versión del protocolo. Abrir una conexión por robot, incluso cuando un mismo programa representa a varios, permite que el broker limite cada conexión a los tópicos de su robot y que el mensaje de última voluntad, que se registra una vez por conexión, anuncie la desconexión de un robot concreto. A cambio, cada robot exige su propia negociación TLS y sus propios paquetes de keep-alive, un costo que se multiplica cuando un solo equipo representa a muchos robots. Además, OpenRobOps ejecuta un único broker, ya que Mosquitto no permite que varias copias del broker funcionen como un solo sistema, repartiéndose los robots o reemplazando a una copia que falla, porque cada una guarda por separado sus mensajes retenidos y sus sesiones. Si ese broker se detiene, todos los robots quedan incomunicados, que es el punto de falla que la Sección 1 atribuye a la arquitectura centralizada.

4.2.2. VDA 5050 e ISO 21423

Sin un estándar común, cada fabricante de robots ofrece su propio sistema de control de flota, es decir, el software que asigna tareas a los robots y coordina sus movimientos. Para operar en conjunto robots de distintas marcas, una planta depende entonces de una plataforma como las de la Sección 4.2.1, que debe adaptarse a la interfaz particular de cada fabricante. VDA 5050 es una especificación, desarrollada por las asociaciones alemanas de la industria automotriz (VDA) y de fabricantes de maquinaria (VDMA), que define cómo se comunican los robots con un sistema de control de flota de cualquier fabricante. A diferencia del protocolo de la Sección 4.2.1, que una sola empresa define para su propia plataforma, se trata de un estándar abierto, que adopta MQTT con JSON y que InOrbit también admite [53].

La especificación exige como mínimo la versión 3.1.1 de MQTT y elige JSON porque permite agregar parámetros nuevos sin afectar a los equipos existentes y porque cada mensaje puede verificarse contra un esquema JSON, un documento aparte que describe los campos y los tipos que debe contener. A diferencia de lo que ocurre con Protocol Buffers, ese esquema no hace falta para leer el mensaje, sino solo para comprobar que es válido, de modo que cualquier fabricante puede interpretar los mensajes sin disponer de él. Los tópicos indican el nombre del protocolo, su versión, el fabricante, el número de serie del robot y el tipo de mensaje [54]. Como la versión forma parte del tópico, robots que usan versiones incompatibles pueden convivir en un mismo broker, ya que un sistema suscrito a los tópicos de una versión nunca recibe los mensajes de otra. Es una estrategia de evolución distinta de la de Protocol Buffers, que mantiene la compatibilidad dentro de un mismo esquema mediante los números de campo, como se describió en la Sección 3.3.

Con el objetivo declarado de reducir la sobrecarga de comunicación, la especificación establece QoS 0 para todos los mensajes, salvo los del tópico que informa si el robot está conectado, que usan QoS 1 y se publican como retenidos. En ese tópico, el robot registra además un mensaje de última voluntad que avisa la pérdida de la conexión. Sin embargo, la reducción de sobrecarga que se busca con el nivel de QoS no se extiende al contenido de los mensajes. La confirmación de QoS 1 agrega apenas unos pocos bytes, mientras que el mensaje con el que el robot informa periódicamente su posición, su batería y la tarea en curso repite en JSON, en cada envío, los nombres de todos sus campos, que es precisamente el costo que Protocol Buffers elimina, como se mostró en la Sección 5.2.

La norma ISO 21423, publicada en 2026, sigue un enfoque análogo. Especifica la comunicación entre robots móviles industriales, sistemas de control de flota y otros equipos del edificio que los robots necesitan accionar para desplazarse, como puertas automáticas o ascensores [55], y adopta también MQTT y JSON [55]. OpenRobOps la admite [1], lo que muestra que una misma plataforma puede ofrecer los dos formatos sobre MQTT, con Protocol Buffers en su protocolo propio y JSON en los estándares que comparte con otros fabricantes.

4.3. Internet de las Cosas industrial

En una planta industrial, donde máquinas y sensores conectados a la red envían datos de forma continua, un sistema de supervisión debe conocer en todo momento el estado de numerosos equipos que a menudo provienen de distintos fabricantes. Al igual que VDA 5050 e ISO 21423 para los robots móviles, este ámbito cuenta con un estándar abierto de interoperabilidad sobre MQTT, que cumple una función similar para los equipos de planta. A diferencia de aquellos, sin embargo, este estándar adopta Protocol Buffers en lugar de JSON. Eclipse Sparkplug, desarrollada por la Fundación Eclipse, una organización sin fines de lucro que mantiene proyectos de software abierto, define cómo se comunican sobre MQTT tres tipos de participantes. Los dispositivos son los equipos que generan los datos, como sensores, motores o las computadoras industriales que comandan las máquinas. Los nodos del *edge* o de borde, llamados así porque se ubican en el límite de la red, junto a las máquinas, reúnen los datos de esos dispositivos, que a menudo no pueden comunicarse por MQTT, y los publican en el broker. Las aplicaciones de supervisión, por último, se suscriben a esos datos para mostrarlos, almacenarlos o enviar órdenes. La especificación completa lo que MQTT deja abierto de manera deliberada, como la estructura de los tópicos, el formato de los mensajes y la forma de conocer qué equipos están en línea [15].

Cada tópico indica la versión de la especificación, un grupo de nodos, el tipo de mensaje, el nodo y el dispositivo. El tipo distingue los datos, las órdenes y los mensajes que anuncian la conexión y la desconexión. El contenido de los mensajes se codifica con Protocol Buffers, mediante un esquema común que representa los datos como una lista de mediciones, llamadas métricas, cada una con su tipo y su momento de registro. Como ese esquema es genérico y sirve para cualquier equipo, el nombre de cada métrica viaja como texto, lo que reintroduce dentro del mensaje binario el costo que Protocol Buffers evita al identificar los campos por número. Para reducirlo, la especificación permite identificar cada métrica con un número corto, llamado alias. El nodo informa la correspondencia entre nombres y alias una sola vez, en el mensaje que publica al conectarse y que se describe a continuación, y los mensajes de datos posteriores llevan solo el número. Con el mismo propósito, recomienda publicar un valor solo cuando cambia, y no de forma periódica.

El aspecto más elaborado de Sparkplug es la forma en que la aplicación de supervisión sabe qué nodos están en línea. Para ello se apoya en el mensaje de última voluntad y el keep-alive descritos en la Sección 2. Al conectarse, cada nodo publica un mensaje que anuncia su presencia y enumera todas las métricas que reportará, que la especificación llama certificado de nacimiento y que no tiene relación con los certificados digitales de TLS. Antes de publicarlo, registra como mensaje de última voluntad un certificado de muerte, que el broker publica si la conexión se interrumpe sin aviso. Al recibirlo, la aplicación de supervisión marca como desactualizados los datos de ese nodo, para que nadie tome los últimos valores recibidos como vigentes. La rapidez con que se detecta la caída depende, por lo tanto, del intervalo de keep-alive analizado en la Sección 1.1.

Este esquema presenta, sin embargo, un problema con el orden de llegada de los certificados. Como el broker solo detecta la caída de una conexión cuando vence el keep-alive, el nodo puede volver a conectarse antes de que eso ocurra. En ese caso, el certificado de muerte de la conexión anterior puede publicarse después del nuevo certificado de nacimiento, y la aplicación daría por desconectado a un nodo que está en línea. Para evitarlo, ambos certificados llevan un número que cambia en cada conexión, y la aplicación solo considera caído al nodo cuando el certificado de muerte corresponde al último nacimiento recibido.

Un problema distinto es la pérdida de datos. Como se publican con QoS 0, los mensajes que están en tránsito cuando se corta una conexión se pierden, aunque TCP sea confiable mientras la conexión existe. Para detectarlo, cada mensaje lleva un número de secuencia, y cuando la aplicación advierte que falta uno, le pide al nodo un nuevo certificado de nacimiento para reconstruir su estado completo. Se trata de un mecanismo de la aplicación que suple la confirmación que QoS 0 no ofrece, en línea con lo analizado en la Sección 1.3.

Las aplicaciones de supervisión anuncian también su propio estado, con un mensaje retenido y un mensaje de última voluntad. Así, los nodos saben si hay alguien recibiendo sus datos y pueden esperar a que la aplicación designada como principal, encargada de registrarlos, esté en línea antes de publicar, para no enviar datos que nadie almacenaría. La especificación exige además que cada conexión comience sin sesión persistente, de modo que el broker no acumula mensajes viejos para los clientes desconectados. Quien se desconecta pierde los datos publicados mientras tanto, pero recupera el estado actual, porque cada nodo vuelve a publicar su estado completo al reconectarse. Cuando se emplea la versión 5.0, este comportamiento se fija con propiedades exclusivas de esa versión, como la duración de la sesión una vez cerrada la conexión.

Estas decisiones tienen también costos. La especificación prohíbe publicar el certificado de nacimiento como mensaje retenido, por lo que una aplicación que se suscribe más tarde debe pedirle al nodo uno nuevo antes de poder interpretar los mensajes que usan alias, salvo que el broker implemente una extensión opcional que lo conserve. Además, como las métricas viajan dentro del contenido de los mensajes, el broker no puede distribuirlas por separado (Sección 3).

4.4. Internet de las Cosas sobre redes LoRa

Los casos anteriores suponen dispositivos conectados directamente a una red IP. LoRa es, en cambio, una tecnología de radio que permite comunicar dispositivos a varios kilómetros de distancia con muy bajo consumo de energía, a costa de una velocidad de transmisión muy reducida, y cuyos enlaces no transportan IP. Por eso, en las redes LoRa no puede emplearse MQTT directamente. Su interés para este trabajo radica en dos aspectos. Por un lado, muestran a Protocol Buffers en el enlace más restringido de todos los casos analizados. Por otro, muestran el papel de MQTT en el punto en que los datos de la radio ingresan a una red IP, donde un equipo intermediario debe hacerlos llegar a un servidor. Los dos proyectos que se describen a continuación, Meshtastic y ChirpStack, ilustran respectivamente el primer y el segundo aspecto.

4.4.1. Meshtastic

Meshtastic es un software de código abierto que se instala en pequeñas radios LoRa de bajo costo y que, junto con una aplicación para teléfonos, permite intercambiar mensajes de texto y datos de posición sin depender de redes celulares ni de Internet. Las radios forman una red mallada, en la que cada una retransmite los mensajes que recibe para que lleguen a radios fuera del alcance del emisor. Por radio, cada mensaje viaja con un encabezado binario fijo y un contenido codificado con Protocol Buffers, que no puede superar los 239 bytes [56], un margen en el que un formato textual como JSON resultaría difícil de sostener.

Una radio que además dispone de conexión a Internet puede actuar como puente entre la malla y un broker MQTT. Publica en el broker los mensajes que recibe de la malla y se suscribe a él para retransmitir en su malla los mensajes que otros puentes publican desde mallas lejanas, de modo que el broker une mallas que no tienen alcance de radio entre sí. Los usuarios de Meshtastic pueden organizar sus conversaciones en grupos, que el proyecto llama canales, y cada mensaje se publica encapsulado en otro mensaje de Protocol Buffers que agrega su canal y el identificador del puente [57]. Como los mensajes de la malla ya están codificados con Protocol Buffers, el puente los reenvía sin traducirlos [13].

Los mensajes de cada canal se cifran con una clave que comparten las radios de ese canal, de modo que solo ellas pueden leerlos. Al publicarlos en el broker, el puente puede conservar ese cifrado o enviarlos descifrados, y la configuración por defecto opta por lo segundo [58]. La diferencia es relevante porque el broker, como intermediario del modelo publicador/suscriptor, recibe el contenido de todos los mensajes. TLS, que también es opcional en la conexión con el broker, protege solo el tramo entre el puente y el broker, por lo que el broker y sus suscriptores siguen pudiendo leer los mensajes. Si el mensaje conserva el cifrado del canal, en cambio, solo pueden leerlo quienes tienen la clave.

La presencia de varios puentes en una misma malla muestra la diferencia entre dos formas de hacer llegar los mensajes, que se vinculan con la comparación entre ruteo por tópicos y ruteo IP de la Sección 3. La malla difunde cada mensaje a todas sus radios, de modo que varios puentes pueden recibirlo y publicarlo. El broker, que no interpreta el contenido, reenvía cada copia a todos los suscriptores, y son estos quienes deben descartar los duplicados (Sección 3). En el sentido inverso aparece otro problema. Los desarrolladores de Meshtastic mantienen un broker público, al que las radios se conectan por defecto y en el que confluye el tráfico de muchas mallas. Si ese tráfico se retransmitiera por cada malla, saturaría enlaces de radio de muy baja capacidad, por lo que los mensajes que llegan desde ese broker se entregan solo a las radios al alcance directo del puente.

4.4.2. ChirpStack

LoRaWAN es un protocolo que organiza las redes LoRa de manera distinta a Meshtastic. En lugar de una malla, los dispositivos transmiten por radio a uno o más equipos fijos, llamados *gateways*, que reenvían por Internet lo que reciben a un servidor central, llamado servidor de red, que verifica cada mensaje, descarta las copias repetidas y lo entrega al programa del usuario que utiliza esos datos [59]. ChirpStack es una implementación de código abierto de ese servidor, y permite seguir un dato desde la radio hasta ese programa a través de tramos con protocolos y formatos distintos.

Los *gateways* suelen reenviar los datos mediante un protocolo sobre UDP, definido por Semtech, el fabricante de los circuitos de radio LoRa, que representa la información en JSON. Según su propia especificación, se trata de un protocolo deliberadamente básico, que no verifica la identidad de los extremos ni retransmite los datagramas perdidos y que, por eso, está pensado para demostraciones o para redes privadas y confiables. El protocolo contempla además que el *gateway* se encuentre detrás de un NAT, por lo que el *gateway* envía periódicamente un mensaje para que el NAT mantenga abierta la asociación por la que le llegan los datos que el servidor envía hacia los dispositivos [60]. El mantenimiento de esas asociaciones se analizó en la Sección 2.1.

ChirpStack incluye un componente, llamado Gateway Bridge, que traduce ese protocolo a MQTT, con Protocol Buffers como codificación recomendada y JSON reservado para la depu-

ración [61]. La documentación recomienda ejecutarlo en el propio *gateway*, de modo que el tramo que atraviesa Internet aproveche la autenticación y el cifrado TLS de la conexión MQTT [62]. Se reemplaza así un protocolo sin confiabilidad ni seguridad por una conexión TCP persistente lo más cerca posible del origen de los datos. Los mensajes que el *gateway* publica incluyen, como campos de Protocol Buffers, información sobre cómo se recibió cada transmisión, como la frecuencia y la intensidad y la calidad de la señal [63], valores que en JSON repetirían sus nombres en cada mensaje.

En cuanto a los mecanismos de MQTT, el Gateway Bridge publica el estado de conexión del *gateway* como mensaje retenido y, cuando se ejecuta en el propio *gateway*, registra además un mensaje de última voluntad que avisa su desconexión [64]. Si se ejecutara en un servidor, su conexión MQTT seguiría abierta aunque el *gateway* dejara de funcionar, y la última voluntad no reflejaría esa caída. El servidor de red, por su parte, emplea la versión 5.0 y, si el broker lo admite, se suscribe a los mensajes de todos los *gateways* mediante una suscripción compartida, que reparte los mensajes entre varias copias del servidor de red que se ejecutan en paralelo [65].

Como en Meshtastic, una misma transmisión de radio puede ser recibida por varios *gateways*, pero en LoRaWAN es el servidor de red el que elimina los duplicados, para lo cual espera un breve intervalo a que lleguen todas las copias. Esa espera, sumada a la latencia del tramo MQTT, demora la respuesta que el servidor envía al dispositivo. Como el dispositivo, para ahorrar energía, solo enciende su receptor durante un breve lapso después de cada transmisión, una respuesta que llega tarde se pierde.

Hacia el programa del usuario, ChirpStack publica los datos en JSON por defecto, con Protocol Buffers como opción [66]. El formato se elige así según el tramo, con la codificación binaria donde se concentra el volumen de mensajes y JSON donde prima la facilidad de integración con otros sistemas.

4.5. Ventajas y limitaciones observadas

Los casos analizados, pese a pertenecer a dominios distintos, comparten el patrón de red de la Sección 2.1, en el que el dispositivo, o el equipo que lo representa, abre una conexión persistente hacia el broker y recibe por ella sus mensajes, entre ellos las órdenes. La mayoría informa además si está en línea mediante los mecanismos de MQTT, aunque con variantes. InOrbit, VDA 5050 y ChirpStack combinan un mensaje retenido con el mensaje de última voluntad, de modo que cualquier suscriptor conoce el estado vigente. Sparkplug aplica ese patrón solo a las aplicaciones de supervisión, mientras que los nodos vuelven a anunciar su estado con un nuevo certificado de nacimiento al reconectarse.

La elección del formato sigue también un criterio común. Protocol Buffers aparece en los tramos que concentran el volumen de mensajes, como los datos de los robots, los mensajes de los *gateways* LoRaWAN o las mediciones de los equipos industriales, y en los enlaces de radio más restringidos. JSON, por el contrario, aparece en los datos que se entregan a programas de otras empresas y en estándares compartidos entre fabricantes, como VDA 5050 o ISO 21423, en los que importa que cualquiera pueda leer los mensajes sin disponer de un esquema, aunque Sparkplug muestra que un estándar abierto también puede adoptar Protocol Buffers cuando el volumen de datos lo justifica.

En los datos que se publican con alta frecuencia predomina QoS 0, ya que cada muestra queda reemplazada por la siguiente. Los casos se apoyan en la entrega fiable de TCP y, cuando la necesitan, agregan mecanismos propios, como los números de secuencia de Sparkplug. QoS 1 y los mensajes retenidos quedan reservados para el estado de conexión y para los datos que

cambian con poca frecuencia, como el mapa de un robot. La reducción del volumen tampoco depende solo de la codificación, ya que Protocol Buffers se combina con decisiones de la aplicación, como limitar la frecuencia de publicación o publicar solo los cambios. Esas optimizaciones pueden tener, sin embargo, un costo, como muestra Sparkplug, donde una aplicación que se suscribe tarde debe pedir un nuevo certificado de nacimiento antes de poder interpretar los datos que usan alias. En cuanto a la magnitud del beneficio, la única medición realizada sobre un sistema en funcionamiento real proviene de Messenger, que redujo a la mitad el tamaño de los mensajes al reemplazar JSON por Thrift, un formato con esquema análogo a Protocol Buffers.

Los casos también exponen los límites del stack. Del lado de MQTT, el broker conserva estado, y cuando la implementación no permite repartirlo entre múltiples réplicas, como ocurre con Mosquitto, se convierte en el punto de falla de todo el sistema. A gran escala, la sesión resulta una unidad demasiado grande para repartir la carga entre servidores y recuperarse ante fallas, y cuando cada dispositivo mantiene su propia conexión, cada uno asume el costo de su negociación TLS y de su keep-alive. El ruteo por tópicos, por su parte, no permite filtrar según el contenido de los mensajes. Del lado de Protocol Buffers, todo programa que recibe los datos debe disponer del esquema, y la convivencia con formatos más antiguos debilita el acuerdo que establece el esquema.

Por último, varias implementaciones permanecen en MQTT 3.1.1 y resuelven en la aplicación o mediante el tópico funciones que la versión 5.0 incorpora, como indicar el formato de los mensajes o asociar cada respuesta con su solicitud, según se vio en la Sección 4.2. Entre los casos analizados, el servidor de red de ChirpStack es el que más aprovecha las funciones exclusivas de esa versión, mientras que Sparkplug solo las emplea para fijar el comportamiento de la sesión. Ninguno de los casos cuya implementación pudo examinarse utiliza las propiedades que indican el formato o asocian respuestas con solicitudes, lo que sugiere que, al menos en la muestra analizada, las ventajas de la versión 5.0 todavía no se reflejan plenamente en la práctica. En conjunto, los casos indican que el stack aporta sus mayores ventajas en los tramos de alta frecuencia y en los dispositivos que operan detrás de redes ajenas, mientras que sus limitaciones se concentran en el broker y en la distribución del esquema.

5. Diseño experimental

Las secciones anteriores analizaron el stack de forma cualitativa. Esta sección lo contrasta con mediciones propias, en dos experimentos que comparten la misma plataforma. En un escenario de telemetría, el experimento de la Sección 5.2 compara MQTT con HTTP y mide el costo en bytes y en latencia de cada camino. El de la Sección 5.3, en cambio, usa solo MQTT con Protocol Buffers en una aplicación en la que cada nodo necesita el dato de todos los demás, y ensaya los mecanismos del stack ante pérdidas, cortes de conexión y caídas del broker. Cada experimento se presenta con la misma estructura, que recorre su planteo, su implementación, sus resultados y el análisis de esos resultados.

5.1. Plataforma común

Los dos experimentos corren sobre el mismo entorno, montado en una sola computadora (distinta en cada experimento), donde cada nodo es un ESP32 emulado con QEMU.

QEMU es un programa que permite ejecutar en una computadora el mismo código que correría en la placa, y se usa la versión que mantiene Espressif, el fabricante del ESP32, que emula su procesador, su memoria y varios periféricos [67]. El programa de cada nodo, o *firmware*, está

escrito en C con ESP-IDF, el entorno de desarrollo de Espressif [68]. Como el ESP32 emulado no tiene Wi-Fi, QEMU le provee una placa Ethernet emulada. Cada instancia se conecta a una interfaz TAP propia, una interfaz de red virtual de Linux que intercambia las tramas Ethernet con un programa, en este caso QEMU, en lugar de enviarlas por un medio físico [69]. Todas se unen en un puente de Linux, que funciona como un conmutador por *software* [70] y las comunica con el broker Mosquitto, que corre en la misma computadora. Esta disposición permite aplicar a cada nodo condiciones de red propias con `netem` y capturar todo el tráfico en un único punto.

La emulación impone algunas restricciones. Los tiempos de ejecución no son los de una placa real, porque dependen de la carga de la computadora anfitriona. Cuando llega una difusión, todas las instancias compiten por la CPU, y el retardo de entrega crece alrededor de 0,3 ms por cada nodo agregado. Además, cada instancia ocupa unos 170 MB de memoria, lo que limita la cantidad de nodos que se pueden emular. El hardware real que reemplazaría a lo emulado, y su costo, se releva en la Sección 6.

5.2. Experimento 1: MQTT contra HTTPS para telemetría

Este experimento es el caso base de la parte experimental. Mide cuánto cuesta, en bytes y en latencia, enviar una lectura de un sensor por MQTT y por HTTP, y separa ese costo en sus componentes, que son la conexión, el protocolo de aplicación, el cifrado y la serialización. Sus resultados sirven de referencia para el segundo experimento (Sección 5.3), que aplica el stack MQTT y Protocol Buffers a la sincronización de osciladores acoplados, una aplicación en la que todos los nodos intercambian mensajes entre sí.

5.2.1. Conceptos previos

Además de lo presentado en el marco teórico, el análisis de este experimento se apoya en dos mecanismos de TCP y en el protocolo con el que se sincronizan los relojes de los nodos. Se resumen a continuación porque explican buena parte de los resultados.

Algoritmo de Nagle. Una aplicación entrega datos a TCP con llamadas a `send()` sobre un *socket*, y TCP decide cómo repartirlos en segmentos. Si cada llamada saliera de inmediato en su propio segmento, una aplicación que escribe de a pocos bytes llenaría la red de segmentos casi vacíos, en los que los 40 bytes de encabezados TCP/IP pesan más que los datos. El algoritmo de Nagle evita eso, porque, mientras haya datos enviados sin confirmar, TCP no envía un segmento más pequeño que el tamaño máximo, sino que acumula los datos nuevos hasta recibir el ACK pendiente [71]. Si no hay nada sin confirmar, el segmento sale enseguida. Así, una escritura aislada no se demora, pero la segunda de dos escrituras seguidas espera el ACK de la primera. La especificación vigente de TCP recomienda implementar Nagle, pero exige que cada aplicación pueda desactivarlo en una conexión [71], opción que en la interfaz de *sockets* se llama `TCP_NODELAY`.

ACK retardado. Del lado del receptor existe una optimización equivalente. En lugar de confirmar cada segmento apenas llega, el receptor puede esperar, hasta 500 ms según la especificación, para enviar el ACK junto con los datos de su respuesta y ahorrar un paquete [28]. Si el temporizador vence antes de que haya una respuesta, el ACK sale solo. Cada mecanismo tiene sentido por separado, pero combinados pueden bloquearse entre sí.

Sincronización de relojes con SNTP. Para medir la latencia extremo a extremo, el nodo y el servidor tienen que compartir la hora. SNTP (*Simple Network Time Protocol*), una versión reducida de NTP para dispositivos con pocos recursos, sincroniza el reloj de un cliente con el de

un servidor. Para ello, el cliente envía una consulta, el servidor responde con su hora y, con las marcas de tiempo de ambos mensajes, el cliente estima el tiempo de viaje y lo descuenta [72].

5.2.2. Planteo

Ocho placas ESP32 emuladas envían telemetría periódica a un servidor por dos caminos, que son MQTT, a través de un broker, y HTTP, con un POST por lectura. El contenido de cada lectura es idéntico en ambos caminos y se serializa en JSON o en Protocol Buffers, lo que permite separar el efecto del transporte del de la serialización.

A diferencia del experimento de sincronización, el caso de uso es neutral, ya que enviar lecturas de los nodos a un servidor es algo que ambas arquitecturas resuelven de forma razonable, sin que el modelo publicador/suscriptor le dé de antemano una ventaja a MQTT.

Motivación
Este experimento corresponde al último de los objetivos específicos (Sección 1), que consiste en medir el costo en bytes y en latencia de MQTT y Protocol Buffers frente a HTTPS y JSON, y su comportamiento ante pérdidas y latencia. También aporta datos al objetivo que analiza MQTT sobre la capa de transporte, en lo que respecta a TCP y TLS. En el marco teórico (Sección 5) se planteó que MQTT envía mensajes más livianos y con menor latencia que HTTP, por su encabezado fijo de dos bytes y su conexión persistente, y que Protocol Buffers produce mensajes más pequeños que JSON, por su codificación binaria. Esas ventajas surgen de comparaciones cualitativas y de estudios con otros mensajes y plataformas. El experimento las mide sobre un mismo sistema. Para atribuir cada parte de la diferencia a su causa, incluye dos variantes de HTTP, corridas con y sin TLS y las dos serializaciones.

Fundamento
La diferencia principal entre ambos caminos no está en el contenido de la lectura, sino en cómo cada protocolo usa la conexión TCP. MQTT mantiene una conexión persistente por nodo, con un único handshake TLS y un encabezado binario de dos bytes, mientras que HTTP agrega a cada mensaje una línea de solicitud, encabezados en texto y una respuesta (Tabla 3). Por eso se prueban dos variantes de HTTP, una con una conexión nueva por lectura y otra con una conexión reutilizada. La segunda es la comparación justa con MQTT; la primera representa el peor caso, el de un dispositivo que duerme entre envíos y abre una conexión cada vez.

Preguntas de investigación
1. ¿Cuántos bytes cuesta enviar una lectura por MQTT y por HTTPS, contando encabezados y handshakes?
2. ¿Cómo se comparan la latencia extremo a extremo y la tasa de entrega de ambos?
3. ¿Cómo se degradan ambos protocolos con pérdida y latencia en la red?
4. ¿Qué parte de la diferencia viene de la conexión persistente y qué parte de TLS?
5. ¿Cuánto cambia el costo al pasar de JSON a Protocol Buffers en cada transporte?

Hipótesis
Antes de las corridas se formularon las siguientes hipótesis, que se contrastan con los resultados al final del análisis (Tabla 20):

- MQTT envía menos bytes por lectura que HTTPS, y la diferencia crece cuando el período de envío es corto.
- HTTPS con conexión reutilizada reduce gran parte de la diferencia, pero los encabezados HTTP siguen pesando más que el encabezado MQTT.

- HTTPS con una conexión por solicitud es el caso más costoso, sobre todo con TLS y latencia alta, porque cada handshake suma idas y vueltas.
- Con pérdida, MQTT QoS 1 mantiene la entrega a costa de duplicados, mientras que MQTT QoS 0 pierde lecturas.
- Protocol Buffers reduce la carga útil en ambos caminos, pero su efecto sobre el total es mayor en MQTT, donde la carga útil es la mayor parte del mensaje. En HTTPS, los encabezados diluyen el ahorro.

5.2.3. Implementación

Arquitectura

El experimento corre sobre la plataforma común (Sección 5.1), con 8 nodos. Cada uno ejecuta un firmware real, por lo que los bytes que se miden son los que producen, sin modificaciones, las bibliotecas que usaría una placa física, que son la pila TCP/IP lwIP, mbedTLS para TLS, esp-mqtt como cliente MQTT y `esp_http_client` como cliente HTTP.

Se usaron interfaces TAP y no la red de usuario de QEMU porque esta termina las conexiones TCP del nodo dentro del propio emulador, de modo que la red no vería los paquetes tal como los arma el ESP32. Sobre cada interfaz se aplica `tc netem` en ambos sentidos para inyectar pérdida y latencia, y `dumpcap` captura en el puente todo el tráfico de la corrida (Figura 2).

Del lado del servidor hay dos procesos, el broker Mosquitto y un backend escrito en Go. El backend recibe los POST por HTTP y por HTTPS y responde `204 No Content`. Además, está suscrito al broker por una conexión local que no pasa por el puente y, por lo tanto, no entra en la captura. Las lecturas de ambos caminos terminan en la misma función de registro, que anota la hora de llegada, de modo que la latencia se calcula igual para los dos. El backend también funciona como servidor SNTP, para que los nodos sincronicen su reloj con el de la computadora anfitriona.

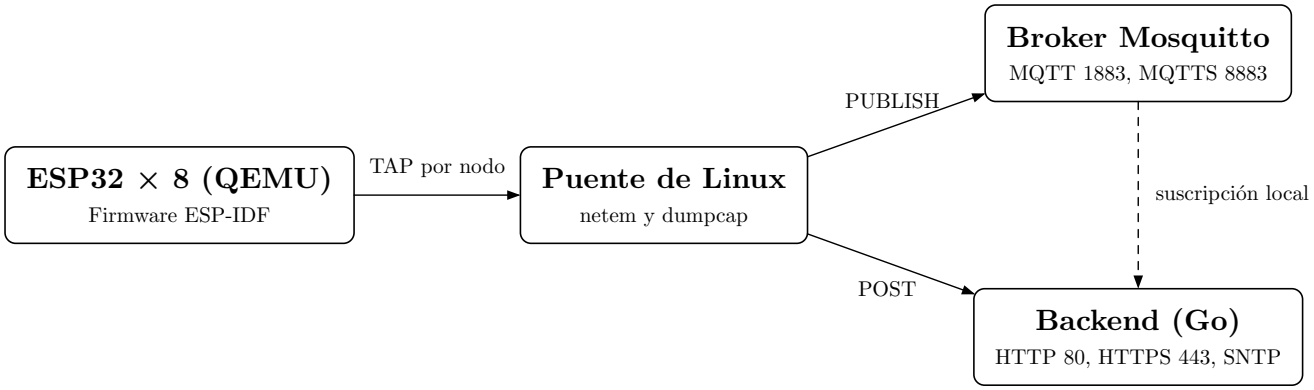


Figura 2: Arquitectura del experimento. La captura se hace en el puente, por donde pasa todo el tráfico de los nodos; la suscripción del backend al broker queda fuera de ella.

Mensaje y caminos

Todas las lecturas usan el mismo mensaje `Telemetry`, definido en un archivo `.proto` compartido por el firmware, que lo codifica con nanopb, y por el backend:

```
message Telemetry {
  uint32 esp_id = 1;
  uint32 seq    = 2;
  uint64 t_us   = 3;  // hora de creación en el nodo (µs)
  float  value  = 4;
  uint32 status = 5;
}
```

Codificado con Protocol Buffers, el mensaje ocupa entre 16 y 19 bytes (18 en promedio), porque `seq` es un entero de longitud variable. En JSON, con los mismos nombres de campo y sin espacios, ocupa unos 72 bytes, cuatro veces más.

La lectura se envía por uno de los cuatro caminos de la Tabla 11. En HTTP, el formato se indica en el encabezado `Content-Type` (`application/x-protobuf` o `application/json`).

Tabla 11: Transportes comparados.

Transporte	**Qué hace el nodo**
MQTT QoS 0	PUBLISH a `exp/tel/{esp}` con QoS 0, sobre una conexión persistente abierta antes de empezar a medir
MQTT QoS 1	Igual, con QoS 1: el broker confirma cada mensaje con un PUBACK
HTTP con conexión reutilizada	POST a `/tel/{esp}` sobre una única conexión que se mantiene abierta (conexión persistente de HTTP/1.1)
HTTP con una conexión por lectura	Abre una conexión por cada lectura, envía el POST con `Connection: close` y la cierra

Los cuatro caminos se corrieron con y sin TLS. Con TLS se usó en el broker y en el backend la misma configuración, con TLS 1.2 [32] y cifrado AES-GCM, un certificado firmado por una autoridad de certificación propia, que el nodo verifica, y sin reanudación de sesión. El cliente MQTT de ESP-IDF admite las versiones 3.1.1 y 5.0 [73], y en este experimento se configuró la 3.1.1 [74]. Para este experimento la diferencia es menor, porque el encabezado fijo es el mismo en ambas versiones y la 5.0 solo agrega a cada PUBLISH un byte con la longitud de sus propiedades.

Firmware

En cada nodo, una tarea productora crea una lectura por período con la hora sincronizada (`t_us`) y la encola, y otra tarea, la emisora, la saca de la cola, la serializa y la envía. Separar la creación del envío hace que el tiempo que una lectura espera en la cola, por ejemplo mientras el emisor está bloqueado en un handshake o en una retransmisión, quede incluido en la latencia medida.

Salvo el keep-alive de MQTT, que se fijó en 60 s, más que el período más largo (10 s), para que el cliente no necesite enviar PINGREQ, se dejaron los valores por defecto de las bibliotecas, que son los que tendría un ESP32 sin ajustes. En MQTT, un PUBLISH de QoS 1 se reenvía si su PUBACK no llega en 1 s. En HTTP, cada envío tiene hasta tres intentos con un tiempo de espera de 5 s, y el algoritmo de Nagle (Sección 5.2.1) está activo. El cliente MQTT abre la conexión antes de que empiece la medición, mientras que el cliente HTTP la abre recién al enviar la primera lectura.

Diseño de las corridas

Se tomó una configuración base con 8 nodos, período de envío de 1 s, TLS activado, sin pérdida ni latencia agregadas y serialización con Protocol Buffers. A partir de ella se varió un factor a la vez (Tabla 12) para cada uno de los cuatro transportes. Cada configuración se repitió cinco veces, con corridas de 120 s y en orden aleatorio con semilla fija. En total fueron 42 configuraciones y 210 corridas, que llevaron 7 h 40 min.

Tabla 12: Factores del experimento.

Factor	**Valores (base en negrita)**	**Pregunta que responde**
Nodos	2, 4, **8**	¿Se satura el broker o el backend?
Período de envío	100 ms, **1 s**, 10 s	¿Cuándo pesa el handshake?
TLS	**Con**, sin	¿Qué parte del costo es cifrado?
Pérdida	**0 %**, 1 %, 5 %	¿Cómo se degrada la entrega?
RTT agregado	**0 ms**, 50 ms, 200 ms	¿Cómo afecta la latencia de la red a cada camino?
Serialización	**Protocol Buffers**, JSON	¿Cuánto ahorra Protocol Buffers en cada camino?

La pérdida se aplica en cada sentido por separado, de forma aleatoria e independiente por paquete. La latencia se expresa como RTT agregado, del que la mitad se agrega en cada sentido, como un retardo fijo.

HTTP con una conexión por lectura se corrió con un período base de 10 s en lugar de 1 s. Con criptografía emulada, cada handshake TLS tarda entre 3 y 4 s, de modo que con un período de 1 s la cola del nodo crece sin límite y solo se entrega cerca del 27 % de las lecturas. Por eso, cuando se comparan los cuatro caminos en la base, ese camino aparece con 10 s, y se lo compara además con las corridas de 10 s de los otros tres.

Además, para verificar la explicación de la latencia de HTTP, se corrió una matriz complementaria con el algoritmo de Nagle activo y desactivado, que se presenta en el análisis del algoritmo de Nagle y el ACK retardado.

Métricas

Los bytes se miden en la red y no en la aplicación, para incluir todo lo que agregan TCP, TLS y cada protocolo. Los bytes por lectura son los de los paquetes IP del puerto del servidor en la captura, divididos por las lecturas entregadas, e incluyen encabezados, ACK, handshakes, cierres y retransmisiones. La latencia es la hora de llegada de cada lectura al servidor menos su hora de creación en el nodo, y se reporta con la mediana y los percentiles 95 y 99. La tasa de entrega es la cantidad de lecturas distintas recibidas sobre las producidas.

Entorno y validez

El experimento se corrió en una computadora con un AMD Ryzen 7 5700U y 14 GB de RAM, con QEMU de Espressif 9.2.2, ESP-IDF 5.5.6, nanopb 0.4.9.1, Mosquitto 2.1.2 y un backend en Go. Las 210 corridas resultaron válidas, ya que los nodos nunca corrieron más lento que el tiempo real, ninguno se atrasó al producir lecturas, no quedaron lecturas sin enviar y no hubo desconexiones de MQTT. Dos corridas en las que un nodo no logró sincronizar su reloj con 5 % de pérdida, antes de empezar a medir, se repitieron con más tiempo de espera para la sincronización.

El cliente SNTP de lwIP no descuenta el tiempo de viaje de la respuesta del servidor (Sección 5.2.1), por lo que el reloj del nodo queda atrasado medio RTT y cada latencia medida resulta medio RTT más larga. Sin latencia agregada, ese error es de alrededor de 1 ms. Con

latencia agregada, como netem la aplica como un retardo fijo, a cada latencia medida se le restó RTT/2.

5.2.4. Resultados

Se presentan la media y el desvío estándar de las cinco repeticiones de cada configuración. Salvo que se indique otra cosa, los valores corresponden a la configuración base con TLS. La entrega fue del 100 % en las 210 corridas.

Bytes y paquetes por lectura

La Tabla 13 muestra el costo de una lectura en cada camino. Todas las diferencias entre transportes resultaron significativas (prueba de Welch, $p < 10^{-4}$), y los intervalos de confianza del 95 % son estrechos, por ejemplo 165,2 ± 0,1 B en MQTT QoS 0, salvo en HTTP con una conexión por lectura (2596 ± 298B).

Tabla 13: Bytes IP y paquetes por lectura entregada. HTTP con una conexión por lectura usa un período de 10 s; el resto, 1 s.

Transporte	**Bytes sin TLS**	**Bytes con TLS**	**Paquetes (TLS)**	**Subida / bajada (TLS)**
MQTT QoS 0	118,7	165,2 ± 0,1	2,25	109 / 56 B
MQTT QoS 1	162,1 ± 0,2	237,7 ± 0,2	3,19	150 / 88 B
HTTP, conexión reutilizada	457,7 ± 0,2	555,0 ± 0,2	6,08	334 / 221 B
HTTP, conexión por lectura	741,8	2596 ± 298	22,5	1005 / 1591 B

Con TLS, una lectura por MQTT QoS 0 ocupó 3,4 veces menos bytes que por HTTP con conexión reutilizada y entre 14 y 16 veces menos que por HTTP con una conexión por lectura. HTTP con una conexión por lectura fue además la única configuración con un desvío grande entre repeticiones.

Latencia en la configuración base

Tabla 14: Latencia extremo a extremo en la configuración base.

Transporte	**p50 (ms)**	**p95 (ms)**	**p99 (ms)**
MQTT QoS 0	2,9	5,5	11,9
MQTT QoS 1	3,0	6,3	9,1
HTTP, conexión reutilizada	44,8	48,5	2737
HTTP, conexión por lectura (10 s)	3383	3924	4129

La Tabla 14 muestra tres grupos separados por un orden de magnitud:

- MQTT tuvo una mediana de unos 3 ms con QoS 0 y con QoS 1, y la diferencia entre ambos no resultó significativa ($p = 0{,}06$).
- HTTP con conexión reutilizada tuvo una mediana de 44,8 ms y una cola larga en el percentil 99. Esa cola se concentra en las primeras lecturas de cada nodo. La primera tardó 3,6 s en promedio, contra 21 ms en MQTT, y desde la décima lectura en adelante el p99 bajó a 48 ms.
- HTTP con una conexión por lectura tuvo una mediana de 3,4 s con TLS y de 10 ms sin TLS.

Pérdida de paquetes

Con 1 % y 5 % de pérdida la entrega siguió siendo del 100 % en todos los caminos. Lo que cambió fueron las retransmisiones, los duplicados y la cola de la latencia (Tabla 15 y Figura 3). Con 5 % de pérdida, el 37 % de las lecturas de HTTP con conexión reutilizada tardó más de 50 ms, contra el 16 % en MQTT QoS 0 y el 10 % en QoS 1. QoS 1 recibió duplicados en el 5,4 % de las lecturas. HTTP con conexión reutilizada tuvo además 1,6 errores por corrida (tiempo de espera de 5 s y reintento de la aplicación), que dejaron 1,4 duplicados.

Tabla 15: Efecto de la pérdida de paquetes. La entrega fue del 100 % en todos los casos. Δ bytes es el aumento de los bytes por lectura respecto de la base.

Transporte	Pérdida	Retransm. por corrida	Duplicados por corrida	p95 (ms)	p99 (ms)	Δ bytes
MQTT QoS 0	1 %	12,8	0	6,7	941	+0,9 %
	5 %	59,2	0	1304	3040	+4,1 %
MQTT QoS 1	1 %	25,4	9,0	6,9	617	+1,5 %
	5 %	124,0	52,0	1017	1749	+7,1 %
HTTP, conexión reutilizada	1 %	30,8	0	1238	3164	+0,7 %
	5 %	169,4	1,4	3810	6638	+4,3 %
HTTP, conexión por lectura	1 %	83,2	0	4491	4784	+9,7 %
	5 %	126,2	0	6287	8492	+12,8 %

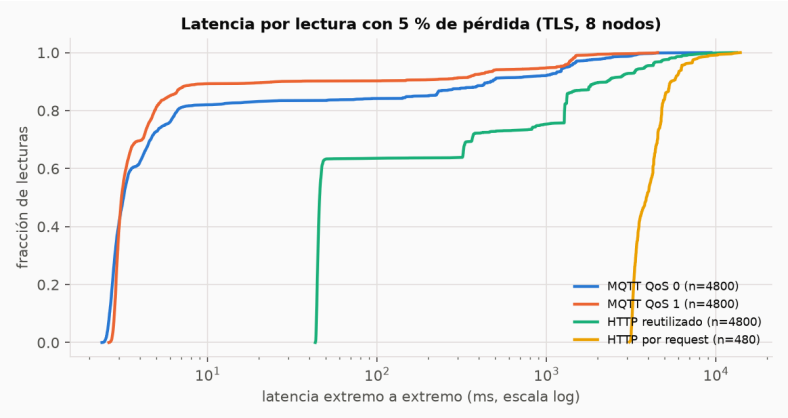


Figura 3: Distribución acumulada de la latencia con 5 % de pérdida (escala logarítmica).

Latencia de la red

La Tabla 16 muestra la mediana de la latencia, ya corregida por el sesgo del reloj, al agregar 50 y 200 ms de RTT. La última columna expresa el aumento con 200 ms en cantidad de RTT. El tiempo de conexión, desde el SYN hasta el fin del handshake, subió entre 340 y 570 ms con 200 ms de RTT. Los bytes por lectura de los caminos con conexión persistente no cambiaron. Los de HTTP con una conexión por lectura bajaron de 2596 a 2297 B.

Tabla 16: Mediana de la latencia, corregida en −RTT/2, según el RTT agregado (TLS).

Transporte	RTT 0	RTT 50 ms	RTT 200 ms	Δ con 200 ms	Δ en RTT
MQTT QoS 0	2,9 ms	27,8 ms	102,7 ms	+99,8 ms	**0,5**
MQTT QoS 1	3,0 ms	27,9 ms	102,9 ms	+99,9 ms	**0,5**
HTTP, conexión reutilizada	44,8 ms	119,5 ms	344,6 ms	+299,8 ms	**1,5**
HTTP, conexión por lectura	3383 ms	3728 ms	4434 ms	+1051 ms	**5,3**

Efecto de TLS

En las conexiones persistentes, TLS sumó entre 46 y 97 B por lectura (entre un 21 % y un 47 %) y menos de 0,6 ms de latencia. El handshake de cada conexión pasó de los 128 B del de TCP a 1525 B en HTTP y 1966 B en MQTT. En HTTP con una conexión por lectura, que paga ese handshake en cada lectura, TLS multiplicó por 3,1 los bytes, tomando el valor de 2296 B sin retransmisiones innecesarias que se explica en el análisis, y sumó 3,4 s de latencia.

Serialización

JSON resultó cuatro veces más grande que Protocol Buffers para este mensaje (72 B contra 18 B) y agregó unos 54 B por lectura en todos los caminos, que equivalen a un 33 % más en MQTT QoS 0, un 23 % en QoS 1, un 9 % en HTTP con conexión reutilizada y un 3 % con una conexión por lectura. La latencia no cambió.

Cantidad de nodos y período de envío

Con 2, 4 y 8 nodos, los bytes, los paquetes y la latencia por lectura no cambiaron (por ejemplo, 165,1, 165,2 y 165,2 B en MQTT QoS 0). Lo único que creció fue la duración de los handshakes, que pasó de 3,01 a 3,32 s en HTTP con una conexión por lectura y de 3,14 a 3,76 s en MQTT QoS 0.

Al acortar el período, los bytes por lectura de los caminos con conexión persistente bajaron. En MQTT QoS 0 fueron 392, 165 y 143 B con períodos de 10 s, 1 s y 100 ms; en HTTP con conexión reutilizada, 673, 555 y 515 B. Con el mismo período de 10 s que HTTP con una conexión por lectura, MQTT QoS 0 usó 392 B, MQTT QoS 1 450 B y HTTP con conexión reutilizada 673 B, contra 2596 B. La latencia de MQTT no cambió con el período. En HTTP con conexión reutilizada y período de 100 ms, la mediana se mantuvo en 44,5 ms, pero el p95 subió a 705 ms.

5.2.5. Análisis de resultados

Anatomía de una lectura

Las capturas permiten ver de qué está hecha cada lectura, paquete por paquete. La Tabla 17 muestra el régimen estable con TLS de los tres caminos con conexión persistente.

Tabla 17: Paquetes de una lectura en régimen estable, con TLS.

Sentido	Paquete	Bytes IP	Composición
MQTT QoS 0: 2 paquetes, 140 B por lectura			
Nodo → servidor	PUBLISH	100	31 B de MQTT, 29 B de TLS y 40 B de TCP/IP
Servidor → nodo	ACK	40	Solo encabezados TCP/IP
MQTT QoS 1: 3 paquetes, unos 215 B por lectura			
Nodo → servidor	PUBLISH	102	33 B de MQTT (2 B más por el identificador de paquete)
Servidor → nodo	PUBACK	73	4 B de MQTT, 29 B de TLS y 40 B de TCP/IP; incluye el ACK de TCP
Nodo → servidor	ACK	40	Sale solo, unos 83 ms después (ACK retardado de lwIP)
HTTP con conexión reutilizada: 6 paquetes, unos 543 B por lectura			
Nodo → servidor	POST y encabezados	203	134 B de HTTP
Servidor → nodo	ACK	40	Llega 40 ms después (ACK retardado de Linux)
Nodo → servidor	Cuerpo	87	18 B de Protocol Buffers, retenidos por Nagle hasta el ACK anterior
Servidor → nodo	ACK	40	Solo encabezados TCP/IP
Servidor → nodo	`204 No Content`	133	64 B de HTTP
Nodo → servidor	ACK	40	Sale solo, unos 230 ms después (ACK retardado de lwIP)

HTTP con una conexión por lectura abre y cierra una conexión completa en cada lectura, con unos 22 paquetes, que corresponden al handshake TCP (3 paquetes, 128 B), al handshake TLS 1.2 (unos 1525 B, de los cuales 655 B son el certificado y los demás mensajes del servidor), al POST con su cuerpo, a la respuesta, al cierre de TLS y al cierre de TCP. Sin TLS son 12 paquetes y 742 B.

El encabezado de MQTT es tan pequeño como plantea el marco teórico. El PUBLISH ocupa 31 bytes, de los cuales 2 son del encabezado fijo, 2 de la longitud del tópico, 9 del tópico `exp/tel/3` y 18 de la lectura. La sobrecarga de MQTT por mensaje es entonces de 13 bytes, contra 134 bytes de la línea de solicitud y los encabezados de HTTP (153 con una conexión por lectura, que agrega `Connection: close`), unas diez veces menos. Además, MQTT QoS 0 no tiene respuesta de aplicación, mientras que HTTP siempre responde (64 B).

Sin embargo, en una lectura de MQTT los encabezados de capa de aplicación no son lo que más pesa. Cada registro TLS agrega 29 bytes (5 de encabezado, 8 del *nonce* explícito y 16 de la etiqueta de autenticación de AES-GCM) y cada paquete, 40 bytes de TCP/IP. En MQTT QoS 0 con TLS, la carga útil es solo el 11 % de los bytes de la lectura, y en HTTP con una conexión por lectura, menos del 1 %.

Los 29 bytes por registro explican casi todo lo que agrega TLS en las conexiones persistentes. MQTT QoS 0 usa un registro por lectura (29 B), al que se suman 16 B del handshake repartido entre las lecturas, 45 B en total, y se midieron 46,5 B. MQTT QoS 1 usa dos registros, el PUBLISH y el PUBACK (74 B con el handshake; se midieron 75,6 B), y HTTP con conexión reutilizada usa tres, los encabezados, el cuerpo y la respuesta (100 B; se midieron 97 B). El cifrado simétrico casi no suma latencia, ya que el costo de TLS en tiempo es el handshake, que solo se paga en cada lectura cuando no se reutiliza la conexión.

Costo fijo por conexión y costo por lectura

En los caminos con conexión persistente, lo que cambia con el período es cuántas lecturas viajan por la misma conexión. Si n es la cantidad de lecturas por conexión, los bytes por lectura se ajustan a un modelo de dos términos:

$$b(n) = v + \frac{C}{n} \tag{1}$$

donde v es el costo de cada lectura y C el costo fijo de abrir y cerrar la conexión, que se reparte entre las n lecturas. El ajuste sobre las corridas con distintos períodos, con TLS, dio un R^2 entre 0,97 y 1,00 (Tabla 18).

Tabla 18: Ajuste del modelo de la Ecuación 1 con TLS.

Transporte	v **(B por lectura)**	C **(B por conexión)**	R^2
MQTT QoS 0	**140**	3020	1,000
MQTT QoS 1	202	2983	0,994
HTTP, conexión reutilizada	526	**1772**	0,973

El valor de v coincide con la anatomía de una lectura, ya que los 140 B de MQTT QoS 0 son exactamente el PUBLISH (100 B) y su ACK (40 B). El valor de C corresponde al handshake TCP y TLS, más el inicio y el cierre de la sesión. El C de MQTT es unos 1250 B mayor que el de HTTP por dos razones. Por un lado, Mosquitto envía, junto con el certificado del servidor, el de la autoridad de certificación (856 B de certificados contra 445 B del backend en Go). Por otro, MQTT agrega el intercambio de CONNECT y CONNACK al abrir la sesión.

En consecuencia, MQTT tiene el costo por lectura más bajo pero el costo fijo más alto, y su ventaja depende de cuánto dure la conexión. Igualando los modelos, MQTT QoS 0 envía menos bytes que HTTP con conexión reutilizada mientras la conexión lleve más de tres o cuatro lecturas. Con una sola lectura por conexión, el modelo da unos 3160 B para MQTT, más que los 2296 B de un POST por HTTPS, aunque eso es una extrapolación fuera del rango medido (entre 12 y 1200 lecturas por conexión). Para un nodo que duerme y se reconecta en cada envío, la ventaja de MQTT desaparece, salvo que se use reanudación de sesión TLS, que en este experimento estuvo desactivada en ambos caminos.

Descomposición de la diferencia

Para separar las causas, la Tabla 19 pasa de HTTP con una conexión por lectura a MQTT QoS 0 cambiando una cosa por vez.

Tabla 19: Descomposición de la diferencia entre HTTP con una conexión por lectura y MQTT QoS 0. Los porcentajes son la parte de la diferencia total que explica cada paso.

Paso	**Con TLS (B por lectura)**	**Sin TLS (B por lectura)**
HTTP con una conexión por lectura	2296	742
→ HTTP con conexión reutilizada (**conexión persistente**)	555 (−1741, **82 %**)	458 (−284, **46 %**)
→ MQTT QoS 1 (**protocolo de aplicación**)	238 (−317, 15 %)	162 (−296, 47 %)
→ MQTT QoS 0 (**sin confirmación de aplicación**)	165 (−73, 3 %)	119 (−43, 7 %)

Con TLS, la conexión persistente explica cuatro quintos de la diferencia, porque evita un handshake de 1525 B en cada lectura. El protocolo de aplicación (encabezados, respuesta

obligatoria y ACK adicionales) explica el 18 % restante. Sin TLS, el handshake TCP pesa mucho menos y la diferencia se reparte casi por mitades entre la conexión y el protocolo. Es decir, el factor más pesado no es el protocolo de aplicación en sí, sino la decisión de mantener la conexión abierta. A eso se suma una parte que no es del protocolo ni de la conexión, sino de cómo el cliente HTTP usa TCP, que se analiza a continuación.

Nagle y ACK retardado

Los dos mecanismos de TCP presentados en la Sección 5.2.1, el algoritmo de Nagle y el ACK retardado, explican buena parte de la latencia de HTTP y varios efectos secundarios observados en el experimento.

Bloqueo en HTTP con conexión reutilizada. `esp_http_client` escribe cada solicitud con dos llamadas a `send()`, primero la línea de solicitud con los encabezados y después el cuerpo, y cada llamada sale como un segmento TCP distinto. La Figura 4 muestra la secuencia medida en una lectura:

1. Los encabezados salen y quedan sin confirmar, por lo que Nagle retiene el cuerpo (18 B) en el ESP32 hasta recibir su ACK.
2. El servidor Linux detecta una conexión de ida y vuelta, en la que contesta cada solicitud, y demora el ACK hasta 40 ms para enviarlo junto con la respuesta. Pero no puede responder, porque todavía no recibió el cuerpo.
3. Cada lado espera al otro, y el bloqueo se rompe recién cuando vence el temporizador de 40 ms del servidor. Entonces llega el ACK, Nagle libera el cuerpo y el servidor responde.

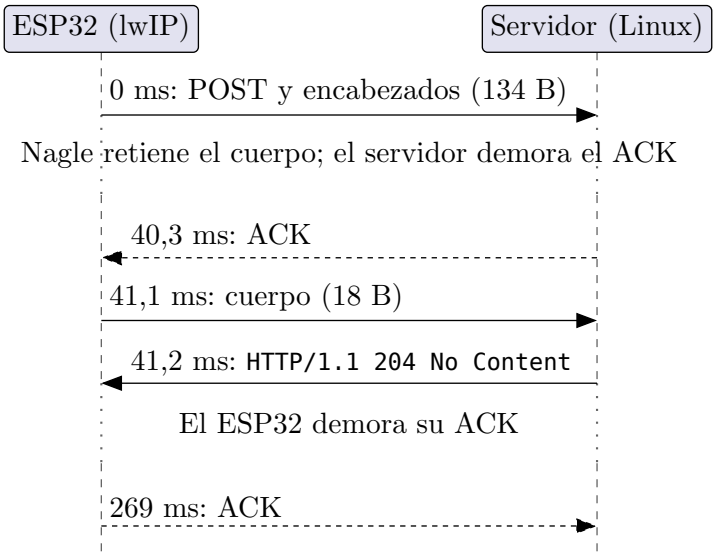


Figura 4: Una lectura de HTTP con conexión reutilizada, sin TLS, tomada de una captura. Las flechas punteadas son ACK sin datos.

La espera medida entre los encabezados y el cuerpo fue de 41,5 ms sin TLS y de 41,7 ms con TLS. Con TLS ocurre lo mismo porque mbedTLS cifra cada `send()` en su propio registro, de modo que también salen dos segmentos. Esa espera explica casi toda la mediana de 44,8 ms de HTTP con conexión reutilizada (Tabla 14).

Prueba con Nagle desactivado. Para verificar la explicación se corrió una matriz complementaria con Nagle activo y desactivado (`TCP_NODELAY`) en los dos caminos HTTP, con tres repeticiones. En HTTP con conexión reutilizada, la latencia bajó unas diez veces, de 44 ms a entre 4 y 5 ms, con y sin TLS, y cada lectura usó un paquete y unos 40 B menos, porque sin la espera el ACK del servidor a los encabezados viaja junto con la respuesta. HTTP con

una conexión por lectura sin TLS no cambió, porque Linux confirma de inmediato los primeros segmentos de una conexión nueva y no sufría la espera.

Efecto de la latencia de la red sobre la penalidad de Nagle. MQTT pagó medio RTT por lectura (Tabla 16), el mínimo posible, porque el PUBLISH sale y llega sin esperar nada. En HTTP, en cambio, el ACK de los encabezados tarda un RTT completo en volver, y el cuerpo espera ese tiempo además de los 40 ms del ACK retardado. Por eso HTTP con conexión reutilizada pagó 1,5 RTT por lectura, que corresponden a medio RTT del viaje de ida y a un RTT de la espera de Nagle. HTTP con una conexión por lectura con TLS pagó 5,3 RTT, porque a eso suma los handshakes de TCP y de TLS 1.2 y otra espera de Nagle dentro de este último, porque el ESP32 escribe en dos llamadas separadas el *Client Key Exchange* y el *Change Cipher Spec* con el *Finished*.

Retransmisiones innecesarias. La variabilidad de HTTP con una conexión por lectura (2596 ± 298 B por lectura) no viene de la red. El ESP32 pasa unos 3 s calculando su parte del handshake TLS con criptografía emulada, y si el tiempo de retransmisión del servidor vence antes de que llegue su ACK, el servidor reenvía los 692 B de sus mensajes de handshake, que no se habían perdido. Los bytes de cada corrida siguen exactamente la cantidad de esas retransmisiones (correlación de 1,000), que ocurrieron en casi todas las conexiones de una corrida o en ninguna. Por eso, en el análisis se toma como referencia de ese camino el valor de 2296 B, sin retransmisiones innecesarias.

Ausencia del bloqueo en MQTT. esp-mqtt arma cada PUBLISH completo en un *buffer* y lo escribe con un solo `send()`, por lo que cada lectura es un único segmento y Nagle no tiene nada que retener. Además, con QoS 0 el broker nunca responde, no detecta una conexión de ida y vuelta y confirma enseguida; con QoS 1, el ACK viaja dentro del PUBACK.

En síntesis, parte de la diferencia entre HTTP y MQTT no se debe al protocolo de aplicación, sino a cómo cada cliente usa TCP. El mismo HTTP cambia su latencia en un orden de magnitud según cómo el cliente reparte los datos en llamadas a `send()`. Con Nagle desactivado, HTTP con conexión reutilizada baja a 5 ms, del mismo orden que MQTT, aunque sigue enviando 3,1 veces más bytes, unos 515 B (40 B menos que con Nagle activo) contra 165 B. En el experimento principal se mantuvo el comportamiento por defecto, con Nagle activo, porque es el que tendría un ESP32 sin ajustes.

Pérdida de paquetes

Con hasta 5 % de pérdida no se perdió ninguna lectura, porque TCP retransmitió cada segmento perdido y ninguna conexión se cortó. MQTT QoS 0 solo perdería lecturas si se cortara la conexión, por ejemplo al vencer el keep-alive, por un *reset* o porque el nodo se reconecta a la red, y eso no ocurrió con pérdidas aleatorias en corridas de 120 s. En estas condiciones, QoS 1 no mejoró la entrega y solo agregó duplicados (Tabla 15), porque esp-mqtt reenvía el PUBLISH si el PUBACK no llega en 1 s, aunque TCP todavía esté recuperando el segmento original, tal como se anticipó en la Sección 1.3.

Lo que se degradó fue la cola de la latencia, y la Figura 3 muestra escalones en valores fijos:

- Entre 1 y 1,5 s, el RTO de lwIP, dado que un segmento perdido se recupera recién cuando vence el temporizador de retransmisión, que lwIP maneja con un reloj de 500 ms. No hay retransmisión rápida, porque requiere tres ACK duplicados y con una lectura por segundo nunca hay tantos segmentos en vuelo.

- Entre 0,05 y 0,5 s, el bloqueo de cabeza de línea, por el que la lectura que sigue a una perdida sale 1 s después y queda retenida hasta que la anterior se recupera. Por eso las lecturas lentas vienen en pares.
- Por encima de 2 s, pérdidas repetidas del mismo segmento, con el RTO duplicado en cada intento [42].

HTTP salió peor por su esquema de solicitud y respuesta en serie. Cualquier paquete perdido, en cualquier sentido, frena el envío hasta que se recupera, y las lecturas siguientes esperan en la cola. Además, cada lectura usa 6 paquetes (22 con una conexión por lectura), contra 2 o 3 de MQTT, por lo que tiene más probabilidad de perder alguno. Entre los dos niveles de MQTT, QoS 1 tuvo una cola algo menor (p99 de 1,7 s contra 3,0 s), porque su ACK viaja dentro del PUBACK, que Linux retransmite a los 200 ms si se pierde, ya que su temporizador de retransmisión no baja de ese valor, aunque la especificación recomienda un mínimo de 1 s [42]. Con QoS 0, en cambio, un ACK perdido deja a Nagle reteniendo la lectura siguiente hasta que vence el tiempo de retransmisión de lwIP, de al menos 1 s.

El costo en bytes de la pérdida fue bajo, entre 1 % y 13 %. El mayor fue el de HTTP con una conexión por lectura, al que se suman las retransmisiones innecesarias del handshake.

Serialización

El ahorro de Protocol Buffers fue de unos 54 B por lectura en todos los caminos, porque el mensaje es el mismo. En HTTP fue 6 B menor, porque `application/json` es más corto que `application/x-protobuf` en el encabezado `Content-Type`. La reducción de la carga útil, del 75 %, está en línea con la de los estudios citados en el marco teórico [26], [27].

En términos relativos, en cambio, el efecto depende de cuánto pesa el resto de la lectura, que en MQTT es mucho menos que en HTTP. Esto confirma lo anticipado en el marco teórico, es decir, que una vez que el transporte reduce sus encabezados al mínimo, la serialización pasa a tener un peso importante. Aun así, la elección del transporte pesa más, ya que pasar de HTTP con conexión reutilizada a MQTT QoS 0 ahorra 390 B por lectura, más de siete veces lo que ahorra Protocol Buffers.

Contraste de hipótesis

La Tabla 20 contrasta las hipótesis del planteo con los resultados.

Tabla 20: Contraste de las hipótesis con los resultados.

Hipótesis	**Resultado**
MQTT envía menos bytes por lectura que HTTPS, y la diferencia crece cuando el período de envío es corto.	**Se confirma, con un matiz.** MQTT envió menos bytes en todos los casos medidos, y su ventaja frente a HTTP con conexión reutilizada creció al acortar el período, de 1,7 a 3,6 veces. Si la conexión dura menos de tres o cuatro lecturas, la relación se invierte (Tabla 18).
HTTPS con conexión reutilizada reduce gran parte de la diferencia, pero los encabezados HTTP siguen pesando más que el encabezado MQTT.	**Se confirma.** Reutilizar la conexión bajó HTTPS de 2296 a 555 B por lectura, pero siguió costando 3,4 veces MQTT QoS 0.
HTTPS con una conexión por solicitud es el caso más costoso, sobre todo con TLS y latencia alta.	**Se confirma.** Fue el más caro en bytes y en latencia, y con 200 ms de RTT pagó 5,3 RTT por lectura.
Con pérdida, MQTT QoS 1 mantiene la entrega a costa de duplicados, mientras que QoS 0 pierde lecturas.	**Se confirma a medias.** QoS 1 generó duplicados, pero QoS 0 no perdió lecturas, porque TCP recuperó todo y ninguna conexión se cortó.
Protocol Buffers reduce la carga útil en ambos caminos, con efecto relativo mayor en MQTT, donde la carga útil es la mayor parte del mensaje.	**Se confirma, aunque no por la razón prevista.** El ahorro fue de unos 54 B en todos los caminos y el relativo fue mayor en MQTT (33 %), pero porque su lectura es más pequeña, ya que la carga útil es solo el 11 % del mensaje.

Limitaciones

- **Criptografía emulada.** Los handshakes TLS tardaron entre 3 y 4 s en QEMU, mucho más que en un ESP32 físico, por lo que los tiempos de conexión y la latencia de HTTP con una conexión por lectura deben leerse en términos relativos. Los bytes y los paquetes, en cambio, son los del firmware real.
- **Un factor a la vez.** No se midieron interacciones entre factores, como pérdida y período o latencia y JSON, salvo la de Nagle y TLS.
- **Red emulada.** Se usó Ethernet emulado y pérdida aleatoria independiente, sin ráfagas. Con pérdidas en ráfaga o cortes de conexión, que no formaron parte de la matriz, las sesiones podrían caerse, y entonces QoS 0 sí perdería lecturas. Las corridas de 120 s tampoco permiten observar el keep-alive ni las reconexiones.
- **Un solo sentido.** Solo se midió la telemetría del nodo al servidor. El envío de comandos del servidor al nodo, donde HTTP necesitaría consultas periódicas, quedó fuera de la comparación.

Implicaciones

La Tabla 21 lleva los resultados a la escala de un despliegue. Supone una conexión que no se corta, por lo que usa el costo por lectura v de la Tabla 18, y una lectura cada 10 s por nodo (8640 por día). El keep-alive no suma tráfico, porque el período es menor que los 60 s configurados. Estos volúmenes son los que determinan el plan de datos necesario en un despliegue sobre una red celular, y sirven de dato de entrada para el mapeo tecnológico.

Tabla 21: Proyección del tráfico con una lectura cada 10 s.

Transporte (TLS)	**Por nodo y por día**	**1000 nodos, por mes**
MQTT QoS 0	1,2 MB	36 GB
MQTT QoS 1	1,7 MB	52 GB
HTTP, conexión reutilizada	4,5 MB	136 GB
HTTP, conexión por lectura	19,8 MB	595 GB

Para un sistema de telemetría como el estudiado, los resultados sugieren dos criterios:

- Lo que más importa es mantener la conexión abierta. La ventaja de MQTT depende de que la conexión dure más que unas pocas lecturas; para dispositivos que duermen y se reconectan en cada envío, pesan más la reanudación de sesión TLS o TLS 1.3 que la elección del protocolo de aplicación.
- Si se usa HTTP, conviene reutilizar la conexión y desactivar Nagle, o escribir cada solicitud con una sola llamada a `send()`. Con eso la latencia queda en el mismo orden que la de MQTT, aunque el costo en bytes sigue siendo unas tres veces mayor.

5.3. Experimento 2: sincronización de osciladores acoplados

La sincronización colectiva aparece cuando un conjunto de osciladores, cada uno con su propio ritmo, termina funcionando a una frecuencia común sin que nadie la imponga, como ocurre con las luciérnagas que destellan al unísono [75]. En este experimento, cada ESP32 ejecuta un oscilador con una frecuencia natural distinta a la de los demás y corrige su fase con las fases que recibe de los otros nodos a través del broker. Cuando esa corrección es suficientemente intensa, el grupo se engancha y todos pasan a girar a una misma frecuencia, sin ningún nodo que actúe de coordinador.

Esa sincronía existe solo a través de los mensajes que intercambian los nodos, y por eso sirve para evaluar el stack. El modelo matemático predice cómo debería comportarse el grupo si la comunicación fuera ideal, de modo que comparar esa predicción con lo que se mide en los nodos muestra qué efecto tiene cada mecanismo del stack sobre la aplicación. A diferencia del experimento de la Sección 5.2, que mide el costo de enviar lecturas desde muchos nodos hacia un único servidor, aquí cada nodo produce un dato que les interesa a todos los demás y consume los de todos ellos, que es la forma de circulación de la información para la que se propone el modelo publicador/suscriptor (Sección 1).

5.3.1. Planteo

Motivación

Este experimento corresponde al último de los objetivos específicos (Sección 1), en lo que respecta al comportamiento del stack ante pérdidas y cortes de conexión. Somete a prueba, en una misma aplicación, el desacoplamiento entre participantes, la compacidad del mensaje, la compatibilidad del esquema, el broker como punto único de falla y el papel del keep-alive y de la última voluntad, que son varias de las virtudes y limitaciones que el marco teórico y el desarrollo atribuyen al stack de forma cualitativa.

Fundamento

El modelo. El experimento se basa en el modelo de osciladores acoplados que propuso Kuramoto [76]. Se consideran N osciladores, cada uno descrito solo por su fase y acoplado a todos los demás con el mismo peso. El oscilador i tiene una frecuencia natural ω_i, en rad/s, que es fija, y una fase θ_i, en radianes, que evoluciona según

$$\dot{\theta}_i = \omega_i + \frac{K}{N}\sum_{j=1}^{N}\sin\big(\theta_j - \theta_i\big), \quad i = 1, ..., N \tag{2}$$

donde el punto indica la derivada respecto del tiempo. El acoplamiento es la influencia que los osciladores ejercen entre sí, representada por el término de la sumatoria, que depende de las diferencias de fase. Su intensidad es K, en rad/s. Con $K = 0$, cada oscilador avanza a su frecuencia natural sin afectar a los demás. El acoplamiento cambia la velocidad instantánea $\dot{\theta}_i$. Si el oscilador j va adelantado menos de media vuelta, $\sin\big(\theta_j - \theta_i\big) > 0$ y el oscilador i se acelera; si va atrasado menos de media vuelta, se frena.

Parámetro de orden. El grado de sincronía se mide con el centroide de las fases sobre el círculo unitario:

$$re^{\mathrm{i}\psi} = \frac{1}{N}\sum_{j=1}^{N} e^{\mathrm{i}\theta_j} \tag{3}$$

donde i es la unidad imaginaria. El módulo r, entre 0 y 1, se denomina parámetro de orden y vale 0 si las fases están dispersas y 1 si coinciden. El ángulo ψ es la fase media del grupo [75]. La Figura 5 muestra ambas situaciones.

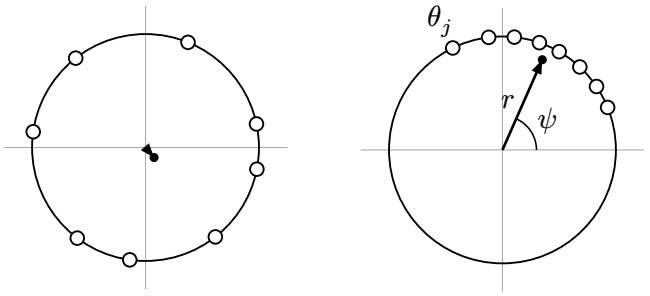


Figura 5: Fases de ocho osciladores sobre el círculo unitario (círculos vacíos) y su centroide $re^{\mathrm{i}\psi}$ (punto negro). A la izquierda, fases dispersas; a la derecha, fases agrupadas.

Frecuencia común y acoplamiento crítico. Se dice que el grupo está enganchado cuando todos los osciladores giran a una misma frecuencia, que se denota Ω, y sus diferencias de fase permanecen constantes. Sin retardo, esa frecuencia es la media de las frecuencias naturales, $\Omega = \overline{\omega}$, porque el acoplamiento redistribuye las velocidades entre los osciladores sin cambiar su suma [77]. Las desintonías $\Delta_i = \omega_i - \overline{\omega}$ indican cuánto se aparta cada frecuencia natural de la media.

Se denomina acoplamiento crítico K_c al menor valor de K para el que existe un estado enganchado. Para una cantidad finita de osciladores, ese estado existe si y solo si tiene solución la condición de autoconsistencia [77]

$$r = \frac{u}{K} = f(u) = \frac{1}{N}\sum_{i=1}^{N}\sqrt{1 - \left(\frac{\Delta_i}{u}\right)^2}, \quad u \geq \max_i|\Delta_i| \tag{4}$$

donde $u = Kr$ es la fuerza con la que el grupo atrae a cada oscilador. El menor acoplamiento para el que hay solución es

$$K_c = \min_{u \geq \max_i|\Delta_i|} \frac{u}{f(u)} \tag{5}$$

y, para un K dado, la Ecuación 4 da el valor de r del estado enganchado y el desfasaje de cada oscilador respecto de la fase media, $\arcsin(\Delta_i/u)$ [77]. Para los 20 osciladores que se usan, con frecuencias naturales a intervalos iguales entre 1,005 y 1,508 rad/s, resulta $K_c = 0{,}335$ rad/s, y en el umbral $r = 0{,}773$, por lo que el enganche aparece con el grupo ya muy ordenado. El cálculo de los autovalores de la matriz jacobiana del modelo linealizado alrededor del estado enganchado da estable el estado enganchado en todos los valores de K usados en el experimento. Es una estabilidad local, que no asegura que el sistema llegue a ese estado desde cualquier condición inicial, y eso se comprueba por simulación.

El modelo en los nodos. El modelo anterior supone que cada oscilador conoce en todo instante la fase exacta de los demás. En la implementación esto no se cumple, porque la fase de cada nodo llega a los otros como un mensaje MQTT, con demora y solo cada cierto tiempo. Si $\tau_{ij}(t)$ es la antigüedad, en el instante t, de la última fase del nodo j que conoce el nodo i, la evolución de cada nodo está dada por

$$\dot{\theta}_i(t) = \omega_i + \frac{K}{N} \sum_{j \neq i} \sin\big(\theta_j\big(t - \tau_{ij}(t)\big) - \theta_i(t)\big) \tag{6}$$

La suma excluye $j = i$ porque cada nodo conoce su propia fase sin demora, y ese término es nulo. El divisor N es la cantidad total de nodos y permanece fijo aunque falte alguna fase, de modo que un mensaje ausente no altera el peso de los demás. Con todos los retardos nulos, la Ecuación 6 coincide con la Ecuación 2.

La antigüedad τ_{ij} reúne varias demoras. Una fase recorre dos conexiones TCP, del publicador al broker y del broker al suscriptor (Sección 1), y pasa por la cola del broker. Además, entre dos publicaciones sucesivas la fase conocida envejece. Si cada nodo publica con período h, la fase guardada envejece desde casi cero hasta casi h, en forma de diente de sierra, y esa espera agrega en promedio alrededor de $h/2$. Por último, MQTT delega la entrega en TCP (Sección 2), de modo que un paquete descartado en la red no se pierde, sino que se retransmite y llega más tarde. Mientras la conexión se mantiene, la pérdida de paquetes se manifiesta entonces como un aumento y una variación de τ_{ij}. Si un mensaje no llega, el nodo sigue usando la última fase recibida, cuya antigüedad continúa creciendo.

Cada nodo integra la Ecuación 6 con el método de Euler y un paso T_s de 10 ms, que no tiene por qué coincidir con h. En el firmware, T_s es el tiempo que realmente transcurrió entre dos pasos, para que una demora en la planificación del emulador no cambie la velocidad de la fase:

$$\theta_i[k+1] = \theta_i[k] + T_s \left(\omega_i + \frac{K}{N} \sum_{j \neq i} \sin\big(\hat{\theta}_j[k] - \theta_i[k]\big) \right) \tag{7}$$

donde $\hat{\theta}_j[k]$ es la última fase recibida del nodo j al llegar el paso k.

Efecto del retardo. El modelo de Kuramoto con un retardo τ constante e igual para todos los pares, incluido el término de cada oscilador consigo mismo, se analiza en el trabajo de Yeung y Strogatz [78]. Para osciladores idénticos, con $\omega_i = \omega_0$, existe una solución en la que todos giran en fase a una frecuencia Ω que cumple

$$\Omega = \omega_0 - K \sin(\Omega \tau) \tag{8}$$

y esa solución es estable si $\cos(\Omega\tau) > 0$ [78]. El retardo, por lo tanto, corre la frecuencia común respecto de la natural, y lo que determina la estabilidad no es τ por sí solo, sino el producto $\Omega\tau$. Con acoplamiento débil, $\Omega \approx \omega_0$ y la condición se cumple mientras el retardo sea menor

que un cuarto del período de oscilación, que a los 0,2 Hz de este experimento equivale a 1,25 s. El retardo se compara, entonces, con el período de oscilación y no con la desintonía. La condición es periódica en τ, de modo que superar $\Omega\tau = \pi/2$ no implica por sí solo perder la estabilidad [78]. Para frecuencias distribuidas y un número infinito de osciladores, el retardo multiplica además el acoplamiento crítico por $1/\cos(\Omega_c\tau)$, donde Ω_c es la frecuencia común en el umbral [78].

Estos resultados no se aplican de forma directa a la Ecuación 6, y se utilizan como referencia. La diferencia del término propio es menor, porque al repetir el planteo para osciladores idénticos sin ese término quedan $N-1$ sumandos con retardo, y la condición pasa a ser $\Omega = \omega_0 - K(N-1)/N\sin(\Omega\tau)$, que tiende a la Ecuación 8 cuando N crece. No se dispone, en cambio, de un resultado cerrado para frecuencias arbitrarias, N finito y retardos que varían en el tiempo. Se espera que la variación del retardo, o *jitter*, actúe como un ruido sobre las fases recibidas, baje r en régimen y suba el K_c medido, aunque no equivale al ruido blanco que se agrega a las frecuencias en el modelo de Yeung y Strogatz [78]. Ese caso se estudia comparando la simulación del modelo con las mediciones.

Retardo uniforme con frecuencias distintas. Los resultados anteriores no alcanzan para las frecuencias del experimento, pero el mismo razonamiento se puede extender. Esta extensión no proviene de las fuentes citadas, sino que es propia de este trabajo. Se supone en la Ecuación 6 un retardo τ constante e igual para todos los pares y un estado enganchado, $\theta_i(t) = \Omega t + a_i$, y se llaman r y ψ al módulo y al ángulo del centroide de los a_j. Con $\beta_i = a_i - \psi + \Omega\tau$, cada oscilador debe cumplir

$$\omega_i + \frac{K}{N}\sin(\Omega\tau) - \Omega = Kr\sin\beta_i \tag{9}$$

y, promediando sobre todos los osciladores, la frecuencia común resulta

$$\Omega = \overline{\omega} - K\left(r^2 - \frac{1}{N}\right)\sin(\Omega\tau) \tag{10}$$

Con $\tau = 0$ se recupera $\Omega = \overline{\omega}$. Con retardo, Ω queda por debajo de la media, de modo que los osciladores más rápidos quedan más lejos de la frecuencia común y son los primeros en no poder seguirla. Resolviendo numéricamente las ecuaciones para las 20 frecuencias, con $\tau = 255$ ms ($h/2$ más unos 5 ms de transporte), la primera solución aparece en $K = 1{,}17\ K_c$, mientras que el factor $1/\cos(\Omega\tau)$ solo sugeriría un aumento cercano al 5 %. La diferencia es justamente el corrimiento de Ω. Se trata de una aproximación, porque la antigüedad real no es constante sino un diente de sierra alrededor de $h/2$, y se contrasta con la simulación y las mediciones.

Acoplamiento fijo y efecto de la red. K es una ganancia del software de cada nodo, que se fija al comienzo de cada corrida y no cambia durante ella. Tampoco modifica el tráfico, que depende de h y del tamaño de los mensajes. La red no altera K, pero sí el efecto que tiene sobre la dinámica, de modo que hace falta un acoplamiento mayor que el teórico para lograr el enganche. Por eso, comparar lo que predice el modelo con lo que se mide indica qué efecto tiene cada mecanismo del stack sobre la aplicación, y el experimento se organiza por mecanismos y no en torno a un único valor de K_c.

Interrupción de la comunicación. Los resultados anteriores suponen que las fases siguen llegando. Cuando dejan de hacerlo, lo que ocurra depende de qué haga cada nodo con las que ya tiene. Si detecta la desconexión y las descarta, la sumatoria se anula y cada oscilador avanza a su

frecuencia natural, $\dot{\theta}_i = \omega_i$. Si parte de un estado enganchado, con desfasajes $\varphi_i = \arcsin(\Delta_i/u)$ respecto de la fase media, el parámetro de orden evoluciona como

$$r(t) = \left| \frac{1}{N} \sum_{i=1}^{N} e^{\mathrm{i}(\varphi_i + \omega_i t)} \right| \tag{11}$$

Las frecuencias distintas hacen que las fases se dispersen, y r cae en un tiempo del orden de π/b, donde b es la mitad del rango de las frecuencias naturales. Si solo un nodo pierde la comunicación, se separa de los demás a una velocidad $\omega_i - \Omega$.

Si, en cambio, el nodo no advierte la interrupción, sigue sumando las últimas fases recibidas, que ya no cambian. El acoplamiento deja de apuntar a la fase actual del grupo y atrae al nodo hacia un valor fijo, mientras el grupo sigue girando, de modo que el nodo se atrasa respecto de él hasta que se detecta el corte.

Datos incorrectos. El modelo supone también que $\hat{\theta}_j$ es efectivamente una fase. Si la mitad de los nodos interpreta de forma incorrecta los mensajes de la otra mitad, cada nodo solo se acopla de manera coherente con los $N/2$ de su grupo. Como el divisor sigue siendo N, el acoplamiento dentro del grupo se reduce a $K/2$, y el grupo solo puede engancharse si ese valor supera el K_c de sus propias frecuencias.

Preguntas de investigación

1. ¿Se sincronizan los nodos como predice el modelo cuando la red no agrega perturbaciones?
2. ¿De qué está hecho un mensaje de fase y cuánto pesa cada parte?
3. ¿Qué ocurre si dos participantes usan versiones distintas del esquema?
4. ¿Cómo responde el grupo a la caída del broker y a la interrupción del enlace de un nodo?
5. ¿Qué aporta QoS 1 frente a QoS 0 con pérdida de paquetes, en una aplicación cuyos datos se reemplazan?
6. ¿Cuánto reduce el alias de tópico el tráfico, y cómo crece la carga del broker con la cantidad de nodos?

Hipótesis

Antes de las corridas se formularon las siguientes hipótesis, que se contrastan con los resultados al final del análisis (Tabla 27):

- Por encima de K_c, los nodos se enganchan con un r algo menor que el teórico, por la antigüedad que introduce el período de publicación.
- Un campo que el receptor no conoce no afecta la sincronía, y dos campos del mismo tipo cruzados se decodifican sin error, pero impiden sincronizar.
- Al caer el broker, el grupo pierde la sincronía y la recupera sin intervención cuando el broker vuelve.
- Un corte de enlace menor que una vez y media el keep-alive no cierra la conexión, y uno mayor la cierra y publica la última voluntad.
- Con pérdida de paquetes, QoS 1 no mejora la sincronización y genera más tráfico que QoS 0.
- El alias de tópico reduce en unos 13 bytes cada mensaje que entrega el broker.

5.3.2. Implementación

El objetivo de la implementación no es sincronizar osciladores de la mejor manera posible, sino construir una aplicación en la que cada mecanismo de MQTT y de Protocol Buffers tenga una consecuencia visible. Los nodos ejecutan la Ecuación 7 tal como se planteó, y lo único que les permite acoplarse es el intercambio de mensajes. Por eso, una decisión del stack, como el nivel

de QoS o el intervalo de keep-alive, se puede observar en el parámetro de orden r (Ecuación 3), que mide cuán agrupadas están las fases, y en la frecuencia común Ω del grupo, y el modelo indica qué debería verse si la comunicación fuera ideal.

Arquitectura

El experimento corre sobre la plataforma común (Sección 5.1), con 20 nodos. En la computadora anfitriona corren además un servidor SNTP con el que los nodos sincronizan su reloj (Sección 5.2.1) y un registrador, `mosquitto_sub`, un cliente MQTT de línea de comandos suscrito a todas las fases. El registrador se conecta por la interfaz local, de modo que no pasa por el puente, no aparece en las capturas ni sufre las condiciones de red que se aplican a los nodos (Figura 6). Las capturas se hacen sobre el puente con `tcpdump`.

En lo que afecta a la entrega de los mensajes, Mosquitto mantiene sus valores por defecto salvo en cuatro ajustes. Desactiva el algoritmo de Nagle (`set_tcp_nodelay`), para que cada mensaje salga apenas se publica. Amplía la cola de mensajes por cliente (`max_queued_messages` a 100.000 y `max_queued_bytes` sin límite), para que las ráfagas no produzcan descartes. Y admite hasta 128 alias de tópico hacia los nodos (`max_topic_alias_broker`). Se conserva, en cambio, el límite por defecto de 20 mensajes en tránsito por cliente (`max_inflight_messages`), que solo afecta a QoS 1. Mientras un cliente tenga 20 mensajes sin confirmar, el broker no le envía el siguiente y lo deja en la cola.

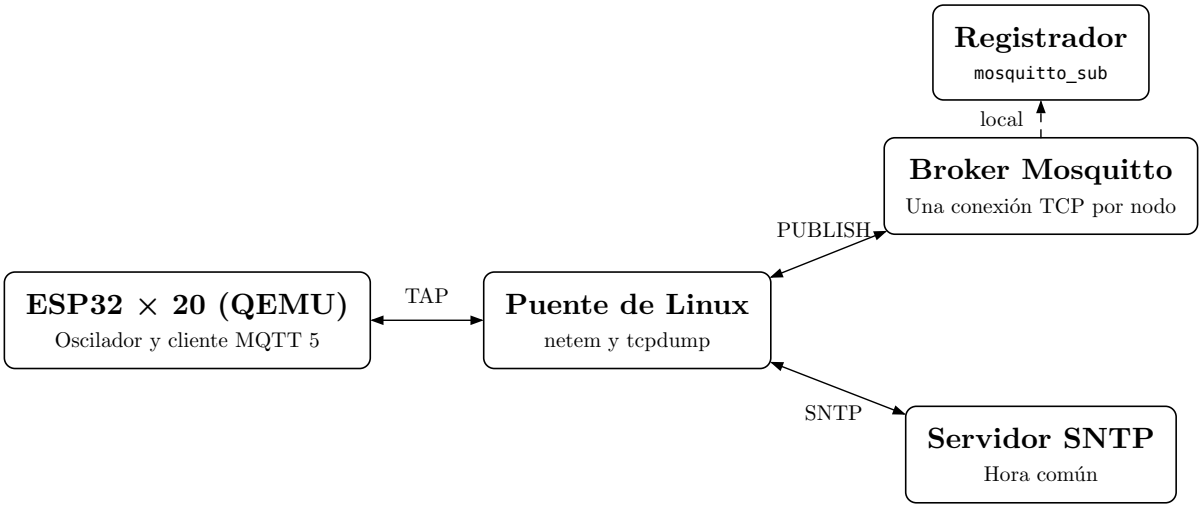


Figura 6: Arquitectura del experimento. Cada nodo se comunica solo con el broker; la suscripción del registrador no pasa por el puente.

Implementación del modelo en el firmware

La frecuencia natural ω_i, el acoplamiento K, la fase inicial y el período de publicación h no están fijados en el código, sino que se leen de la memoria *flash* del nodo, de modo que el firmware se compila una sola vez para todas las corridas. Dos tareas reparten el trabajo. La de integración avanza la fase según la Ecuación 7 cada 10 ms, y la de publicación toma la última fase cada $h = 500$ ms, la serializa y la entrega al cliente MQTT. Al estar separadas, un envío demorado no detiene la integración, que es el desacoplamiento de sincronización de la Sección 1 llevado al código. Cada nodo publica con un desfase propio, elegido al azar dentro de h, de modo que los nodos no publican todos a la vez y la fase conocida de cada otro nodo envejece de manera repartida, como supone la antigüedad media cercana a $h/2$ estimada en el planteo.

Recorrido de una fase

Una fase atraviesa el stack completo antes de influir en otro nodo, y cada paso aporta a la antigüedad τ_{ij}:

1. La tarea de publicación copia la fase, su velocidad y el instante en que se calculó.
2. La fase se serializa con Protocol Buffers mediante nanopb, una biblioteca en C pensada para sistemas con muy poca memoria [79].
3. El cliente esp-mqtt, la biblioteca MQTT de ESP-IDF [73], arma un paquete PUBLISH de QoS 0, sin confirmación de entrega (Sección 2), con esos bytes como carga útil, en el tópico `kuramoto/phase/<id>`, y lo escribe en la única conexión TCP del nodo, que administra lwIP, la pila TCP/IP liviana que incluye ESP-IDF.
4. El broker recibe el paquete y compara el tópico con las suscripciones. Sin interpretar la carga útil, reenvía una copia a cada nodo suscrito al filtro `kuramoto/phase/+`, donde el comodín `+` coincide con cualquier identificador. El emisor no recibe su propia fase gracias a la opción *No Local* de MQTT 5.0, que pide al broker no devolverle al cliente sus propias publicaciones.
5. Cada receptor decodifica la carga útil. Descarta el mensaje si no es más nuevo que el último de ese nodo, comparando el número de secuencia, y si lo es, reemplaza la entrada de la tabla.
6. En el siguiente paso de integración, a lo sumo 10 ms después, esa fase ya participa de la sumatoria, y sigue envejeciendo hasta que llegue la próxima.

El mensaje es el mismo para todos los nodos, y su definición en un único archivo `.proto` funciona como contrato entre ellos (Sección 4):

```
message PhaseReport {
  uint32  id = 1;          // nodo emisor, 1..N
  float   theta = 2;       // fase en [0, 2*pi), en el instante t_us
  float   velocidad = 3;   // último valor de la derivada de theta, en rad/s
  uint32  seq = 4;         // número de reporte, desde 0
  fixed64 t_us = 5;        // instante de muestreo, en microsegundos
}
```

Codificado, ocupa entre 21 y 24 bytes, porque `seq` es un entero de longitud variable y, en proto3, la versión del lenguaje de esquemas que se usa, su valor inicial 0 no se transmite (Sección 3). El encabezado MQTT del PUBLISH suma otros 21 o 22 bytes, de los cuales 16 o 17 son el nombre del tópico. Como cada PUBLISH contiene exactamente un mensaje, la delimitación queda resuelta por MQTT (Sección 4), y el tópico alcanza para saber qué esquema aplicar. Con 20 nodos publicando cada 500 ms, el broker realiza $N(N-1)/h = 760$ entregas por segundo.

Se publica con QoS 0 y sin mensajes retenidos, es decir, sin pedirle al broker que guarde la última fase para futuros suscriptores (Sección 2), por el criterio de la Sección 4.5, porque cada fase queda reemplazada por la siguiente.

Conexión y desconexión

El cliente vuelve a suscribirse cada vez que se conecta, porque el broker no conserva su sesión, es decir, sus suscripciones, entre una conexión y la siguiente (Sección 2). Si la conexión se pierde, vuelve a intentarla. El tiempo de reconexión por defecto de esp-mqtt es de 10 s, pero la tarea del cliente lo comprueba cada 5 s y exige superarlo, de modo que en la práctica el intento siguiente ocurre 15 s después de la desconexión, como confirman las mediciones. Cuando recibe el aviso de desconexión, invalida toda la tabla de fases. A partir de ahí la sumatoria no tiene términos y el nodo avanza a su frecuencia natural, que es el comportamiento previsto en el planteo para un corte detectado.

Ese aviso no llega apenas se interrumpe la red. Si el broker cierra la conexión, el nodo se entera enseguida. Si el camino simplemente deja de transportar paquetes, el nodo sigue usando las últimas fases recibidas hasta que el keep-alive o TCP detecten el problema, por lo que durante un tiempo se acopla a valores congelados. El keep-alive es el tiempo máximo que puede pasar entre dos paquetes enviados por el cliente (Sección 2). Si el broker no recibe nada durante una vez y media ese intervalo, cierra la conexión (Sección 2.4). En los ensayos de corte se configura en 5 s, en lugar de los 120 s por defecto de esp-mqtt, para que los regímenes de la Tabla 8 se puedan recorrer en pocos minutos.

También se puede registrar un mensaje de última voluntad en el tópico `kuramoto/estado/<id>`. Es un mensaje que el cliente deja registrado al conectarse y que el broker publica si la conexión se cierra de forma anormal (Sección 2). Cuando el broker lo publica, los demás nodos invalidan solo la fase de ese nodo, sin esperar a detectar nada por su cuenta. Es el mismo patrón que usa Sparkplug para marcar como desactualizados los datos de un nodo caído (Sección 4.3). La fase vuelve a usarse cuando llega un mensaje nuevo.

Punto de operación

Se usan 20 nodos, con frecuencias naturales repartidas a intervalos iguales, y no sorteadas, dentro de un 20 % hacia cada lado de $\overline{f} = 0{,}2$ Hz, y $h = 500$ ms. Las frecuencias van de 1,005 a 1,508 rad/s, y para ellas el acoplamiento crítico exacto de la Ecuación 5 vale $K_c = 0{,}335$ rad/s. Al ser un conjunto fijo, K_c se calcula de forma exacta y cada corrida usa las mismas frecuencias. Los valores de K se expresan como múltiplos de ese K_c, el umbral sin retardo, también en las corridas en las que el umbral real es otro.

La elección responde a la capacidad de la computadora anfitriona y al modelo. Las 760 entregas por segundo son la mitad de las 1560 que la plataforma sostuvo de forma estable con 40 nodos publicando cada 1 s. Del lado del modelo, lo que importa es el producto de la frecuencia por la antigüedad. Con una antigüedad media cercana a $h/2 = 250$ ms y la frecuencia media de las frecuencias naturales, resulta $\Omega\tau \approx 0{,}31$ rad (0,29 rad con la frecuencia común que se mide después), lejos de $\Omega\tau = \pi/2$, el valor en el que deja de cumplirse la condición de estabilidad $\cos(\Omega\tau) > 0$ asociada a la Ecuación 8. Es el mismo valor que con 0,1 Hz y 1 s, pero con todos los transitorios, es decir, el tiempo que tarda el grupo en llegar al régimen, reducidos a la mitad.

Salvo en el barrido de K, donde interesa observar cómo se sincroniza el grupo, cada corrida arranca con las fases del estado enganchado que predice la teoría, $\theta_i - \psi = \arcsin(\Delta_i/u)$, donde Δ_i es la desintonía del nodo y $u = Kr$ es la fuerza con la que el grupo lo atrae, con lo cual no hace falta esperar el transitorio para aplicar cada perturbación.

Automatización y condiciones de red

Un programa en Python lee una matriz de corridas en JSON. Para cada corrida calcula las frecuencias y el acoplamiento, escribe la configuración en una copia de la imagen *flash* por nodo, levanta el broker, el servidor SNTP y el registrador, lanza las instancias de QEMU y espera a que todos los nodos terminen. El instante de inicio es una hora común, que cada nodo conoce por SNTP. Las acciones programadas, como detener el broker o cortar un enlace, se ejecutan a su instante y quedan registradas con su hora.

La pérdida y el corte de un enlace se aplican con `tc netem`, una función del kernel de Linux que emula condiciones de red como retardo y pérdida [80], en los dos sentidos de cada nodo. Un corte es una pérdida del 100 %. Las condiciones de red se aplican recién cuando todos los nodos terminaron de arrancar, para que la primera sincronización del reloj no se vea afectada.

Medición

La fase de cada nodo se escribe por su puerto serie (UART), una salida de texto que QEMU guarda en un archivo por nodo, cada 500 ms y desde la tarea de integración. El registrador no alcanza para esto, porque recibe las fases a través del broker y no ve nada cuando este se detiene o cuando un enlace está cortado, que son justamente los momentos de interés. Además, el puerto serie muestra la fase real aunque el nodo esté publicando con un esquema distinto. De esas muestras se reconstruye la fase continua de cada nodo, usando la velocidad para resolver las vueltas completas, y se interpola a una grilla común para calcular $r(t)$ y $\psi(t)$. El r en régimen es la media de $r(t)$ en el último 20 % de la corrida, y la frecuencia efectiva de cada nodo es la pendiente de su fase en esa ventana. Se considera que el grupo engancha si la mayor y la menor frecuencia efectiva difieren en menos de 0,01 rad/s. Los bytes, la anatomía del PUBLISH, la antigüedad de cada entrega y las retransmisiones se obtienen de las capturas en el puente, y los tiempos de detección, reconexión y última voluntad, de los registros de los nodos y del registrador.

Como `netem` actúa sobre los paquetes que salen de una interfaz, la pérdida de la bajada se aplica después del puente, a la salida hacia el nodo, y la de la subida antes de llegar a él. Por eso, la captura en el puente ve la bajada antes de que se apliquen las pérdidas y la subida después. Por eso, un segmento que se pierde en la subida no aparece en la captura, y las retransmisiones que se reportan corresponden a la bajada.

Simulación del modelo

Además de la emulación se usó una simulación en Python, sin red real, que integra la Ecuación 7 con un paso de 10 ms. Muestra qué hace el modelo de los nodos cuando la única imperfección de la comunicación es la que se le programa. Cada nodo publica su fase cada h, con un desfase propio sorteado dentro de h, y cada copia llega a cada receptor con un retardo. El receptor conserva la última fase recibida de cada nodo, y mientras no reciba ninguna, ese término no aporta a la sumatoria. El criterio de enganche es el mismo que en la emulación.

La simulación de referencia usa las frecuencias del experimento, $h = 500$ ms y 150 s de duración. Repite cada valor de K con 5 conjuntos de fases iniciales al azar y calcula r y las frecuencias efectivas en los últimos 30 s. El retardo de cada copia se sortea entre 13 y 17 ms, a partir de lo medido en la plataforma común con 39 nodos, y se redondea hacia arriba al paso siguiente de la simulación, por lo que resulta de 20 ms. Los análisis puntuales del umbral usan, en cambio, un retardo fijo de un paso (10 ms) y arrancan del estado enganchado teórico.

Diseño de las corridas

Cada mecanismo del stack se activa desde la configuración del nodo, sin recompilar, y cada variante se comparó contra la teoría o contra una corrida de referencia (Tabla 22). Cada condición se corrió una vez, salvo QoS con pérdida, que se corrió con tres semillas. Las corridas de umbral, de período, de QoS sin pérdida, las repeticiones de QoS y el corte con captura se agregaron después de analizar la primera tanda, para aislar efectos que esta no permitía separar. Las emulaciones aportan una realización por condición, y la dispersión entre condiciones iniciales se estima con la simulación del modelo. Con 5 fases iniciales distintas, r varía entre 0,863 y 0,871 a 1,2 K_c, y menos de 0,0001 a 1,5 y $2K_c$.

Tabla 22: Corridas del experimento. Salvo que se indique otra cosa, $N = 20$ y $K = 1{,}5\ K_c$.

Corrida	**Condición**	**Duración**
Calibración	$K = 1{,}5\ K_c$, sin eventos, con captura	60 s
Modelo	$K = 0{,}8$; 1,2; 1,5 y $2K_c$, con fases iniciales al azar	150 s
Iguales	Frecuencias iguales, $K = 0{,}4$ rad/s. Los nodos impares agregan al mensaje un campo 6, que los demás no conocen y deben ignorar (Sección 3.3)	90 s
Esquema cruzado	Los nodos impares intercambian los campos 2 y 3 al escribir y al leer. Como ambos son `float`, el mensaje se decodifica sin error, con datos equivocados (Sección 4)	120 s
Broker	Se detiene el broker durante 5 s y durante 20 s (Sección 4.5)	240 s
Unión	Un nodo arranca con los demás, pero se conecta al broker a los 45 s, sin que nadie más se reconfigure (Sección 1)	90 s
Keep-alive	Cortes de 2 s y 15 s del enlace de un nodo, con y sin última voluntad (dos corridas) (Sección 2.4)	150 s
QoS	QoS 0 y QoS 1, con 5 % de pérdida por sentido (dos corridas)	90 s
Alias	El broker usa alias de tópico en sus entregas (Sección 4), con captura	30 s
Escala	$N = 5$ y $N = 10$, con captura. Como cada N tiene su propio K_c, K vale 0,572 y 0,525 rad/s	45 s
Umbral	$K = 1{,}05$; 1,1; 1,15 y 1,25 K_c, desde el estado enganchado	200 s
Período	$h = 250$ ms, con $K = 1{,}1$ y 1,5 K_c, desde el estado enganchado	200 s
QoS sin pérdida	QoS 1 sin pérdida, con captura	90 s
Repetición de QoS	QoS 0 y QoS 1 con 5 % de pérdida, con otras dos semillas (cuatro corridas)	90 s
Corte con captura	Corte de 2 s de un nodo, keep-alive de 5 s y captura	60 s

Entorno y validez

El experimento se corrió en una computadora con un AMD Ryzen 5 5600G y 13 GB de RAM, con QEMU de Espressif 9.2.2, ESP-IDF 5.5.6, nanopb 0.4.9.1 y Mosquitto 2.1.2. Los tiempos internos de un ESP32 emulado no son los de una placa real, y la computadora anfitriona ejecuta todos los nodos a la vez. Por eso cada corrida registra si la tarea de integración se atrasó y a qué intervalo se producen las muestras de fase, y se controla la carga de la computadora, que durante la primera tanda de corridas estuvo en promedio un 88 % ociosa, con un mínimo del 66 %. Cada nodo resincroniza su reloj por SNTP cada 15 s. Como en el experimento anterior, el cliente SNTP de lwIP no descuenta el tiempo de viaje, de modo que la hora de cada nodo queda atrasada en el tiempo que tarda en llegar la respuesta. Ese error no afecta a la dinámica, que usa solo las fases y las diferencias de tiempo del propio nodo, y su efecto en las curvas de $r(t)$ es despreciable frente a los períodos de oscilación.

5.3.3. Resultados

Las 28 corridas terminaron sin fallos. La tarea de integración solo se atrasó en 4 pasos, todos en una misma corrida, sobre millones en total, y las muestras de fase de cada nodo llegaron cada 500 ms, con un máximo de 510 ms entre dos consecutivas. La corrida con alias de tópico se repitió una vez, porque en la primera el broker quedó limitado a sus 10 alias por defecto.

Sincronización con la red limpia

Partiendo de fases al azar y con la red sin perturbaciones, el grupo se comporta como predice el modelo de los nodos. Con K por debajo del umbral, r fluctúa sin estabilizarse, y por encima crece

hasta un valor que depende de K. Las corridas que arrancan del estado enganchado muestran que ese umbral no es el K_c sin retardo, ya que a 1,05, 1,1 y 1,15 K_c el grupo pierde el enganche, y a 1,25 K_c lo conserva. La Figura 7 compara el régimen medido con la autoconsistencia de la Ecuación 4, que no contempla retardo, con la condición con retardo de la Ecuación 9 y con la simulación del modelo de los nodos.

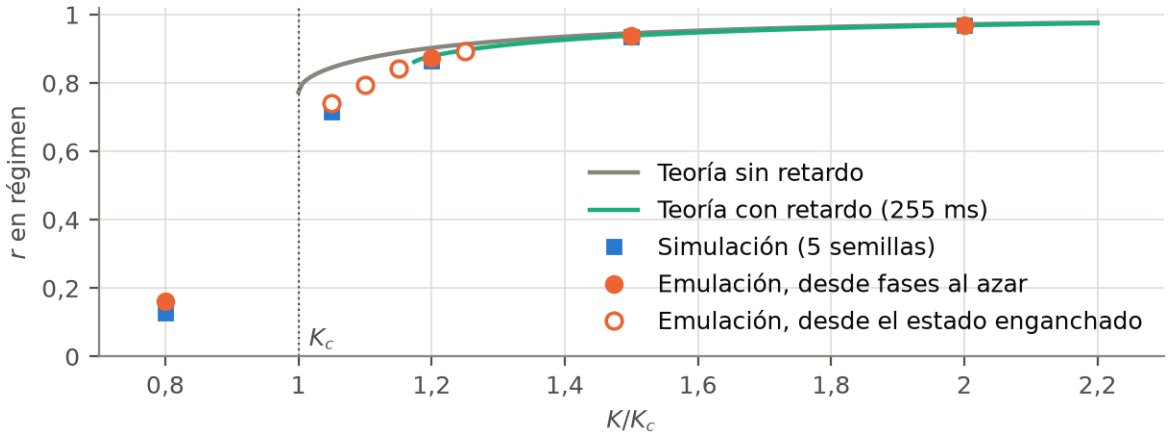


Figura 7: Parámetro de orden en régimen: teoría sin retardo y con retardo, simulación del modelo y emulación. Los círculos rellenos arrancan de fases al azar, y los vacíos, del estado enganchado teórico.

La corrida con frecuencias iguales, $K = 0{,}4$ rad/s y la mitad de los nodos publicando un campo adicional alcanzó $r = 1{,}000$, con una frecuencia común de 1,1510 rad/s. Para esa corrida, la frecuencia común que predice la condición con retardo, usando la antigüedad media estimada, es 1,1468 rad/s.

Período de publicación

Con $h = 250$ ms y la misma red, el grupo engancha a 1,1 K_c, con $r = 0{,}860$ y $\Omega = 1{,}219$ rad/s, un valor de K en el que con $h = 500$ ms no engancha. A 1,5 K_c resulta $r = 0{,}944$ y $\Omega = 1{,}195$ rad/s, frente a 0,937 y 1,142 rad/s con $h = 500$ ms.

Anatomía de una fase

La Tabla 23 descompone, a partir de la captura de la corrida de calibración, una entrega típica de una fase con identificador de dos cifras. A diferencia del experimento de la Sección 5.2, que cuenta bytes IP, aquí se cuentan las tramas completas, que agregan los 14 B del encabezado Ethernet.

Tabla 23: Composición de una trama que entrega una fase, sin alias de tópico.

Parte	Bytes	Proporción
Encabezados Ethernet, IP y TCP	54	54,5 %
Encabezado fijo MQTT, largo de tópico y largo de propiedades	5	5,1 %
Nombre del tópico	17	17,2 %
Mensaje de Protocol Buffers	23	23,2 %
Trama completa	**99**	**100 %**

Contrato de datos

En la corrida con un campo desconocido, los nodos recibieron las 3420 fases esperadas por nodo y la sincronía no se vio afectada, porque los lectores ignoraron el campo 6.

En la corrida con los campos 2 y 3 cruzados, cada nodo recibió las 4560 fases esperadas y ninguno informó un error, pero el grupo no se sincronizó (Figura 8). En la ventana final, r valió 0,159 en el conjunto, 0,428 entre los nodos con el esquema normal y 0,653 entre los que tenían el esquema cruzado.

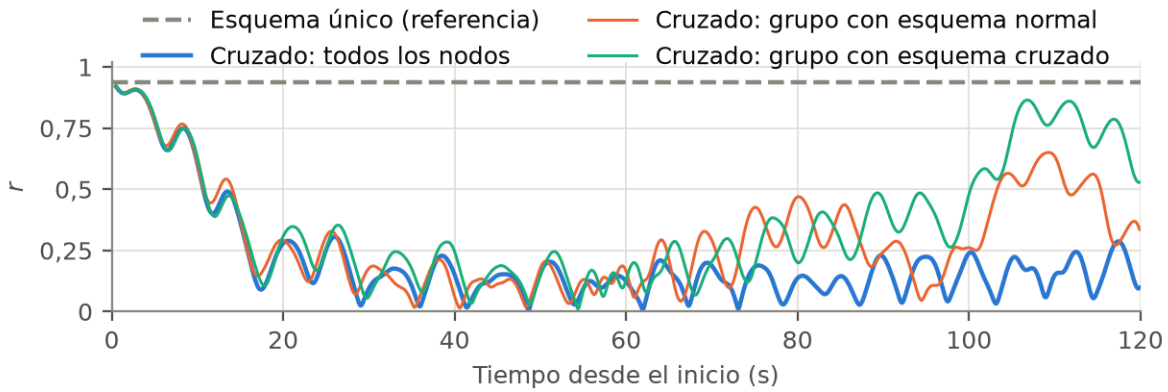


Figura 8: Parámetro de orden con la mitad de los nodos usando el esquema con los campos cruzados.

Caída del broker

La Figura 9 muestra el efecto de detener el broker. Antes de cada caída, r valía 0,937, y llegó a 0,013 en la de 5 s y a 0,004 en la de 20 s. En ambas caídas, los nodos se enteraron enseguida y, cuando el broker volvió, tardaron unos 10 s más en reconectarse (10,9 y 9,8 s desde la vuelta real del broker). La primera caída estaba programada en 5 s, pero duró 4,2 s, porque la orden de detener el broker se ejecutó 1,05 s tarde; la segunda duró 20,2 s. La figura usa los instantes programados. Desde que volvió el broker, r tardó 27,7 y 37,0 s en mantenerse por encima del 95 % de su valor previo.

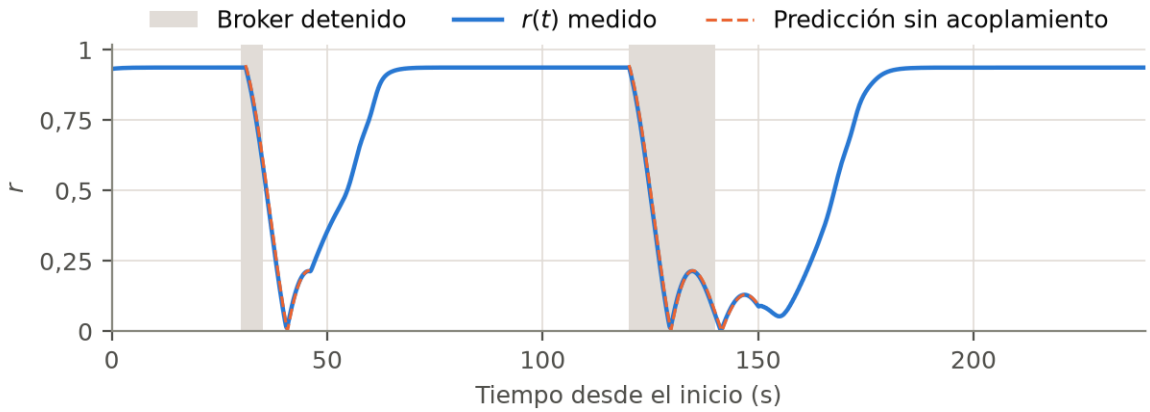


Figura 9: Parámetro de orden durante dos caídas del broker, y la predicción de la Ecuación 11 mientras los nodos están desconectados.

Keep-alive y última voluntad

Con un keep-alive de 5 s, se cortó el enlace de un nodo durante 2 s y después el de otro durante 15 s, con y sin última voluntad (Tabla 24). El efecto sobre r es moderado, porque solo un nodo de 20 queda afuera, por lo que la Figura 10 muestra la fase del nodo cortado respecto de la fase media del grupo.

Tabla 24: Efecto de cortar el enlace de un nodo. La detección y la publicación se miden desde el comienzo del corte, la reconexión desde su fin, y el desfase es el del nodo respecto de ψ al terminar.

Corte	Última voluntad	El nodo detecta	Reconexión tras el corte	Voluntad publicada	r mínimo	Desfase final (rad)
2 s	No	4,5 s	17,5 s	–	0,897	0,51
2 s	Sí	3,1 s	16,1 s	3,3 s	0,917	0,51
15 s	No	4,1 s	4,1 s	–	0,874	1,07
15 s	Sí	5,0 s	5,0 s	6,9 s	0,854	1,40

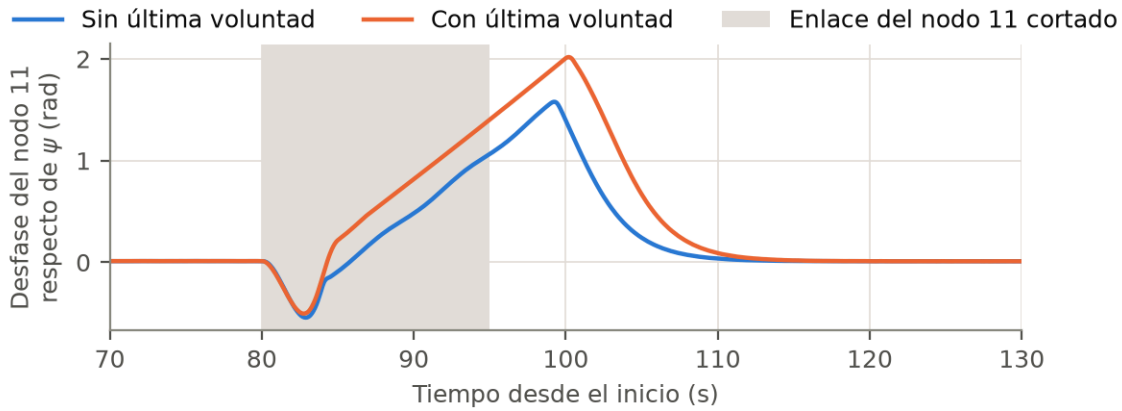


Figura 10: Desfase del nodo 11 respecto de la fase media durante su corte de 15 s.

Incorporación de un nodo

El nodo 20 arrancó junto con los demás, pero se conectó al broker a los 45 s. Hasta entonces avanzó a su frecuencia natural y, 7,5 s después de conectarse, su fase ya estaba estabilizada en el valor que le corresponde dentro del grupo, es decir, a menos de 0,15 rad de su desfase final, sin que ningún otro nodo cambiara su configuración.

QoS con pérdida de paquetes

Con un 5 % de pérdida en cada sentido, los dos niveles de QoS se comportaron de forma muy distinta (Figura 11 y Tabla 25). No hubo desconexiones en ninguna de las dos corridas. La antigüedad de una entrega es la diferencia entre el instante en que el broker la envía a la red y el instante en que se calculó la fase. Con otras dos semillas, QoS 0 alcanzó $r = 0{,}913$ y 0,878, y enganchó en la primera de ellas; QoS 1 quedó en $r = 0{,}094$ y 0,064, con antigüedades medianas de 13,1 y 12,5 s.

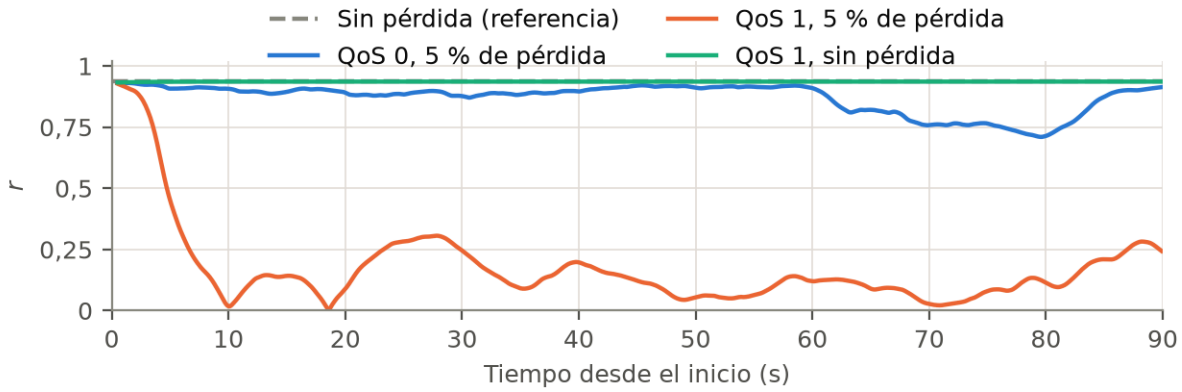


Figura 11: Parámetro de orden con 5 % de pérdida por sentido, para QoS 0 y QoS 1, y con QoS 1 sin pérdida.

Tabla 25: QoS 0 y QoS 1 con 5 % de pérdida por sentido (semilla 1), y QoS 1 sin pérdida. Sin pérdida y con QoS 0, cada entrega cuesta 133 B.

	QoS 0, 5 %	**QoS 1, 5 %**	**QoS 1, sin pérdida**
r en régimen	0,800	0,148	0,936
Engancha	No	No	Sí
Fases recibidas por nodo (de 3420)	3409 a 3416	1759 a 2611	3418 y 3419
Antigüedad de la entrega, mediana	8,6 ms	13,4 s	21,7 ms
Antigüedad de la entrega, percentil 95	1,2 s	29,2 s	46,9 ms
Segmentos retransmitidos en la bajada	3041	3421	0
Tráfico total	101,5 kB/s	88,0 kB/s	164,2 kB/s
Bytes por entrega en la captura	134 B	149 B	216 B

Alias de tópico

Con alias de tópico en las entregas del broker, el PUBLISH que recibe cada nodo pasa de 45 a 31 bytes, y su trama de 99 a 85. El 98 % de las entregas de la corrida ya lleva el alias, y solo la primera por cada par de nodos transporta todavía el tópico. El tráfico de bajada por nodo baja de 3767 a 3269 B/s, un 13,2 %, y el total de 100,8 a 91,0 kB/s, un 9,7 %.

Carga del broker

Sin perturbaciones, la cantidad de entregas crece con $N(N-1)$, y la mayor parte del tráfico sale del broker (Tabla 26). Cada nodo publica la misma cantidad de mensajes sin importar cuántos otros haya; la subida crece menos que la bajada y también incluye las confirmaciones TCP de lo que el nodo recibe.

Tabla 26: Tráfico y antigüedad de las entregas según la cantidad de nodos, con $h = 500$ ms.

N	**Entregas/s**	**Total (kB/s)**	**Subida por nodo (B/s)**	**Bajada por nodo (B/s)**	**Antigüedad mediana (ms)**
5	40	6,7	462	876	2,3
10	180	25,6	736	1820	3,7
20	760	100,8	1274	3767	5,0

5.3.4. Análisis de resultados

Umbral de sincronización y efecto del stack

Con la red limpia, la emulación coincide con la simulación del modelo de los nodos, con diferencias de hasta 0,007 en r a 1,2, 1,5 y 2 K_c. Las dos quedan por debajo de la teoría sin retardo, y la diferencia de la emulación respecto de ella crece al acercarse a K_c, ya que es de 0,031 a 1,2 K_c y de 0,0035 a 2 K_c. Con una antigüedad media de unos 250 ms, la información vieja desplaza el umbral hacia arriba, y ese corrimiento se midió. El grupo no engancha a 1,05, 1,1 ni 1,15 K_c, aunque sin retardo allí existe un estado enganchado estable, y sí engancha a 1,2 y 1,25 K_c. La corrida de 1,15 K_c queda muy cerca del criterio de enganche, con una dispersión de 0,012 rad/s contra el límite de 0,01. El umbral medido queda así entre 1,15 y 1,2 K_c, en acuerdo con el 1,17 K_c que predice la Ecuación 9 y con la simulación del modelo de los nodos con retardo fijo de 10 ms, que deja de derivar entre 1,18 y 1,19 K_c. El corrimiento, cercano al 18 %, es más de tres veces el 5 % que sugeriría el factor $1/\cos(\Omega\tau)$, porque la mayor parte se debe a la baja de la frecuencia común. Una consecuencia es que la corrida de 1,2 K_c está apenas por encima del umbral real, y eso explica que su diferencia con la teoría sin retardo sea la mayor.

La frecuencia común también refleja el retardo. Baja de 1,176 a 1,098 rad/s al pasar de 1,2 a 2 K_c, por debajo de la media de 1,257 rad/s, y el freno aumenta con K, como indica la Ecuación 10. Resuelta con $\tau = 255$ ms, esa relación reproduce las frecuencias medidas con un error menor que el 0,5 %, y los valores de r con diferencias de hasta 0,007.

En este punto de operación, la antigüedad con la que el broker emite cada entrega, de unos 5 ms, es muy pequeña frente a los 250 ms de envejecimiento entre publicaciones. Esa medida no incluye el último tramo hasta el nodo ni su procesamiento, pero eso no cambia el orden de magnitud. De los dos componentes de la antigüedad, el que depende del período de publicación, una decisión de la aplicación, es por lejos el mayor. Las corridas con $h = 250$ ms aíslan ese efecto. Para esa antigüedad, de unos 131 ms, la Ecuación 9 predice el umbral en 1,06 K_c, por lo que 1,1 K_c ya debería enganchar, y así ocurre, mientras que con $h = 500$ ms no engancha. También predice r y Ω con diferencias de a lo sumo 0,001 y 0,004 rad/s. Reducir el período de publicación a la mitad, sin tocar la red, es lo que permitió enganchar con menor acoplamiento, al costo de duplicar la cantidad de mensajes.

Composición de los bytes

De los 99 bytes de una entrega, 23 son la fase. Protocol Buffers cumple lo prometido, ya que codifica cinco campos, entre ellos un `fixed64` y dos `float`, en 23 bytes. El encabezado fijo de MQTT también es pequeño. Pero el costo restante se reparte en 54 bytes de encabezados de las capas inferiores y 17 del nombre del tópico, que se repite en cada mensaje y pesa casi tanto como el dato, dos partes que MQTT y Protocol Buffers no reducen por sí solos. En mensajes tan cortos, el costo por mensaje queda dominado por lo que rodea al dato. Por eso resulta adecuado limitar la frecuencia de publicación, como hacen los casos de la Sección 4.5, y resulta útil el alias de tópico, que recorta 14 bytes por entrega a cambio de que el broker mantenga una tabla por conexión. Como se aplica solo por conexión, y el alias se usa en la mayor parte de las entregas, en este patrón todos-a-todos alcanza con activarlo en el tramo del broker hacia los nodos.

El contrato de datos

El ensayo muestra las dos caras de un mensaje identificado por números de campo. Agregar un campo es inocuo, porque los lectores que no lo conocen lo ignoran, y el sistema sigue

funcionando (Sección 3.3). Cruzar dos campos del mismo tipo, en cambio, no se detecta en ninguna capa. El broker no interpreta la carga útil, TCP la entrega intacta y el decodificador encuentra dos `float` válidos. La única señal es que la aplicación deja de funcionar. El modelo explica por qué no engancha cada mitad. Cada nodo solo se acopla de forma coherente con la suya, con un acoplamiento efectivo $K/2 = 0{,}251$ rad/s, por debajo de los 0,3315 rad/s del K_c de cada subgrupo. Los mensajes de la otra mitad traen un valor cercano a la velocidad, casi constante, que tira hacia un ángulo fijo. Su amplitud, de $K/2$, es menor que la frecuencia natural de cualquier nodo, por lo que no detiene la rotación, sino que modula su velocidad. Es la consecuencia práctica de que el mensaje no sea autodescriptivo (Sección 3). La compatibilidad depende, entonces, de una disciplina sobre el esquema, que el protocolo no puede verificar.

El broker como punto único de falla

Con el broker detenido, ningún nodo puede saber nada de los demás, y el grupo pierde la sincronía en pocos segundos, como anticipa la Sección 1. Una vez desconectados, cada nodo oscila libremente, y $r(t)$ sigue casi exactamente la Ecuación 11. La diferencia cuadrática media con la curva predicha es de 0,014, y el primer mínimo aparece a los 9,6 s del corte, contra 9,8 s de la predicción (Figura 9). Con $b = 0{,}251$ rad/s, el tiempo característico π/b es de 12,5 s. Tanto en la caída de 5 s como en la de 20 s, r llegó cerca de 0, y la recuperación fue más larga en la segunda. En ambas, la reconexión llegó en múltiplos de 15 s desde la desconexión, a los 15 s en la primera caída y a los 30 s en la segunda, después de un intento fallido a los 15 s. Por eso los nodos tardaron unos 10 s en reconectarse después de que volvió el broker, una demora que no es el período de reintento, sino lo que restaba de él. Durante ese tiempo siguen desacoplados aunque el broker ya esté disponible, de modo que la caída del servicio dura, para la aplicación, más que la del broker. El experimento no probó otros valores de ese período. Una vez reconectados, el grupo vuelve a sincronizarse sin intervención, gracias a que cada nodo se vuelve a suscribir por su cuenta. Descontada la espera hasta reconectarse, tarda entre 17 y 27 s, del mismo orden que los 13 s que tardó en sincronizarse desde fases al azar en las corridas con la red limpia, porque tras una caída larga las fases quedan prácticamente dispersas.

Cortes de un enlace y última voluntad

El corte de 15 s se comporta como anticipa la Tabla 8. El nodo y el broker cierran la conexión, la última voluntad se publica a los 6,9 s, muy cerca de los 7,5 s que corresponden a una vez y media el keep-alive, y el nodo se reconecta 4 a 5 s después de que vuelve el enlace, es decir, 15 s después de haber detectado la desconexión.

El corte de 2 s muestra un matiz que esa tabla no contempla. Aunque dura menos que el umbral de 7,5 s del broker, el nodo igual cerró la conexión, a los 3 a 4,5 s de comenzado el corte. El umbral de una vez y media el keep-alive es el del broker, pero el cliente esp-mqtt envía un PINGREQ a la mitad del intervalo y da la conexión por perdida si la respuesta no llega antes de que se complete el intervalo. Con un keep-alive de 5 s, la respuesta tiene un margen de 2,5 s, y un corte breve alcanza para perder ese intercambio. Una corrida adicional con captura, con el mismo corte de 2 s, muestra por qué el fin del corte no alcanza para evitarlo. El último PINGRESP pasa 2,2 s antes del corte y el siguiente PINGREQ no llega al broker. Medido desde el comienzo programado del corte, el broker retransmite a los 0,3, 0,8 y 1,7 s, con esperas cada vez mayores, de modo que su intento siguiente recién sale a los 3,5 s. Así, aunque el enlace vuelve a los 2 s, durante más de un segundo no circula nada por la conexión, y a los 3,1 s el cliente agota su plazo y la cierra. Es el mismo efecto de los reintentos de TCP cada vez más espaciados de la Sección 2.4. La conexión del lado del cliente es, entonces, más frágil de lo que indica el umbral del broker, y la reconexión cuesta más que el propio corte. Como el cliente

espera 15 s desde que detecta la desconexión, el nodo tardó entre 16 y 17,5 s en volver después de terminado el corte. Además, como el cliente cierra la conexión sin enviar DISCONNECT, el broker publica la última voluntad, a los 3,3 s del comienzo del corte, aunque el enlace ya esté restablecido.

Con un solo nodo cortado sobre 20, el efecto en el grupo es pequeño, y r baja como mucho a 0,854. El comportamiento del nodo cortado coincide con el planteo. Mientras no detecta la desconexión, sigue acoplado a fases congeladas y se atrasa hasta 0,5 rad respecto del grupo. Una vez desconectado, avanza a su frecuencia natural y se adelanta a 0,115 y 0,117 rad/s, cerca de los 0,128 rad/s de $\omega_i - \Omega$. La última voluntad cumplió su función de avisar la caída, pero en estas corridas no mejoró la sincronía, ya que los valores con y sin ella son del mismo orden y, con una sola corrida por condición, sus diferencias no permiten sacar una conclusión.

QoS y frescura de los datos

El resultado más llamativo es que, con pérdida, QoS 1 empeoró la sincronización de forma drástica respecto de QoS 0, que ya sufría. Con QoS 0, las retransmisiones de TCP hacen que algunas fases lleguen tarde. La mediana se mantiene en milisegundos, pero el percentil 95 llega a 1,2 s. Casi todas las fases llegan. En la corrida de la tabla, dos nodos, el 18 y el 20, entre los de mayor desintonía, se separan del grupo, y el valor de r de 0,80 corresponde a una ventana final que coincide con el peor tramo, ya que durante los primeros 60 s r se mantuvo entre 0,87 y 0,93. El grupo enganchó en una sola de las tres corridas, de modo que, con esta pérdida, queda en el límite del enganche.

Con QoS 1, el broker entregó las fases con una antigüedad mediana de 13,4 s, y cada nodo recibió entre la mitad y las tres cuartas partes de lo publicado, sin que se cortara ninguna conexión. La captura muestra una acumulación en el broker. En los 20 nodos, las entregas sin confirmar llegan a 20, el límite de Mosquitto, y permanecen en él entre el 87 % y el 96 % del tiempo, es decir, la ventana está saturada casi siempre. Con ese límite, cuando una pérdida frena la conexión TCP de un nodo, tampoco salen mensajes nuevos hacia él. El broker atiende unas 600 entregas por segundo contra las 760 que se publican, y el resto se acumula en la cola, que está ampliada a 100.000 mensajes. La antigüedad crece entonces de forma sostenida, desde una mediana de unos 2 s en los primeros 10 s hasta unos 26 s en los últimos 10 s, de modo que los 13,4 s de la tabla no describen un régimen estable. Las fases que faltan no se perdieron, sino que quedaron en cola al terminar la corrida.

Las condiciones de estabilidad de la Ecuación 8 no aplican aquí, porque valen para un retardo constante y son periódicas en τ. En este caso τ crece durante toda la corrida y varía entre una entrega y otra, y no existe un retardo alrededor del cual el grupo pueda engancharse.

La corrida sin pérdida separa los efectos. QoS 1 sin pérdida engancha igual que QoS 0, con $r = 0{,}936$, sin acumulación, con a lo sumo 5 mensajes en tránsito y una antigüedad mediana de 21,7 ms. El problema no es QoS 1 en sí, sino su combinación con la pérdida. El límite de 20 mensajes no se varió, por lo que el experimento no lo aísla como causa única, aunque la saturación permanente lo señala.

La conclusión práctica coincide con la de la Sección 4.5 y la amplía. La confirmación de entrega asegura que un mensaje llegue, y en esta aplicación, cuando hay pérdida, esa garantía es contraproducente, porque una fase vieja no sirve y además retrasa a las más nuevas. Sin pérdida no perjudicó la sincronía, pero cada entrega costó unos 216 B, frente a unos 133 B de QoS 0 sin pérdida, cerca de un 63 % más. Para datos que se reemplazan continuamente, importa la frescura y no la entrega de cada muestra.

Contraste de hipótesis

La Tabla 27 contrasta las hipótesis del planteo con los resultados.

Tabla 27: Contraste de las hipótesis con los resultados.

Hipótesis	**Resultado**
Por encima de K_c, los nodos se enganchan con un r algo menor que el teórico.	**Se confirma en parte.** Engancharon a 1,2, 1,25, 1,5 y 2 K_c, con r 0,0035 a 0,031 menor que la teoría sin retardo y hasta 0,007 respecto de la simulación. Pero entre K_c y 1,15 K_c no engancharon: el umbral real está entre 1,15 y 1,2 K_c.
Un campo desconocido no afecta la sincronía, y dos campos cruzados se decodifican sin error, pero impiden sincronizar.	**Se confirma.** Con el campo adicional $r = 1{,}000$. Con los campos cruzados, $r = 0{,}159$, sin errores de decodificación.
Al caer el broker, el grupo pierde la sincronía y la recupera sin intervención.	**Se confirma.** r cae hasta 0,013 y se recupera por sí solo; la caída sigue la oscilación libre prevista.
Un corte menor que una vez y media el keep-alive no cierra la conexión, y uno mayor la cierra y publica la última voluntad.	**Se confirma a medias.** El corte de 15 s cerró la conexión y publicó la voluntad a los 6,9 s. El de 2 s también cerró la conexión, del lado del cliente.
Con pérdida, QoS 1 no mejora la sincronización y genera más tráfico que QoS 0.	**Se confirma.** QoS 1 fue mucho peor (r de 0,148 contra 0,800, en la primera semilla). El tráfico total fue menor (88,0 contra 101,5 kB/s) solo porque se entregaron menos fases: por entrega costó 149 B contra 134 B. Sin pérdida, QoS 1 no perjudica la sincronía.
El alias de tópico reduce unos 13 bytes por entrega.	**Se confirma.** El PUBLISH entregado pasa de 45 a 31 bytes, 14 menos, y el tráfico total baja un 9,7 %.

Limitaciones

- **Una realización por condición.** Salvo QoS con pérdida, cada ensayo se corrió una vez. Las diferencias grandes, como las de QoS, son claras, pero las pequeñas, como el efecto de la última voluntad, no se pueden distinguir de la variación entre corridas.
- **Emulación.** El broker, los 20 nodos y la red comparten una computadora. Los retardos internos no son los de una placa física con Wi-Fi.
- **Perturbaciones sintéticas.** La pérdida de 5 % es aleatoria e independiente y no representa la de un enlace real, en el que suele darse en ráfagas.
- **Un punto de operación acotado.** $\overline{f}$ y el ancho de las frecuencias se fijaron, y h solo se varió en dos corridas, por la capacidad de la computadora y el tiempo disponible. Otros valores cambian el peso relativo del período de publicación y de la red.
- **Sin TLS.** Las corridas no usan cifrado, cuyo costo se analiza en el experimento de la Sección 5.2.

Implicaciones

Para una aplicación en la que todos los nodos necesitan el dato de todos, los resultados sugieren los siguientes criterios:

- La frescura importa más que la entrega de cada mensaje. Conviene usar QoS 0 y descartar lo viejo con un número de secuencia. QoS 1 no perjudicó la sincronía sin pérdida, pero aumentó el costo por mensaje y, con pérdida, generó una cola que crece sin límite si el broker la permite.

- El período de publicación pesó mucho más que el retardo de la red en este punto de operación, y corrió el umbral de enganche en cerca de un 18 %. Se lo debe elegir en función de la dinámica de la aplicación, y no solo del ancho de banda.
- Tras una interrupción, el tiempo de reconexión del cliente puede pesar más que la propia interrupción, y es un parámetro que debe ajustarse. Hay que medirlo en el cliente real, ya que en esp-mqtt un valor configurado de 10 s dio un reintento efectivo de 15 s.
- Un keep-alive muy corto detecta antes una caída, pero vuelve frágil la conexión ante cortes breves.
- El esquema es parte del sistema. Los cambios incompatibles no producen errores, por lo que necesitan control de versiones y pruebas propias.

6. Mapeo tecnológico

Los experimentos se realizaron sobre placas emuladas, una red simulada y un broker corriendo en la misma computadora que el emulador. Llevar el sistema a un despliegue real requiere reemplazar cada una de esas piezas por un equipo o un servicio que se pueda comprar o contratar. A continuación se analizan los costos que este paso implica para el escenario del Experimento 1 (Sección 5.2), con ocho nodos que envían una lectura cada 10 s, la misma frecuencia que se usó para proyectar el tráfico en la Tabla 21. El criterio no es buscar la alternativa de mayor prestación, sino una solución viable que equilibre la calidad técnica, el plazo de implementación y el presupuesto. Los precios, a fin de generalizar, se expresan en dólares estadounidenses, son aproximados y corresponden a noviembre de 2026, sin impuestos ni costos de importación.

6.1. Nodos

Cada nodo se construye sobre el mismo microcontrolador que se emuló en los experimentos, el ESP32 de Espressif. Al tratarse del mismo chip, el firmware se compila sin cambios, salvo por la interfaz de red. La emulación conectaba cada nodo por Ethernet, mientras que el chip físico trae integrada un módulo Wi-Fi. El ESP32 se comercializa montado en placas de distintos fabricantes, y una placa de referencia como la ESP32-DevKitC, del propio Espressif, cuesta alrededor de USD 10 [81].

Cuando el lugar de instalación no cuenta con una red Wi-Fi, el nodo necesita conectarse a la red celular. Para ese caso se considera la placa LilyGO T-A7670SA, que combina el mismo microcontrolador con un módem celular y cuesta alrededor de USD 28 [82]. El módem opera en LTE, el estándar de telefonía celular de cuarta generación, en su categoría 1, una variante de LTE pensada para dispositivos que transmiten a baja velocidad y que funciona sobre la red 4G común de los operadores. Existen variantes más eficientes en consumo, como LTE-M, pero su disponibilidad en Argentina no está confirmada para todos los operadores, por lo que la categoría 1 resulta la opción de menor riesgo.

6.2. Conectividad

Con Wi-Fi, el nodo aprovecha la red ya existente en el lugar y no agrega costo de conectividad. Como es el dispositivo el que abre la conexión hacia el broker, no necesita aceptar conexiones entrantes y funciona detrás del NAT y del firewall de esa red (Sección 2.1).

Con conectividad celular, en cambio, cada nodo requiere un plan de datos. Para aplicaciones industriales o de Internet de las Cosas no corresponden los planes de telefonía personal, sino los planes M2M (*machine to machine*), pensados para dispositivos. Los operadores locales no

publican las tarifas de estos planes, pero los proveedores globales de tarjetas SIM para IoT sí lo hacen. Su precio depende principalmente del volumen de datos transferido, al que algunos suman un cargo fijo mensual por tarjeta.

En este punto, las mediciones del Experimento 1 se traducen directamente en dinero. El operador factura los bytes IP que atraviesan su red, con los encabezados de TCP y TLS incluidos, que son los que se midieron. Con MQTT sobre TLS, una lectura ocupa unos 140 B (Tabla 18), lo que equivale a unos 36 MB por mes en cada nodo. Con la tarifa de Hologram, por ejemplo, ese volumen cuesta alrededor de USD 2 por nodo y por mes [83]. Las decisiones analizadas en el experimento se reflejan entonces en la factura. Enviar el mismo contenido en JSON aumentaría ese volumen en alrededor de un 40 %, y usar HTTP con conexión reutilizada lo multiplicaría casi por cuatro. Este volumen es, de todos modos, un piso, porque en una red celular real se suman las reconexiones, cada una con un costo fijo de unos 3 KB entre los handshakes de TCP y TLS, el CONNECT y el cierre (Tabla 18), las consultas DNS y los mensajes de keep-alive.

Las redes LoRa, presentadas entre los casos de uso (Sección 4.4), son otra alternativa para zonas sin cobertura celular, pero cambian la arquitectura del nodo y quedan fuera de este relevamiento.

6.3. Broker

Para el broker se comparan dos alternativas. La primera es un servidor propio que ejecute Mosquitto, el mismo broker usado en los experimentos. Un servidor virtual privado (VPS), es decir, una máquina virtual alquilada a un proveedor de nube, de la menor capacidad disponible alcanza holgadamente para ocho nodos y cuesta alrededor de USD 5 por mes [84], mientras que Mosquitto es software libre y no tiene costo de licencia. Conviene alojarlo en la región del proveedor más cercana a los dispositivos, que en este caso es São Paulo, porque así el trayecto atraviesa menos sistemas autónomos y queda menos expuesto a la convergencia de BGP (Sección 2.3).

La segunda alternativa es un servicio administrado, en el que el proveedor opera el broker y cobra según el uso. AWS IoT Core, por ejemplo, costaría alrededor de USD 2 por mes en este escenario, pero su forma de facturar contrasta con la del plan de datos. El servicio cobra por mensaje y mide cada uno en bloques de 5 KB [85], de modo que una lectura de 18 B en Protocol Buffers cuesta lo mismo que una de 72 B en JSON. Con este tipo de tarifa, achicar el mensaje no reduce el costo, y lo único que lo reduce es enviar menos mensajes.

Se elige el servidor propio. A esta escala la diferencia de precio es mínima, y la decisión la definen el plazo y la continuidad con lo ya probado, ya que se trata del mismo broker y la misma configuración del experimento, sin necesidad de adaptar el sistema a un servicio distinto.

6.4. Costo del sistema

La Tabla 28 resume los componentes relevados.

Tabla 28: Componentes del sistema y precios aproximados en dólares estadounidenses (noviembre de 2026, sin impuestos).

Componente	**Equipo o servicio**	**Precio aproximado**
Nodo con Wi-Fi	Placa con ESP32 (por ejemplo, ESP32-DevKitC)	USD 10
Nodo celular	Placa LilyGO T-A7670SA (ESP32 y módem LTE)	USD 28
Datos celulares	Plan IoT de un proveedor global de SIM	USD 2 por mes por nodo, más unos USD 3 por SIM
Broker	VPS en São Paulo con Mosquitto	USD 5 por mes

Con conectividad Wi-Fi, poner en marcha los ocho nodos requiere unos USD 80 en placas y USD 5 por mes para el broker. Con conectividad celular, la inversión inicial sube a unos USD 250 entre placas y tarjetas SIM, y el costo mensual a unos USD 21, de los cuales 16 corresponden a los datos. En ambos casos, el sistema estudiado puede implementarse con equipos de bajo costo y de amplia disponibilidad. La principal incertidumbre no es el precio sino la cobertura. Ninguno de los proveedores globales con tarifas públicas declara operar sobre la red de Personal, por lo que antes de un despliegue habría que verificar qué operador tiene señal en el lugar de instalación.

6.5. Escalabilidad

El escenario de ocho nodos es solo un caso de uso entre muchos posibles, y cada componente de la Tabla 28 crece con una variable distinta.

- Las placas y las tarjetas SIM crecen con la cantidad de nodos y se pagan una sola vez.
- El cargo fijo mensual de las tarjetas crece con la cantidad de nodos.
- Los datos celulares crecen con la cantidad de nodos, la frecuencia de envío y el tamaño de cada lectura.
- El broker contratado crece con la cantidad de mensajes, sin importar su tamaño.
- El broker propio crece de a escalones, según el tamaño de servidor que exige la carga, y por transferencia solo cuando el tráfico supera el volumen incluido en el plan.

Según el caso de uso, el costo total queda dominado por uno u otro de estos componentes.

Cuando se supervisan variables que cambian lentamente, como una temperatura o el nivel de un tanque, basta con enviar una lectura por minuto o incluso menos. Un nodo celular con esa frecuencia transfiere unos 6 MB por mes y, con las tarifas relevadas, paga alrededor de USD 1,2, de los cuales USD 1 corresponde al cargo fijo de la tarjeta [83]. En este caso, reducir el tamaño de los mensajes casi no modifica la factura, y el costo lo definen el hardware y la cantidad de nodos.

La situación se invierte en una flota grande conectada por celular que envía datos con frecuencia, como la de mil nodos con una lectura cada 10 s considerada en la Tabla 21. Los datos pasan a ser el componente principal, del orden de USD 2000 por mes para toda la flota, y cada decisión del stack se multiplica por la cantidad de nodos. Enviar JSON en lugar de Protocol Buffers aumentaría el volumen de datos de toda la flota en alrededor de un 40 %, y usar HTTP en lugar de MQTT lo multiplicaría casi por cuatro, con el consiguiente aumento de la parte del costo que depende del volumen. También cambia la elección del broker. Con esa carga, un servicio que cobra por mensaje costaría alrededor de USD 260 por mes. Un servidor propio también

tiene costos que crecen con el uso, por el cómputo y la transferencia de datos, pero lo hacen de a escalones. El tráfico de toda la flota, de algunas decenas de gigabytes por mes, queda muy por debajo del volumen que incluyen incluso los planes más chicos, y aun un servidor varias veces más grande que el de la Tabla 28 cuesta una fracción del servicio contratado. El servidor propio pasa a ser así la alternativa más económica, a pesar del trabajo que implica operarlo. A esta escala, de todos modos, los precios de lista pierden representatividad, porque los operadores negocian los planes M2M con descuentos por volumen, y las cifras deben leerse como órdenes de magnitud.

En un depósito con robots móviles (Sección 4.2), el tráfico también es intenso, porque cada robot publica su posición y su estado de forma continua, pero viaja por la red Wi-Fi de la instalación, que no cobra por byte. El costo de comunicación se reduce al broker y resulta despreciable frente al valor de los robots. En este caso, la ventaja del stack no se mide en dinero, sino en la latencia y en el uso de la red.

La mensajería (Sección 4.1) se ubica en el extremo opuesto. Los datos viajan por el plan de telefonía de cada usuario, que es quien paga el tramo celular, y el costo de la empresa se concentra en mantener conexiones persistentes con millones de dispositivos móviles. A esa escala, el broker deja de ser un producto que se contrata y pasa a desarrollarse a medida, como muestra la evolución de Messenger.

Estos casos muestran que el ahorro de bytes del stack se convierte en dinero solo cuando la conexión se cobra por volumen y la frecuencia de envío es alta. En los demás, la elección de MQTT y Protocol Buffers se justifica por la latencia y el uso de la red más que por el costo.

Conclusiones

El trabajo se propuso explicar por qué MQTT y Protocol Buffers resultan adecuados para sostener una comunicación continua sobre redes que el diseñador no controla. La respuesta que surge del análisis y de las mediciones es que esa adecuación no proviene principalmente del tamaño de los encabezados ni de la codificación binaria, sino de una decisión de arquitectura donde el dispositivo abre una única conexión hacia el broker y la mantiene abierta. Esa conexión le permite atravesar NAT y firewalls sin aceptar conexiones entrantes, reparte el costo del handshake de TLS entre todos los mensajes y le da al broker un canal para entregar datos sin que el dispositivo los pida. La misma decisión concentra también las debilidades del stack, que dependen de que esa conexión y el broker sigan en pie. A continuación se retoma cada uno de los objetivos específicos.

El modelo publicador/suscriptor desacopla a las partes en el espacio, en el tiempo y en la sincronización, y MQTT lo implementa con un broker que distribuye los mensajes según su tópico, con niveles de calidad de servicio y con sesiones que sobreviven a la desconexión del cliente. Protocol Buffers lo complementa con una codificación binaria compacta, generada a partir de un esquema que comparten los participantes. La comparación con las alternativas no dio un ganador absoluto, sino un ámbito de uso para cada tecnología. MQTT es preferible cuando se necesita una entrega confiable en ambos sentidos con dispositivos detrás de NAT o firewalls; CoAP, para dispositivos muy restringidos que envían actualizaciones pequeñas sin necesidad de entrega garantizada, y HTTP, para intercambios puntuales iniciados por el cliente o integrados con la Web. Del mismo modo, Protocol Buffers conviene cuando el tamaño de los mensajes es crítico y es posible distribuir un esquema común, y JSON, cuando importan la legibilidad y la interoperabilidad sin acuerdos previos. Los casos de uso analizados siguen ese mismo criterio.

En la capa de transporte, el análisis mostró que MQTT debe resolver por su cuenta algo que TCP no ofrece en la práctica y es saber si el otro extremo sigue alcanzable. El keep-alive de MQTT cumple esa función, y la elección de su intervalo es un compromiso entre detectar rápido una caída y tolerar los cortes breves de la red. Un valor de entre 30 y 60 s detecta una caída en menos de un minuto y medio y absorbe los cortes breves del ruteo, mientras que uno muy corto vuelve frágil la conexión, como se vio en el segundo experimento, en el que, con un intervalo de 5 s, un corte de 2 s alcanzó para que el cliente la cerrara. El costo de TLS, por su parte, depende de cuántos mensajes viajan por cada conexión. En las conexiones persistentes agregó entre 46 y 97 B por lectura y menos de 0,6 ms, mientras que con una conexión por lectura multiplicó por 3,1 los bytes. TLS, además, protege cada tramo por separado y no el recorrido completo, porque el broker ve el contenido en claro.

En cuanto a la confiabilidad, TCP y los niveles de calidad de servicio de MQTT cubren riesgos distintos. Mientras la conexión se mantiene, TCP recupera los segmentos perdidos, y en el primer experimento, con 5 % de pérdida, QoS 0 entregó el 100 % de las lecturas, mientras que QoS 1 solo agregó duplicados y bytes. Lo que aporta QoS 1 es una confirmación de la aplicación en cada tramo y el reenvío de los mensajes pendientes cuando la conexión se corta y la sesión se retoma, es decir, justamente lo que TCP no puede dar, porque su confiabilidad termina con la conexión. La elección del nivel depende, entonces, de qué significa que falte un mensaje. Para órdenes y eventos que no se repiten conviene QoS 1 con sesión persistente y un receptor que tolere duplicados. Para datos que se reemplazan continuamente alcanza QoS 0, eventualmente con un número de secuencia, y el segundo experimento mostró que, con pérdida, QoS 1 puede

ser incluso contraproducente ya que los datos llegaron con una antigüedad mediana de 13,4 s y el grupo de nodos dejó de sincronizarse.

En la capa de red, el modelo de MQTT se ajusta a las restricciones de las redes que el diseñador no controla. Como todos los participantes son clientes y es el dispositivo quien abre la conexión, el único requisito sobre la red del cliente es que permita las conexiones salientes hacia el broker. La contracara es que el NAT descarta las entradas inactivas, y el keep-alive, al impedir que la conexión quede inactiva, las mantiene vigentes. Respecto del ruteo, la convergencia de OSPF, RIP, EIGRP y BGP es invisible para TCP y para MQTT, y lo que le ocurre a la conexión depende de cómo se compara la duración del corte con el temporizador de retransmisión de TCP y con el umbral de una vez y media el keep-alive. Los cortes de pocos segundos, como los de un IGP con detección rápida o los de EIGRP con un sucesor factible, solo agregan demora. Los menores que ese umbral, como los de OSPF con sus temporizadores predeterminados, no cierran la conexión, pero entregan datos atrasados que pueden haber perdido su valor. Los mayores, como los de RIP o los de BGP al retirar una ruta, cierran la conexión, hacen que el broker publique la última voluntad y obligan a repetir los handshakes, y es en ese régimen donde los niveles de calidad de servicio y las sesiones persistentes cobran sentido. Un cambio de NAT, por último, corta la conexión aunque la red converja de inmediato. El segundo experimento confirmó este comportamiento con cortes de duración controlada. Ante un corte de 15 s, la última voluntad se publicó a los 6,9 s, cerca de los 7,5 s previstos.

El contraste entre el ruteo IP y el ruteo por tópicos mostró que el broker es, en la capa de aplicación, un mecanismo de encaminamiento análogo a un router. Ambos deciden a partir de una tabla y de una clave que viaja en el encabezado, y el comodín `#` cumple en una suscripción el papel de un prefijo en una tabla de ruteo. Las diferencias marcan los límites del broker. Como no interpreta la carga útil, no puede encaminar según el contenido, y por eso en Sparkplug no es posible suscribirse a una métrica individual. Tampoco reconoce las copias de un mismo mensaje, como las que publican varios puentes de Meshtastic, de modo que descartarlas queda a cargo de los suscriptores. Además, el ruteo por tópicos no reemplaza al ruteo IP, sino que se apoya en él y funciona como una red superpuesta cuyo único nodo intermedio es el broker, y cada conexión con él se encamina por IP de manera independiente. El nombre del tópico tiene, por último, un costo concreto, ya que en el segundo experimento ocupó 17 de los 99 bytes de cada entrega, casi tanto como el dato, y el alias de tópico redujo el tráfico total un 9,7 %.

Los casos de uso analizados, pese a pertenecer a dominios distintos, comparten el mismo patrón ya que el dispositivo, o el equipo que lo representa, abre una conexión persistente hacia el broker y recibe por ella también las órdenes. Usan Protocol Buffers en los tramos que concentran el volumen de mensajes y en los enlaces de radio más restringidos, y JSON en los datos que se comparten entre empresas y en los estándares entre fabricantes, como VDA 5050 o ISO 21423. Publican con QoS 0 los datos de alta frecuencia y reservan QoS 1 y los mensajes retenidos para el estado de conexión y los datos que cambian poco. Las limitaciones que exponen se concentran en el broker, que conserva estado y, cuando la implementación no permite replicarlo, como ocurre con Mosquitto, se convierte en el punto único de falla del sistema.

Las mediciones confirmaron lo anterior y le pusieron números. En el primer experimento, una lectura por MQTT QoS 0 con TLS ocupó 165 B, 3,4 veces menos que por HTTP con conexión reutilizada y entre 14 y 16 veces menos que con una conexión por lectura, con una mediana de latencia de unos 3 ms, frente a 45 ms y 3,4 s. De estos datos se desprende que, con TLS, mantener la conexión abierta explica el 82 % del ahorro, y el protocolo de aplicación, el 18 % restante. Protocol Buffers redujo la carga útil un 75 % respecto de JSON, unos 54 B por lectura

en todos los caminos, pero cambiar de HTTP con conexión reutilizada a MQTT QoS 0 ahorró más de siete veces eso. El experimento mostró también los límites de esa ventaja. MQTT tiene el costo fijo por conexión más alto, por lo que deja de convenir si la conexión dura menos de tres o cuatro lecturas, como en un dispositivo que duerme y se reconecta en cada envío. Además, parte de la diferencia de latencia con HTTP no se debe al protocolo, sino a cómo el cliente usa TCP, ya que con el algoritmo de Nagle desactivado HTTP con conexión reutilizada bajó de 44 a 5 ms, aunque siguió enviando tres veces más bytes que MQTT.

El segundo experimento mostró que el stack puede sostener una aplicación en la que cada nodo necesita el dato de todos los demás, con un comportamiento que coincide con el modelo teórico cuando este incluye la antigüedad de los datos. En ese régimen, el período de publicación, que es una decisión de la aplicación, pesó mucho más que el retardo de la red, con unos 250 ms frente a 5 ms. Las virtudes atribuidas al stack se observaron en la práctica, ya que un nodo nuevo se integró al grupo sin que los demás cambiaran su configuración, un campo desconocido no afectó a la sincronía y el grupo se recuperó solo después de cada interrupción. También quedaron a la vista sus límites. La caída del broker desordenó a todo el grupo, y la reconexión del cliente prolongó la interrupción unos 10 s más. Cruzar dos campos del esquema, a su vez, produjo mensajes que se decodifican sin error y un grupo que no se sincroniza, sin ninguna señal en la red, porque el esquema es un acuerdo que el protocolo no puede verificar.

El mapeo tecnológico permite cerrar con la viabilidad económica del sistema. El escenario del primer experimento, con ocho nodos que envían una lectura cada 10 s, puede implementarse con equipos de bajo costo y de amplia disponibilidad. Con conectividad Wi-Fi requiere unos USD 80 en placas con ESP32 y USD 5 por mes para un servidor propio con Mosquitto, y con conectividad celular, unos USD 250 de inversión inicial y USD 21 por mes, de los cuales 16 corresponden a los datos. La principal incertidumbre no es el precio, sino la cobertura del operador en el lugar de instalación. El análisis de escala muestra, además, que el ahorro de bytes del stack se traduce en dinero solo cuando la conexión se cobra por volumen y la frecuencia de envío es alta. En una flota de mil nodos celulares con una lectura cada 10 s, los datos rondan los USD 2000 por mes, y usar JSON en lugar de Protocol Buffers los aumentaría alrededor de un 40 %, mientras que usar HTTP en lugar de MQTT los multiplicaría casi por cuatro. En cambio, con un broker que cobra por mensaje, achicar los mensajes no reduce el costo, y en una red Wi-Fi propia, como la de un depósito con robots, la ventaja del stack se mide en latencia y en uso de la red más que en dinero.

Estas conclusiones tienen el alcance de las condiciones en que se obtuvieron. Los experimentos se realizaron sobre placas emuladas, con tiempos de criptografía y de procesamiento que no son los de un microcontrolador físico, sin radio real ni medición del consumo, y con pérdidas aleatorias e independientes en lugar de las ráfagas de un enlace real. La convergencia del ruteo se analizó en el desarrollo y se representó con cortes de duración controlada, pero no se reprodujo. Tampoco se evaluaron un broker en clúster ni la escala de miles de clientes, y el segundo experimento se apoya, salvo en el ensayo de QoS con pérdida, en una corrida por condición. Quedan como continuación natural repetir las mediciones con placas físicas sobre una red celular, evaluar la reanudación de sesión TLS y TLS 1.3 para los dispositivos que se reconectan en cada envío, y medir el comportamiento de un broker replicado.

En síntesis, MQTT y Protocol Buffers son una opción adecuada para el intercambio frecuente de datos sobre redes que no se controlan, siempre que el diseño respete tres condiciones. Hay que mantener la conexión abierta, con un keep-alive elegido en función de la red, elegir el nivel de calidad de servicio según lo que significa que falte un mensaje y tratar al esquema y al broker

como partes críticas del sistema, porque las fallas del primero no se detectan solas y las del segundo afectan a todos los participantes a la vez.

Bibliografía

[1] OpenRobOps/oro, «oro: OpenRobOps open-source robot fleet management and operations platform». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/OpenRobOps/oro

[2] D. E. Comer, *Internetworking with TCP/IP, Volume One: Principles, Protocols, and Architecture*, 6.ª ed. Upper Saddle River, NJ: Pearson, 2014.

[3] J. F. Kurose y K. W. Ross, *Redes de computadoras. Un enfoque descendente*, 7.ª ed. Madrid: Pearson Educación, 2017.

[4] P. T. Eugster, P. A. Felber, R. Guerraoui, y A.-M. Kermarrec, «The many faces of publish/subscribe», *ACM Computing Surveys*, vol. 35, n.º 2, pp. 114-131, 2003, doi: 10.1145/857076.857078.

[5] W. Stallings, *Comunicaciones y redes de computadores*, 7.ª ed. Madrid: Pearson Educación, 2004.

[6] A. Banks, E. Briggs, K. Borgendale, y R. Gupta, Eds., «MQTT version 5.0», OASIS, OASIS Standard, 2019. [En línea]. Disponible en: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[7] Google, «Overview». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/overview/

[8] protocolbuffers, «protobuf: Protocol Buffers - Google's data interchange format». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://github.com/protocolbuffers/protobuf

[9] Google, «Encoding». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/programming-guides/encoding/

[10] Google, «Language Guide (proto 3)». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/programming-guides/proto3/

[11] Google, «Techniques». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/programming-guides/techniques/

[12] Amazon Web Services, «Working with binary payloads». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://docs.aws.amazon.com/iot/latest/developerguide/binary-payloads.html

[13] Meshtastic, «MQTT: Integrations Overview». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://meshtastic.org/docs/software/integrations/mqtt/

[14] Eclipse Foundation, «The Eclipse Foundation Announces Sparkplug as an International Standard for a "Plug and Play" Industrial IoT». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://newsroom.eclipse.org/node/39754

[15] Eclipse Sparkplug Contributors, «Sparkplug 3.0.0: Sparkplug Specification», Specification, 2022. Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://sparkplug.eclipse.org/specification/version/3.0/documents/sparkplug-specification-3.0.0.pdf

[16] R. T. Fielding, M. Nottingham, y J. Reschke, Eds., «HTTP Semantics», RFC 9110, 2022. doi: 10.17487/RFC9110.

[17] S. Loreto, P. Saint-Andre, S. Salsano, y G. Wilkins, «Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP», RFC 6202, 2011. doi: 10.17487/RFC6202.

[18] Z. Shelby, K. Hartke, y C. Bormann, «The Constrained Application Protocol (CoAP)», RFC 7252, 2014. doi: 10.17487/RFC7252.

[19] K. Hartke, «Observing Resources in the Constrained Application Protocol (CoAP)», RFC 7641, 2015. doi: 10.17487/RFC7641.

[20] N. Naik, «Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP», en *2017 IEEE International Systems Engineering Symposium (ISSE)*, Vienna, Austria: IEEE, 2017. doi: 10.1109/SysEng.2017.8088251.

[21] D. Thangavel, X. Ma, A. Valera, H.-X. Tan, y C. K.-Y. Tan, «Performance evaluation of MQTT and CoAP via a common middleware», en *2014 IEEE Ninth International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP)*, Singapore: IEEE, 2014. doi: 10.1109/ISSNIP.2014.6827678.

[22] D. P. Proos y N. Carlsson, «Performance comparison of messaging protocols and serialization formats for digital twins in IoV», en *2020 IFIP Networking Conference (Networking)*, Paris, France: IFIP, 2020.

[23] C. Bormann, S. Lemay, H. Tschofenig, K. Hartke, B. Silverajan, y B. Raymor, «CoAP (Constrained Application Protocol) over TCP, TLS, and WebSockets», RFC 8323, 2018. doi: 10.17487/RFC8323.

[24] A. Stanford-Clark y H. L. Truong, «MQTT For Sensor Networks (MQTT-SN) Protocol Specification, Version 1.2», Especificación técnica de IBM, 2013.

[25] T. Bray, Ed., «The JavaScript Object Notation (JSON) Data Interchange Format», RFC 8259, 2017. doi: 10.17487/RFC8259.

[26] J. C. Viotti y M. Kinderkhedia, «A Benchmark of JSON-compatible Binary Serialization Specifications», *arXiv preprint arXiv:2201.03051*, 2022, doi: 10.48550/arXiv.2201.03051.

[27] D. Friesel y O. Spinczyk, «Data Serialization Formats for the Internet of Things», *Electronic Communications of the EASST*, vol. 80, 2021, doi: 10.14279/tuj.eceasst.80.1134.

[28] R. Braden, Ed., «Requirements for Internet Hosts – Communication Layers», RFC 1122, 1989. doi: 10.17487/RFC1122.

[29] J. Moy, «OSPF Version 2», RFC 2328, 1998. doi: 10.17487/RFC2328.

[30] ChirpStack, «ChirpStack Gateway Bridge: Configuration». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://www.chirpstack.io/docs/gateway-bridge/configuration.html

[31] Digitransit, «Digitransit MQTT». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://www.digitransit.fi/en/developers/apis/5-realtime-api/vehicle-positions/digitransit-mqtt

[32] T. Dierks y E. Rescorla, «The Transport Layer Security (TLS) Protocol Version 1.2», RFC 5246, 2008. doi: 10.17487/RFC5246.

[33] E. Rescorla, «The Transport Layer Security (TLS) Protocol Version 1.3», RFC 8446, 2018. doi: 10.17487/RFC8446.

[34] S. Perreault, I. Yamagata, S. Miyakawa, A. Nakagawa, y H. Ashida, «Common Requirements for Carrier-Grade NATs (CGNs)», RFC 6888, 2013. doi: 10.17487/RFC6888.

[35] S. Guha, K. Biswas, B. Ford, S. Sivakumar, y P. Srisuresh, «NAT Behavioral Requirements for TCP», RFC 5382, 2008. doi: 10.17487/RFC5382.

[36] D. Katz y D. Ward, «Bidirectional Forwarding Detection (BFD)», RFC 5880, 2010. doi: 10.17487/RFC5880.

[37] G. Malkin, «RIP Version 2», RFC 2453, 1998. doi: 10.17487/RFC2453.

[38] D. Savage, J. Ng, S. Moore, D. Slice, P. Paluch, y R. White, «Cisco's Enhanced Interior Gateway Routing Protocol (EIGRP)», RFC 7868, 2016. doi: 10.17487/RFC7868.

[39] Y. Rekhter, T. Li, y S. Hares, Eds., «A Border Gateway Protocol 4 (BGP-4)», RFC 4271, 2006. doi: 10.17487/RFC4271.

[40] C. Labovitz, A. Ahuja, A. Bose, y F. Jahanian, «Delayed Internet Routing Convergence», en *Proceedings of the Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '00)*, Stockholm, Sweden: ACM, 2000.

[41] M. Shand y S. Bryant, «A Framework for Loop-Free Convergence», RFC 5715, 2010. doi: 10.17487/RFC5715.

[42] V. Paxson, M. Allman, J. Chu, y M. Sargent, «Computing TCP's Retransmission Timer», RFC 6298, 2011. doi: 10.17487/RFC6298.

[43] The Linux Kernel Developers, «IP Sysctl». Accedido: 6 de noviembre de 2026. [En línea]. Disponible en: https://docs.kernel.org/networking/ip-sysctl.html

[44] V. Fuller y T. Li, «Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan», RFC 4632, 2006. doi: 10.17487/RFC4632.

[45] Eclipse Mosquitto, «mosquitto.conf(5): the configuration file for mosquitto». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://mosquitto.org/man/mosquitto-conf-5.html

[46] EMQ Technologies, «EMQX Clustering». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://docs.emqx.com/en/emqx/latest/deploy/cluster/introduction.html

[47] EMQ Technologies, «Wildcard Subscription». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://docs.emqx.com/en/emqx/latest/messaging/mqtt-wildcard-subscription.html

[48] L. Zhang, «Building Facebook Messenger». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://engineering.fb.com/2011/08/12/android/building-facebook-messenger/

[49] Apache Software Foundation, «About Apache Thrift». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://thrift.apache.org/about

[50] J. Fein, «Building Mobile-First Infrastructure for Messenger». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://engineering.fb.com/2014/10/09/production-engineering/building-mobile-first-infrastructure-for-messenger/

[51] J. Barber *et al.*, «Bladerunner: Stream Processing at Scale for a Live View of Backend Data Mutations at the Edge», en *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*, ACM, 2021, pp. 708-723. doi: 10.1145/3477132.3483572.

[52] inorbit-ai, «edge-sdk-python: InOrbit Python Edge SDK». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/inorbit-ai/edge-sdk-python

[53] InOrbit, «Interoperability Standards (InOrbit Docs)». Accedido: 1 de noviembre de 2026. [En línea]. Disponible en: https://github.com/inorbit-ai/docs/blob/main/ground-control/robot-integration/interoperability.mdx

[54] VDA y VDMA, «VDA 5050: Interface for the Communication between Mobile Robots and a Fleet Control». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/VDA5050/VDA5050/blob/3.0.0/VDA5050_EN.md

[55] ISO, «Robotics – Industrial mobile robots – Communications and interoperability», International Standard ISO 21423:2026, oct. 2026. Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://www.iso.org/standard/86749.html

[56] Meshtastic, «Mesh Broadcast Algorithm». Accedido: 1 de noviembre de 2026. [En línea]. Disponible en: https://meshtastic.org/docs/overview/mesh-algo/

[57] Meshtastic, «Meshtastic Protobuf Definitions». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/meshtastic/protobufs

[58] Meshtastic, «MQTT Module Configuration». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://meshtastic.org/docs/configuration/module/mqtt/

[59] LoRa Alliance, «What is LoRaWAN». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://lora-alliance.org/about-lorawan/

[60] Semtech, «Basic communication protocol between Lora gateway and server». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/Lora-net/packet_forwarder/blob/master/PROTOCOL.TXT

[61] ChirpStack, «ChirpStack Gateway Bridge: Events». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://www.chirpstack.io/docs/chirpstack-gateway-bridge/payloads/events.html

[62] ChirpStack, «ChirpStack Gateway Bridge: Semtech UDP packet-forwarder». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://www.chirpstack.io/docs/chirpstack-gateway-bridge/backends/semtech-udp.html

[63] ChirpStack, «gw.proto». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/chirpstack/chirpstack/blob/master/api/proto/gw/gw.proto

[64] ChirpStack, «backend.go (ChirpStack Gateway Bridge, integración MQTT)». Accedido: 9 de noviembre de 2026. [En línea]. Disponible en: https://github.com/chirpstack/chirpstack-gateway-bridge/blob/a5ae780f1318ca9678c9e781bd9125340a314406/internal/integration/mqtt/backend.go

[65] ChirpStack, «chirpstack: ChirpStack open-source LoRaWAN Network Server». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/chirpstack/chirpstack

[66] ChirpStack, «config.rs (ChirpStack, configuración por defecto)». Accedido: 9 de noviembre de 2026. [En línea]. Disponible en: https://github.com/chirpstack/chirpstack/blob/62b1eb9975ee390798d93c5b51e96bac2272ac56/chirpstack/src/config.rs

[67] Espressif Systems, «QEMU Emulator». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/tools/qemu.html

[68] Espressif Systems, «Get Started». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/index.html

[69] The Linux Kernel, «TUN/TAP device driver». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://docs.kernel.org/networking/tuntap.html

[70] The Linux Kernel, «Ethernet Bridging». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://docs.kernel.org/networking/bridge.html

[71] W. M. Eddy, Ed., «Transmission Control Protocol (TCP)», RFC 9293, 2022. doi: 10.17487/RFC9293.

[72] D. Mills, J. Martin, J. Burbank, y W. Kasch, «Network Time Protocol Version 4: Protocol and Algorithms Specification», RFC 5905, 2010. doi: 10.17487/RFC5905.

[73] Espressif Systems, «ESP-MQTT». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/mqtt.html

[74] A. Banks y R. Gupta, Eds., «MQTT Version 3.1.1», OASIS, OASIS Standard, 2014. [En línea]. Disponible en: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[75] S. H. Strogatz, «From Kuramoto to Crawford: exploring the onset of synchronization in populations of coupled oscillators», *Physica D: Nonlinear Phenomena*, vol. 143, n.º 1–4, pp. 1-20, 2000, doi: 10.1016/S0167-2789(00)00094-4.

[76] Y. Kuramoto, «Self-entrainment of a population of coupled non-linear oscillators», en *International Symposium on Mathematical Problems in Theoretical Physics*, en Lecture Notes in Physics, vol. 39. Springer, 1975, pp. 420-422. doi: 10.1007/BFb0013365.

[77] M. Verwoerd y O. Mason, «Global phase-locking in finite populations of phase-coupled oscillators», *SIAM Journal on Applied Dynamical Systems*, vol. 7, n.º 1, pp. 134-160, 2008, doi: 10.1137/070686858.

[78] M. K. S. Yeung y S. H. Strogatz, «Time delay in the Kuramoto model of coupled oscillators», *Physical Review Letters*, vol. 82, n.º 3, pp. 648-651, 1999, doi: 10.1103/PhysRevLett.82.648.

[79] P. Aimonen, «Nanopb: protocol buffers with small code size». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://jpa.kapsi.fi/nanopb/

[80] Linux man-pages, «tc-netem(8)». Accedido: 8 de noviembre de 2026. [En línea]. Disponible en: https://man7.org/linux/man-pages/man8/tc-netem.8.html

[81] Espressif Systems, «ESP32-DevKitC-32E». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://www.mouser.com/ProductDetail/Espressif-Systems/ESP32-DevKitC-32E

[82] LilyGO, «T-A7670SA». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://wiki.lilygo.cc/products/t-sim-series/t-a7670/t-a7670sa/

[83] Hologram, «Pricing». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://www.hologram.io/pricing/

[84] Amazon Web Services, «Amazon Lightsail Pricing». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://aws.amazon.com/lightsail/pricing/

[85] Amazon Web Services, «AWS IoT Core Pricing». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://aws.amazon.com/iot-core/pricing/